Q1. **CLEARAUTH AI - PRIOR AUTHORISATION SCREENING AND REVIEW PLATFORM**

PROBLEM STATEMENT

Brightwater Health Plan asks for prior authorisation before its members receive five services: **MRI of the lumbar spine**, **MRI of the cervical spine**, **total knee replacement**, **bariatric surgery** and the **in-laboratory sleep study**. Requesting offices send each request by provider portal, fax or EDI feed, with a short questionnaire for its service and the clinical documents the office attached. Each service has its own medical policy of exact criteria, and every request is decided by a UM Nurse Reviewer.

The utilisation management team has three problems. Every channel writes the questionnaire its own way: heights in feet and inches, weights in stones and pounds, treatment in days, member numbers in three notations, and a criterion compared with an answer that was never converted gives the wrong result. The five medical policies share one template, and the lumbar and cervical spine policies word their criteria alike but differ in a figure and in their red flags, so a criterion read from the wrong document passes and fails the wrong requests. And the documents attached to a request sometimes say something other than its questionnaire, which a reviewer working through the queue can miss.

Provider Services has a problem of its own. Its representatives answer requesting offices all day, and they may confirm a request only to a caller who gives both its reference and the member number on it. They may never predict whether a request will be approved.

Brightwater requires a system that screens every open request against its own medical policy, reads every sentence of the documents sent with it, estimates how likely approval is and how long an inpatient stay will be, prepares a review with a recommended decision, and then stops. Nothing is decided, written or sent on the word of a model.

OBJECTIVE

Build **ClearAuth AI**, a working prior authorisation screening and review platform that a utilisation management team can use end to end. The finished system must:

- Clean five raw extracts that arrive with real data-quality defects, and produce the evidence that each one needed cleaning.
- Read every service's criteria, with their own figures, out of that service's own medical policy. No criterion is written into the code or the configuration.
- Read every questionnaire answer as the Prior Authorisation Rules read it, and screen each request: every criterion **Met**, **Not met** or **Not documented**, the screening, and the decisions it leaves open.
- Train two machine learning models on tabular data, one estimating the approval likelihood of a request and one estimating the expected inpatient stay, and one NLP model that names the kind of evidence a sentence of a clinical document gives.
- Build a searchable knowledge base from sixteen plan documents, so that every statement the system makes can be traced to the document it came from.
- Read every sentence of the documents sent with a request, record the criteria each one bears on, and flag every sentence whose weeks of treatment disagree with the questionnaire.
- Run a multi-agent workflow in which each agent is a single language model call that interprets facts the tools established, and calculates nothing of its own.
- Stop that workflow before anything is decided, wait for a UM Nurse Reviewer who did not open the review, and survive the machine being switched off while it waits.
- After the decision, write exactly what that decision calls for: a provider authorisation letter and a member notice, an information request letter, or a referral summary.
- Record every run, every review, every decision and every language model call in a database that can be audited afterwards.
- Serve all of it through a REST service and a dashboard, with every figure the dashboard shows behind that service.
- Require everyone to sign in, and show each of the three roles only the pages their role covers, with the service refusing the matching request on its own account.
- Answer questions from the plan's documents and from nothing else, answer every question about a request from the review records by the asker's role, keep member numbers away from the language model, and never predict an outcome to a Provider Services Representative.

PROJECT STRUCTURE

```
Project/
|-- .env                              <- pre-loaded, your key goes in it
|-- .streamlit/                       <- pre-loaded
|   '-- config.toml
|-- access/
|   |-- password_security.py
|   '-- permissions.py
|-- agentic/
|   |-- agents.py
|   |-- graph.py
|   |-- nodes.py
|   '-- state.py
|-- api/
|   |-- routes.py
|   '-- schemas.py
|-- chat/
|   |-- authorisation_assistant.py
|   |-- conversation_memory.py
|   '-- request_status_answer.py
|-- config/
|   '-- config.py                     <- pre-loaded, complete, do not edit
|-- dashboard/
|   |-- api_client.py
|   |-- audit_trail.py
|   |-- authorisation_chat.py
|   |-- decision_queue.py
|   |-- request_detail.py
|   |-- review_queue.py
|   '-- sign_in.py
|-- data/                             <- pre-loaded
|   |-- docs/                         <- the sixteen PDF documents
|   |-- input/
|   |   |-- open_requests.csv
|   |   '-- request_notes.csv
|   |-- authorisation_history_eval.csv
|   |-- authorisation_history_train.csv
|   |-- clinical_note_sentences.csv
|   |-- surgical_stays_eval.csv
|   '-- surgical_stays_train.csv
|-- data_pipeline/
|   |-- common_cleaning.py
|   |-- dataset_loader.py
|   |-- feature_builder.py
|   |-- history_cleaning.py
|   |-- note_cleaning.py
|   |-- quality_profiler.py
|   |-- request_cleaning.py
|   |-- sentence_cleaning.py
|   '-- stay_cleaning.py
|-- database/                         <- empty
|   |-- clearauth.db                  <- created by run_database_setup.py
|   |-- checkpoint.db                 <- created by run_api_service.py
|   '-- chroma_store/                 <- created by run_knowledge_indexing.py
|-- knowledge/
|   |-- chunk_splitting.py
|   |-- pdf_extraction.py
|   |-- policy_reading.py
|   '-- vector_index.py
|-- modeling/
|   |-- approval_model.py
|   |-- evidence_model.py
|   |-- model_evaluation.py
|   |-- model_registry.py
|   '-- stay_model.py
|-- output/                           <- empty
|   |-- models/                       <- empty; filled by run_model_training.py
|   |-- data_quality_report.json      <- created by run_data_pipeline.py
|   '-- model_metrics.json            <- created by run_model_training.py
|-- persistence/
|   |-- database.py
|   |-- models.py
|   '-- repository.py
|-- processed/                        <- empty; filled by run_data_pipeline.py
|-- tools/
|   |-- decision_tools.py
|   |-- evidence_tools.py
|   |-- letter_tools.py
|   |-- prediction_tools.py
|   |-- request_data_tools.py
|   |-- retrieval_tools.py
|   '-- screening_tools.py
|-- requirements.txt                  <- pre-loaded
|-- run_api_service.py
|-- run_dashboard.py
|-- run_data_pipeline.py
|-- run_database_setup.py
|-- run_knowledge_indexing.py
'-- run_model_training.py
```

DATASET DESCRIPTION

authorisation_history_train.csv —

Column	Type	What it holds
request_reference	Text	The request, **PA-** and six digits. **config.REQUEST_REFERENCE_COLUMN** names it
submission_sequence	Number	Which submission of the request the row is. A provider may submit again before review begins, and the latest submission is the request. **config.SUBMISSION_SEQUENCE_COLUMN** names it
received_channel	Category	Portal, Fax or Electronic, in several spellings
urgency	Category	Standard or Urgent, in several spellings. **config.URGENCY_COLUMN** names it
member_number	Text	The member, **BWH-** and seven digits once read; each channel writes it its own way. **config.MEMBER_NUMBER_COLUMN** names it
member_age	Number	Age in years at submission. **config.MEMBER_AGE_COLUMN** names it
member_sex	Category	Recorded for fairness checks only
member_region	Category	Recorded for fairness checks only
plan_code	Category	The member's plan
provider_id	Text	The requesting practice
provider_name	Text	The practice's name, with runs of spaces on some rows
provider_specialty	Category	The practice's specialty
provider_network_status	Category	In network or Out of network, in several spellings
provider_prior_approval_rate_pct	Number	The share of the practice's requests approved over the previous year, with a percent sign on some rows
service_code	Category	One of the five service codes, in lower case or with a space on some rows. **config.SERVICE_CODE_COLUMN** names it
service_name	Category	The service's name
diagnosis_code	Text	The ICD-10-CM diagnosis; the EDI feed writes it without its dot. **config.DIAGNOSIS_CODE_COLUMN** names it
site_of_service	Category	Where the service would be given. **config.SITE_OF_SERVICE_COLUMN** names it
procedure_detail	Category	One knee or both, or the bariatric operation; blank for the other services. **config.PROCEDURE_DETAIL_COLUMN** names it
planned_facility_name	Text	The planned hospital or surgery centre; surgical services only. **config.PLANNED_FACILITY_COLUMN** names it
enhanced_recovery_programme	Boolean	The planned hospital runs an enhanced recovery programme; inpatient surgery only
surgery_weekday	Category	The weekday the operation is planned for, in several spellings
requested_inpatient_days	Number	Days in hospital the surgeon asks the plan to authorise, with a unit on some rows. **config.REQUESTED_INPATIENT_DAYS_COLUMN** names it
asa_physical_status	Number	The ASA physical status class, written as a number, a Roman numeral or with **ASA** before it. **config.ASA_CLASS_COLUMN** names it
smoking_status	Category	Never, Former or Current, in several spellings
supporting_document_count	Number	The clinical documents attached. **config.SUPPORTING_DOCUMENT_COUNT_COLUMN** names it
conservative_treatment_weeks	Number	Conservative treatment, written in weeks, in weeks with a unit, or in days. **config.TREATMENT_WEEKS_COLUMN** names it
red_flag_symptom	Category	The red flag reported, or that there is none, in several spellings. **config.RED_FLAG_COLUMN** names it
previous_scan_of_region	Boolean	The same region has been scanned before. **config.PREVIOUS_SCAN_COLUMN** names it
months_since_last_scan	Number	Whole months since that scan, with a unit on some rows. **config.MONTHS_SINCE_SCAN_COLUMN** names it
new_neurological_symptoms	Boolean	New neurological symptoms since that scan. **config.NEW_NEUROLOGICAL_SYMPTOMS_COLUMN** names it
xray_grade	Number	The Kellgren-Lawrence grade of the knee, **0** to **4**, with its scale name on some rows. **config.XRAY_GRADE_COLUMN** names it
knee_function_score	Number	The Oxford Knee Score, **0** to **48**, written over its scale top on some rows. **config.KNEE_FUNCTION_SCORE_COLUMN** names it
height_cm	Number	Height, in centimetres once read; written in centimetres, metres or feet and inches. **config.HEIGHT_COLUMN** names it
weight_kg	Number	Weight, in kilograms once read; written in kilograms, pounds or stones and pounds. **config.WEIGHT_COLUMN** names it
other_conditions	Text	The member's long-term conditions, written freely, or that there are none. **config.OTHER_CONDITIONS_COLUMN** names it
weight_programme_months	Number	Whole months in a supervised weight-management programme, with a unit on some rows. **config.PROGRAMME_MONTHS_COLUMN** names it
psychological_assessment	Category	The pre-operative psychological assessment, Completed or Not completed, in several spellings. **config.PSYCHOLOGICAL_ASSESSMENT_COLUMN** names it
sleepiness_score	Number	The Epworth Sleepiness Scale score, **0** to **24**, written over its scale top on some rows. **config.SLEEPINESS_SCORE_COLUMN** names it
home_sleep_test_result	Category	The result of a home sleep apnoea test, in several spellings. **config.HOME_SLEEP_TEST_COLUMN** names it
information_requested	Boolean	More information was requested during review. Known only after submission
reviewer_level	Category	Who decided, a nurse or a physician. Known only after review
decision_turnaround_days	Number	Days from submission to decision, with a unit on some rows. Known only after the decision
approval_decision	Category	**The answer.** Approved or Denied, in several spellings. **config.HISTORY_TARGET_COLUMN** names it

authorisation_history_train.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
request_reference	PA-424135	PA-455853	PA-415573
submission_sequence	1	1	1
received_channel	Fax	Fax	Electronic
urgency	Standard	Standard	Standard
member_number	8276601	7822890	BWH3060498
member_age	56	63	55
member_sex	Female	Female	Male
member_region	Northdale	Northdale	Central
plan_code	BW-FAM	BW-FAM	BW-ESS
provider_id	PRV-33508	PRV-32914	PRV-31427
provider_name	Brookvale Neurology Clinic	Southvale Mill Lane Orthopaedic Clinic	Willowmere Medical Centre
provider_specialty	Neurology	Orthopaedic surgery	Family medicine
provider_network_status	In network	Out of network	In network
provider_prior_approval_rate_pct	83.9%	88 %	79.3 %
service_code	RAD-LSM	ORT-TKR	SLP-PSG
service_name	MRI of the lumbar spine	Total knee replacement	In-laboratory sleep study
diagnosis_code	M54.16	M17.0	G4730
site_of_service	Independent imaging centre	Hospital inpatient	Sleep laboratory
procedure_detail		Unilateral	
planned_facility_name		Fernlea Royal Infirmary	
enhanced_recovery_programme		No	
surgery_weekday		Monday	
requested_inpatient_days		3 days	
asa_physical_status		ASA III	
smoking_status		Never	
supporting_document_count	0	2	4
conservative_treatment_weeks	50 days	31 wks	
red_flag_symptom	No red flag		
previous_scan_of_region	NO		
months_since_last_scan			
new_neurological_symptoms			
xray_grade		KL 4	
knee_function_score		13/48	Not recorded
height_cm		5 ft 6 in	
weight_kg		13 st 8 lb	
other_conditions		asthma	Type 2 diabetes, osteoarthritis
weight_programme_months			
psychological_assessment			
sleepiness_score			16/24
home_sleep_test_result			Not done
information_requested	No	No	No
reviewer_level	Nurse	Nurse	Physician
decision_turnaround_days	2	3	3
approval_decision	Approved	Approved	Denied

authorisation_history_eval.csv —
The same columns, for the requests that came after the training ones. The approval model is scored on this file and never fitted on it.

surgical_stays_train.csv —
One row per completed inpatient stay for total knee replacement or bariatric surgery. This is what the stay model learns from.

Column	Type	What it holds
admission_reference	Text	The admission, **ADM-** and seven digits. No two different rows share one. **config.ADMISSION_REFERENCE_COLUMN** names it
member_number	Text	The member, **BWH-** and seven digits once read, written in other notations on some rows
member_age	Number	Age in years on admission
member_sex	Category	Recorded for fairness checks only
member_region	Category	Recorded for fairness checks only
plan_code	Category	The member's plan
hospital_id	Text	The hospital
hospital_name	Text	The hospital's name, padded on some rows
service_code	Category	**ORT-TKR** or **BAR-WLS**
procedure_detail	Category	The operation, in several spellings
asa_physical_status	Number	The ASA physical status class, written several ways
body_mass_index	Number	Body mass index as the hospital recorded it, with **kg/m2** on some rows. **config.BODY_MASS_INDEX_COLUMN** names it
diabetes_recorded	Boolean	Type 2 diabetes is recorded. **config.DIABETES_RECORDED_COLUMN** names it
smoking_status	Category	Smoking status at pre-assessment
comorbidity_count	Number	How many of the plan's twelve long-term conditions are recorded. **config.COMORBIDITY_COUNT_COLUMN** names it
enhanced_recovery_programme	Boolean	The hospital runs an enhanced recovery programme
surgery_weekday	Category	The weekday of the operation, in several spellings
complication_recorded	Boolean	A complication was recorded during the stay. Known only after the stay
discharge_destination	Category	Where the member went on discharge. Known only after the stay
readmission_recorded	Boolean	Readmitted within the following weeks. Known only after the stay
length_of_stay_days	Number	**The answer.** Nights in hospital, with a unit on some rows. **config.STAY_TARGET_COLUMN** names it

surgical_stays_train.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
admission_reference	ADM-2252844	ADM-2269726	ADM-2264917
member_number	BWH8272849	bwh-2797357	BWH-5975925
member_age	71	72	72
member_sex	Female	Female	Female
member_region	Southvale	Central	Eastmoor
plan_code	BW-SEN	BW-SEN	BW-SEN
hospital_id	HSP-531	HSP-475	HSP-510
hospital_name	Southvale District Hospital	Brookvale University Hospital	Larchmont Community Hospital
service_code	ORT-TKR	BAR-WLS	ORT-TKR
procedure_detail	Unilateral	Gastric bypass	Unilateral
asa_physical_status	ASA 2	ASA IV	3
body_mass_index	34.1 kg/m2	46.3	32.1 kg/m2
diabetes_recorded	Yes	True	No
smoking_status	Current	Never	Former
comorbidity_count	2	5	1
enhanced_recovery_programme	Yes	No	n
surgery_weekday	Fri	Wednesday	FRIDAY
complication_recorded	Yes	No	No
discharge_destination	Home with support	Home	Home
readmission_recorded	false	No	No
length_of_stay_days	4 nights	7 nights	5

Again as they arrive: member numbers in three notations, ASA classes written three ways, a body mass index carrying its unit, yes and no written many ways, a weekday shortened and a weekday shouted, a padded hospital name, and stays written in nights.

surgical_stays_eval.csv —
The same columns, for the stays that came after the training ones. The stay model is scored on this file and never fitted on it.

clinical_note_sentences.csv —
One row per sentence of the documents attached to older requests. This is what the evidence model learns from.

Column	Type	What it holds
sentence_id	Text	The sentence, **SEN-** and seven digits. No two different rows share one. **config.SENTENCE_ID_COLUMN** names it
request_reference	Text	The older request the document was attached to
service_code	Category	That request's service, in lower case or with a space on some rows
source_document_type	Category	The kind of document the sentence comes from, in another case on some rows
sentence_number	Number	The sentence's place in its document
sentence_text	Free text	The sentence, with markup, escaped characters, no-break spaces, padding or capitals on some rows. **config.SENTENCE_TEXT_COLUMN** names it
sentence_length_characters	Number	Characters in the sentence, measured on the raw text on some rows. **config.SENTENCE_LENGTH_COLUMN** names it
evidence_type	Category	**The answer.** The kind of evidence the sentence gives a reviewer, one of nine, in several spellings. **config.SENTENCE_TARGET_COLUMN** names it

clinical_note_sentences.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
sentence_id	SEN-5138034	SEN-5148032	SEN-5130802
request_reference	PA-308583	PA-310798	PA-306923
service_code	ORT-TKR	SLP-PSG	SLP-PSG
source_document_type	Physiotherapy report	Sleep clinic letter	Specialist letter
sentence_number	2	6	2
sentence_text	She cannot climb stairs without a rail.	In addition, a home sleep apnoea test has not been performed.	THERE ARE NO FEATURES TO SUGGEST A NEUROMUSCULAR CAUSE.
sentence_length_characters	52	66	55
evidence_type	Functional limitation	Prior investigation	Ruled out

The sentences arrive wrapped in markup, with escaped characters, written in capitals, and with lengths measured before any of that was repaired.

open_requests.csv —
The live queue, held under **data/input/** and named by **config.OPEN_REQUESTS_FILE**. It carries the authorisation history's first **40** columns, from **request_reference** to **home_sleep_test_result**, in the same order and the same notations, including earlier submissions and repeated rows. It carries **no review columns and no answer column**: deciding the request is what the system prepares for. Read through the same cleaning as the history, it holds **40** requests.

request_notes.csv —
The documents sent with the open requests, held under **data/input/** and named by **config.REQUEST_NOTES_FILE**, one row per document.

Column	Type	What it holds
note_id	Text	The document's identifier. Every document has its own. **config.NOTE_ID_COLUMN** names it
request_reference	Text	The open request it was sent with
note_number	Number	The document's place among its request's documents, as the office numbered them. **config.NOTE_NUMBER_COLUMN** names it
note_author_role	Category	The kind of document, in another case on some rows. **config.NOTE_AUTHOR_ROLE_COLUMN** names it
note_text	Free text	The document's text, its sentences in reading order, with markup or escaped characters on some rows. **config.NOTE_TEXT_COLUMN** names it

A request has as many documents as its **supporting_document_count** says, and a request with none has no rows. Once the repeated rows are removed the file holds **102** documents.

open_requests.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
request_reference	PA-561204	PA-561426	PA-561648
submission_sequence	1	1	1
received_channel	Fax	Portal	Portal
urgency	Standard	Standard	Standard
member_number	2525994	BWH-2631675	BWH-9252915
member_age	46	67	63
member_sex	Female	Female	Male
member_region	Central	Westbridge	Westbridge
plan_code	BW-SEL	BW-SEN	BW-FAM
provider_id	PRV-34034	PRV-33100	PRV-34811
provider_name	Highgate Rehabilitation Centre	Hillcrest Bone and Joint Centre	Southvale Parkside Sleep Disorders Clinic
provider_specialty	Physical medicine and rehabilitation	Orthopaedic surgery	Sleep medicine
provider_network_status	In network	In network	In network
provider_prior_approval_rate_pct	79.5%	86.1 %	74.4
service_code	RAD-LSM	ORT-TKR	SLP-PSG
service_name	MRI of the lumbar spine	Total knee replacement	In-laboratory sleep study
diagnosis_code	M51.26	M17.0	G47.33
site_of_service	Independent imaging centre	Hospital inpatient	Sleep laboratory
procedure_detail		Unilateral	
planned_facility_name		Westbridge University Hospital	
enhanced_recovery_programme		No	
surgery_weekday		Tuesday	
requested_inpatient_days		3 days	
asa_physical_status		ASA II	
smoking_status		Never	
supporting_document_count	2	3	2
conservative_treatment_weeks	35 days	16 weeks	
red_flag_symptom	Nil		
previous_scan_of_region	No		
months_since_last_scan			
new_neurological_symptoms			
xray_grade		KL grade 4	
knee_function_score		19/48	
height_cm		5 ft 4 in	
weight_kg		180 lb	
other_conditions		Hypertension	COPD, high blood pressure
weight_programme_months			
psychological_assessment			
sleepiness_score			14
home_sleep_test_result			Not performed

request_notes.csv: SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
note_id	NTE-880896	NTE-880800	NTE-880525
request_reference	PA-561204	PA-561426	PA-561648
note_number	1	2	2
note_author_role	Family physician letter	PHYSIOTHERAPY REPORT	Family physician letter
note_text	I saw this 46-year-old woman in clinic today. The back ...	Mrs Fraser has had 16 weeks of physiotherapy for the knee.	The Epworth Sleepiness Scale score is 14. He has had ne...

The live files arrive as the channels wrote them: a member number as seven digits, a red flag written **Nil**, treatment in days, a height in feet and inches and a weight in pounds, conditions written as an abbreviation and in words, a home test written **Not performed**, and a document type shouted.

THE DEFECTS IN THE RAW EXTRACTS

Every defect below is present in the extracts or the live files. Where **config/config.py** holds what is needed to repair one, the constants are named; a dash means the repair needs no lookup. Nothing downstream may assume a clean table.

Defect	How it looks	Where the repair values live
Padding and runs of spaces	**" Highgate Rehabilitation Centre"**, **" Larchmont Community Hospital "**	**config.REQUEST_NAME_COLUMNS**, **config.STAY_NAME_COLUMNS**, **config.WHITESPACE_RUN_PATTERN**
Missing written many ways	**"N/A"**, **"NA"**, **"-"**, **"unknown"**, **"Not recorded"**, blank	**config.NULL_TOKENS**
Duplicated rows	The same row twice, identical in every column	—
A request submitted again	An earlier submission with the same request reference and a lower **submission_sequence**	**config.SUBMISSION_SEQUENCE_COLUMN**
Spelling variants	**"EDI"**, **"Facsimile"**, **"Routine"**, **"Expedited"**, **"RYGB"**, **"Fri"**, **"Bladder or bowel dysfunction"**, **"Abnormal"**, **"Pending"**	**config.HISTORY_VALUE_MAPPINGS**, **config.REQUEST_VALUE_MAPPINGS**, **config.STAY_VALUE_MAPPINGS**, **config.SENTENCE_VALUE_MAPPINGS**, **config.NOTE_VALUE_MAPPINGS**
Yes and No written many ways	**"Y"**, **"True"**, **"NO"**, **"n"**, **"false"**	**config.BOOLEAN_MAPPING**, **config.BOOLEAN_YES**, **config.BOOLEAN_NO**
Member numbers and diagnosis codes in a channel's notation	**"8276601"**, **"BWH3060498"**, **"bwh-2797357"**; **"G4730"**	**config.MEMBER_NUMBER_PATTERN**, **config.DIAGNOSIS_CODE_PATTERN**, **config.CANONICAL_MEMBER_NUMBER_FORMAT**, **config.CANONICAL_DIAGNOSIS_CODE_FORMAT**
Numbers written with units	**"3 days"**, **"4 nights"**, **"14 mo"**, **"83.9%"**, **"88 %"**, **"34.1 kg/m2"**	**config.HISTORY_WHOLE_NUMBER_PATTERNS**, **config.REQUEST_DECIMAL_NUMBER_PATTERNS**, **config.STAY_WHOLE_NUMBER_PATTERNS**, **config.STAY_DECIMAL_NUMBER_PATTERNS**
Grades and scores with their scale	**"KL grade 3"**, **"Grade 0"**, **"13/48"**, **"16/24"**	**config.XRAY_GRADE_PATTERN**, **config.KNEE_FUNCTION_SCORE_PATTERN**, **config.SLEEPINESS_SCORE_PATTERN**
Treatment in days or with a unit	**"50 days"**, **"31 wks"**, **"16 weeks"**	**config.TREATMENT_DURATION_PATTERN**, **config.DAYS_PER_WEEK**
ASA class written three ways	**"ASA III"**, **"IV"**, **"ASA 2"**	**config.ASA_CLASS_PATTERN**, **config.ASA_CLASS_NUMBERS**
Heights in three notations	**"5 ft 6 in"**, **"1.72 m"**, **"172 cm"**	**config.HEIGHT_NOTATIONS**, **config.CENTIMETRES_PER_HEIGHT_UNIT**, **config.MEASUREMENT_STEP**
Weights in three notations	**"13 st 8 lb"**, **"180 lb"**, **"84.3 kg"**	**config.WEIGHT_NOTATIONS**, **config.KILOGRAMS_PER_WEIGHT_UNIT**, **config.MEASUREMENT_STEP**
Conditions written freely	**"Asthma, HTN"**, **"COPD, high blood pressure"**, **"none"**	**config.CONDITION_MAPPING**, **config.NO_OTHER_CONDITIONS_SPELLINGS**, **config.CONDITION_LIST_SEPARATOR_PATTERN**, **config.CONDITION_SEPARATOR**, **config.NO_OTHER_CONDITIONS**
Impossible readings	A grade of **5**, a knee score of **50**, a sleepiness score of **25**, a rate of **170.0**, a turnaround of **-2**, a body mass index of **207** or **2.3**	**config.REQUEST_MAXIMUM_VALUES**, **config.HISTORY_MINIMUM_VALUES**, **config.STAY_MINIMUM_VALUES**, **config.STAY_MAXIMUM_VALUES**
An impossible answer	A stay of **0** nights	**config.STAY_MINIMUM_VALUES**
Missing answer column	A row with no value in the column the model must learn	—
Markup in free text	HTML tags such as ****, **<p>** and ** ** around or inside a sentence	**config.MARKUP_PATTERN**
Escaped characters and no-break spaces	An HTML character reference, named such as ** ** or numeric such as **-**, where a plain character belongs	**config.ENCODING_ARTIFACT_PATTERN**, **config.WHITESPACE_RUN_PATTERN**
Sentences written in capitals	A whole sentence shouted, which the evidence model must not learn as words of its own	—
Sentences too short to learn from	A sentence of one or two words	**config.MINIMUM_SENTENCE_WORDS**
A stale derived column	**sentence_length_characters** measured before the text was repaired	**config.SENTENCE_LENGTH_COLUMN**
Document types in another case	**"PHYSIOTHERAPY REPORT"**, **"specialist letter"**	**config.DOCUMENT_TYPE_MAPPING**

THE SIXTEEN DOCUMENTS

The documents sit under **data/docs/** as PDFs. They are Brightwater's own, and they are the only source of any criterion, rule, check or answer the system gives.

Document	What it settles
Medical-Policy-Lumbar-Spine-MRI.pdf	The criteria for an MRI of the lumbar spine (**RAD-LSM**), the red flags that waive its treatment length, and the documentation to send
Medical-Policy-Cervical-Spine-MRI.pdf	The same for an MRI of the cervical spine (**RAD-CSM**), with its own treatment length and red flags
Medical-Policy-Total-Knee-Replacement.pdf	The criteria for a total knee replacement (**ORT-TKR**) and the documentation to send
Medical-Policy-Bariatric-Surgery.pdf	The criteria for bariatric surgery (**BAR-WLS**), its listed conditions, and the documentation to send
Medical-Policy-In-Laboratory-Sleep-Study.pdf	The criteria for the in-laboratory sleep study (**SLP-PSG**), its listed conditions, and the documentation to send
Utilisation-Management-Policy.pdf	The four roles, the separation of opening and deciding, the three decisions and when each is open, the letters and the notice, the queue, the two figures, and what Provider Services may say
Prior-Authorisation-Rules.pdf	How every answer is read: identifiers, resubmissions, heights and weights, the body mass index, durations, scales, answers in words, criterion results, the screening, documents against the questionnaire, and a worked example
Past-Reviews-Casebook.pdf	Eight past reviews, one per kind of evidence. It names members and practices, and is the one document the chat may not read
Evidence-Guide-Conservative-Treatment.pdf	What this evidence covers, what the reviewer checks, which criteria it bears on, and what to tell the requesting office
Evidence-Guide-Failed-Medication.pdf	The same, for failed medication
Evidence-Guide-Red-Flag-Symptom.pdf	The same, for a red-flag symptom
Evidence-Guide-Functional-Limitation.pdf	The same, for a functional limitation
Evidence-Guide-Prior-Investigation.pdf	The same, for a prior investigation
Evidence-Guide-Weight-Management-Programme.pdf	The same, for a weight-management programme
Evidence-Guide-Other-Condition.pdf	The same, for another condition
Evidence-Guide-Ruled-Out.pdf	The same, for something ruled out

Every document carries one header line of a fixed form under its title, recognised by **config.DOCUMENT_HEADER_PATTERN**, and is divided into sections numbered from **1**, recognised by **config.SECTION_HEADING_PATTERN**.
The five medical policies are written from one template: the same headings, and in many places the same sentences. Section 4 of each lists its numbered coverage criteria, each written in a fixed form, and **config.CRITERION_SENTENCE_PATTERNS** holds the forms a criterion can take. The configuration names the policy of each service in **config.SERVICE_POLICY_DOCUMENTS** and the guide of each kind of evidence in **config.EVIDENCE_GUIDE_DOCUMENTS**. It holds no criterion's figure, no red flag list and no listed condition of any policy: every one of them is read from that policy's own document at run time.
Each policy's documentation to send sits under **config.DOCUMENTATION_SECTION_HEADING**, and each guide's checks and message to the office under **config.REVIEWER_CHECKS_SECTION_HEADING** and **config.OFFICE_MESSAGE_SECTION_HEADING**. **config.CHAT_EXCLUDED_DOCUMENTS** names the casebook, which is indexed with the others and kept out of every chat search.

DOMAIN KNOWLEDGE CONTEXT
Nobody is expected to arrive knowing utilisation management. This section gives only what is needed to read the requirements that follow. Every value named here already exists in **config/config.py**, except each policy's criteria, which exist only in the policies themselves.

WHAT A PRIOR AUTHORISATION REQUEST IS
Before a member receives one of the five services, the requesting office asks the plan to authorise it. The request carries a questionnaire, whose answers are what each criterion is judged on, and the clinical documents the office chose to attach: letters, reports and assessments that are evidence, but never answers. An Intake Coordinator opens the review, and a UM Nurse Reviewer who did not open it decides.

FIVE SERVICES, FIVE POLICIES

Service code	Service	Medical policy	What its criteria cover
RAD-LSM	MRI of the lumbar spine	Medical-Policy-Lumbar-Spine-MRI.pdf	A minimum age, conservative treatment unless a listed red flag is reported, and when a recent scan may be repeated
RAD-CSM	MRI of the cervical spine	Medical-Policy-Cervical-Spine-MRI.pdf	The same three, with its own treatment length and its own red flags
ORT-TKR	Total knee replacement	Medical-Policy-Total-Knee-Replacement.pdf	A minimum age, the X-ray grade, conservative treatment, the knee score and a body mass index ceiling
BAR-WLS	Bariatric surgery	Medical-Policy-Bariatric-Surgery.pdf	A minimum age, a body mass index floor that is lower with a listed condition, a weight-management programme and a psychological assessment
SLP-PSG	In-laboratory sleep study	Medical-Policy-In-Laboratory-Sleep-Study.pdf	A minimum age, the sleepiness score, and a need for the laboratory: a listed condition or a home test that settled nothing

config.SERVICE_NAMES gives each code its name, and **config.SERVICE_POLICY_DOCUMENTS** its policy.

THE CRITERIA ARE NOT IN THE CODE

Each policy states its own criteria in section 4, each in a fixed form, in its own order: what is asked, and the figure it is judged against. Those sentences live in that policy's PDF and nowhere else. The system reads them at run time, from that policy's own document, and keeps each criterion's name as the policy writes it, the sentence it was read from, its kind from **config.CRITERION_SENTENCE_PATTERNS**, and its own figures, red flags or listed conditions. A submission that writes a criterion's figure into the code, or reads one policy's criteria for another service, is wrong even when the number happens to match.

THE QUESTIONNAIRE, AS THE RULES READ IT

The Prior Authorisation Rules turn every way an office writes an answer into one canonical value:

- A request reference is **PA-** and six digits, a member number **BWH-** and seven digits, and an ICD-10-CM code gets its dot back after the third character. Any other notation is refused, not guessed.
- Conservative treatment is whole weeks: days are divided by **config.DAYS_PER_WEEK** with the remainder dropped. Months are whole months.
- Heights are centimetres and weights kilograms, each worked out exactly and rounded once, half up, to **config.MEASUREMENT_STEP**. Only the notations in **config.HEIGHT_NOTATIONS** and **config.WEIGHT_NOTATIONS** are read, and a measurement of nought is refused.
- A grade or score is read in every way its scale is written. A reading beyond its scale cannot be true: it becomes missing and the row stays.
- Answers in words take one spelling, and a list of conditions becomes the plan's canonical names, sorted and joined with **config.CONDITION_SEPARATOR**, or **config.NO_OTHER_CONDITIONS**.
- An answer written as one of **config.NULL_TOKENS** is no answer. A blank is never the same as "none".

THE LATEST SUBMISSION

A provider may submit a request again before its review begins. Every submission carries the same request reference and a higher **submission_sequence**. The submission with the highest sequence is the request, wherever it stands among the rows, and every earlier submission is set aside, even when it gives other answers or a decision.

MET, NOT MET AND NOT DOCUMENTED

Each criterion takes one of three results, **config.CRITERION_MET**, **config.CRITERION_NOT_MET** or **config.CRITERION_NOT_DOCUMENTED**:

- A minimum is met at the figure itself, and a maximum likewise; a ceiling written as "below" is met only under it.
- A criterion that can be met in two ways is Met when either way is met, even with the other way's answer blank; Not met only when both ways are known to fail, which a blank answer prevents only when it could still meet its way; Not documented otherwise.
- A follow-on question, such as the months since a scan, is not needed when the first answer settles the criterion.
- The body mass index is weight in kilograms divided by the square of height in metres, worked out exactly from the canonical height and weight, rounded once, half up, to one decimal, and compared as rounded. With either answer blank there is no index and the criterion is Not documented.
- Each criterion that is Not documented names the questionnaire answers it needs that were left blank, from **config.CRITERION_ANSWER_COLUMNS**.

THE SCREENING AND THE DECISIONS OPEN

Criteria results	Screening	Decisions open
Any criterion Not met	**config.DECISION_REFER**	**config.DECISION_REFER** only
None Not met, at least one Not documented	**config.DECISION_REQUEST_INFORMATION**	**config.DECISION_REQUEST_INFORMATION** and **config.DECISION_REFER**
Every criterion Met	**config.DECISION_APPROVE**	**config.DECISION_APPROVE** and **config.DECISION_REFER**

config.DECISIONS_OPEN pairs each screening with its decisions, and **config.SCREENING_CLOSED_REASONS** says why a decision is closed. The counts of Met, Not met and Not documented, under **config.CRITERIA_COUNT_COLUMNS**, are what the approval model reads.

THE WORKED EXAMPLE

Section 12 of the Prior Authorisation Rules works one fax through a policy that belongs to no service: a minimum age of 21, 8 whole weeks of treatment unless a red flag is reported, and a body mass index below 38.0. Treatment written **52 days** is 7 whole weeks; a red flag written **Nil** is no red flag; **5 ft 10 in** and **15 st 7 lb** are **177.8** cm and **98.4** kg, an index of **31.1**. The screening is REFER TO PHYSICIAN. With **56 days** every criterion would be Met; with the red flag blank the screening would be REQUEST MORE INFORMATION.

THE APPROVAL LIKELIHOOD AND ITS BAND

The approval model gives the share of comparable past requests that were approved, from the inputs **config.APPROVAL_NUMERIC_FEATURES** and **config.APPROVAL_CATEGORICAL_FEATURES** name, the three criteria counts among them. It is shown to **config.LIKELIHOOD_DECIMALS** decimal places and banded on the figure shown: **config.HIGH_APPROVAL_BAND** from **config.HIGH_APPROVAL_MINIMUM**, **config.MEDIUM_APPROVAL_BAND** from **config.MEDIUM_APPROVAL_MINIMUM**, otherwise **config.LOW_APPROVAL_BAND**. It decides nothing.

THE EXPECTED STAY, AND THE LONGER STAY

For an inpatient operation only, at **config.INPATIENT_SITE**, the stay model gives the nights members like this one stay after the same operation, shown to **config.STAY_DECIMALS** decimal place. When the figure shown is more than **config.LONGER_STAY_MARGIN_DAYS** day above the inpatient days requested, a longer stay is expected; with no days requested, nothing is compared. For either figure, an input left blank is filled as the model's training filled it, as **config.BLANK_INPUT_RULE_TEXT** states. The stay decides nothing.

READING A CLINICAL DOCUMENT

Each document is split into its sentences, in the order they were written, and each sentence is given one of the evidence types in **config.EVIDENCE_TYPE_MAPPING**. Each type bears on the criterion kinds **config.EVIDENCE_CRITERION_KINDS** lists, and a sentence records the criteria of its own request's policy it bears on; Background bears on none.

A sentence read as **config.CONSERVATIVE_TREATMENT_EVIDENCE** that gives weeks of treatment other than the questionnaire's is flagged as **config.DISAGREES_WITH_QUESTIONNAIRE**, and one that gives weeks the questionnaire left blank as **config.DESCRIBES_BLANK_ANSWER**. Weeks are read in digits, decimals, words and compounds; a time such as "six weeks ago" says when, not how long, and is not read. Nothing is flagged under a policy that asks no weeks of treatment. A flag never changes an answer.

THE LETTERS, THE NOTICE AND THE REFERRAL SUMMARY

Decision	What is written
config.DECISION_APPROVE	A provider authorisation letter to the requesting office, showing the request reference, the member number, the service and its code, the site of service, the medical policy and, for an inpatient operation, the inpatient days authorised; then a member notice saying, in plain words, that the service is authorised and where
config.DECISION_REQUEST_INFORMATION	An information request letter naming each criterion not documented with the answers it needs, the policy's documentation to send, the time to reply and what happens without a reply
config.DECISION_REFER	No letter: a referral summary of the criteria, the evidence and every disagreement, for a Physician Reviewer

The details a letter shows are added from the request by the tools, never written by a model. No letter or notice speaks of any of **config.LETTER_FORBIDDEN_TERMS**, names a reviewer, or leaves a gap to fill in, and each ends with **config.LETTER_SIGN_OFF**. The notice carries no clinical detail beyond the service's name.

THE THREE ROLES

Role	Reaches	Does not reach
config.ROLE_NURSE	Every area, and is the only role that may decide, never a review they opened	—
config.ROLE_COORDINATOR	**config.AREA_INTAKE**, **config.AREA_REVIEW**, **config.AREA_AUDIT** and **config.AREA_CHAT**	**config.AREA_DECISION**
config.ROLE_REPRESENTATIVE	**config.AREA_CHAT** only	Everything else

The roles are **UM Nurse Reviewer**, **Intake Coordinator** and **Provider Services Representative**. **config.ROLE_AREA_ACCESS** gives each role its areas, **config.ROLE_PAGE_ACCESS** its pages, and **config.AREA_BY_ROUTE** the area of every route. The separation between opening and deciding comes from the Utilisation Management Policy, and is the reason the workflow stops at a gate.

THE CONFIRMATION LINE

A Provider Services Representative answers requesting offices, so what they may hear about a request is fixed by section 10 of the Utilisation Management Policy:

- Asked whether a request will be approved, or how likely that is, they hear **config.CLINICAL_DETERMINATION_LINE** and nothing more, even with the reference and the member number.
- Asked about a request without naming it, they hear **config.CHAT_REQUEST_NOT_NAMED_MESSAGE**.
- With the request reference and the member number on that request, they hear its service and its status, as **config.CHAT_REQUEST_CONFIRMED_TEMPLATE**, and, where information was requested, the answers asked for.
- Asked any other way, with no member number, another request's member number or a reference that is not open, they hear **config.CHAT_NO_CONFIRMATION_TEMPLATE**, filled with the reference asked about, which never confirms that a request exists.

An Intake Coordinator and a UM Nurse Reviewer, named in **config.ROLES_SEEING_EVERY_REVIEW**, hear a review's whole record.

A SHORT GLOSSARY

Term	Meaning
Request	A requesting office's application for a service to be authorised
Request reference	The request's identifier, **PA-** and six digits
Member number	The member's identifier, **BWH-** and seven digits
Submission	One version of a request; the latest is the request
Medical policy	A service's own document of criteria
Criterion	One condition a policy sets, with its own figure
Screening	What the criteria results add up to: APPROVE, REQUEST MORE INFORMATION or REFER TO PHYSICIAN
Decisions open	The decisions the screening allows a UM Nurse Reviewer to take
Evidence type	The kind of evidence a sentence of a document gives
Disagreement	A sentence whose weeks of treatment are not the questionnaire's
Approval likelihood	The share of comparable past requests that were approved
Expected stay	The nights members like this one stay after the same operation
Review	The agents' write-ups and recommendation, waiting for a decision
Gate	The point where the workflow stops for a UM Nurse Reviewer
Referral summary	What a Physician Reviewer receives with a referred request

TASKS

For each module the responsibility is stated, and then every function within it is specified by its signature, its parameters, its purpose, the requirements its result must satisfy, and what it returns.
Names matter. Every function name, parameter name, module path and dictionary key given below is read by other parts of the system and by the marking suite, and must be reproduced exactly as written, including case.
Every path, spelling correction, validation limit, threshold, vocabulary, pattern and business rule already exists in **config/config.py**, which is supplied complete.

========== DATA PREPARATION ==========

MODULE: data_pipeline/dataset_loader.py
The only data preparation module that reads or writes CSV files. Raw files are read without guessing any column's type; cleaned files are read with each column's type taken from its contents.

FUNCTION SIGNATURE

`load_raw_dataset(file_path)`

PARAMETERS

file_path — **str**. The path of one raw extract under **data/**, or of a live file under **data/input/**, passed by **run_data_pipeline.py** or by the live-file readers of **tools/request_data_tools.py** from config, for example **config.RAW_AUTHORISATION_HISTORY_TRAIN_FILE** or **config.OPEN_REQUESTS_FILE**.

PURPOSE

Read one raw file as text, guessing nothing.

REQUIREMENTS

- Every column holds text, and no value carries leading or trailing whitespace.
- Only a value that matches one of **config.NULL_TOKENS**, ignoring case, is a missing value (NaN); every other value is kept as text, the answer "None" among them, and so is a value that merely contains such a token.
- The row count and the column count match the file on disk.

RETURNS

pandas.DataFrame. Every row of the named file, every column held as text and every missing value as NaN.

FUNCTION SIGNATURE

`save_processed_dataset(dataframe, file_path)`

PARAMETERS

dataframe — **pandas.DataFrame**. A finished cleaned table, passed by **run_data_pipeline.py**.

file_path — **str**. The destination under **processed/**, passed by **run_data_pipeline.py** from config, for example **config.CLEAN_SURGICAL_STAYS_TRAIN_FILE**.

PURPOSE

Save a cleaned table where the modelling stage will look for it.

REQUIREMENTS

- The folder the file belongs in exists afterwards, whether or not it existed before.
- The file holds the table's own columns and no added index column, so reading it back gives the same rows, columns and values.

RETURNS

None.

FUNCTION SIGNATURE

`load_processed_dataset(file_path)`

PARAMETERS

file_path — **str**. The path of a cleaned table under **processed/**, passed by **run_model_training.py** from config, for example **config.CLEAN_AUTHORISATION_HISTORY_TRAIN_FILE**.

PURPOSE

Read a table the pipeline has already cleaned.

REQUIREMENTS

- Column types follow the file's contents, and only an empty cell is read as missing.
- Every row and every column of the file is returned, unfiltered and unsorted.

RETURNS

pandas.DataFrame. The file as it stands on disk.

MODULE: data_pipeline/common_cleaning.py

The repairs shared by the cleaning runs and by the detail profiles. Every function takes a table or a column and returns a repaired copy, leaving what it was given untouched. No function here reads or writes a file, and none removes a column. A value in a notation a function does not recognise is refused with a **ValueError** naming the column and the values.

FUNCTION SIGNATURE

`remove_duplicate_records(dataframe)`

PARAMETERS

dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by the calling cleaning function.

PURPOSE

Drop rows the extract wrote twice.

REQUIREMENTS

- Rows identical in every column appear once, and the first of them is the one kept.
- Rows that differ anywhere are all kept, and the result is numbered from nought with no gaps.

RETURNS

pandas.DataFrame. The table without exact repeats.

FUNCTION SIGNATURE

`refuse_missing_values(dataframe, required_columns)`

PARAMETERS

dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by the calling cleaning function or by **load_request_notes**.
required_columns — **list[str]**. The columns that must be given on every row, passed by the caller, for example **[config.ADMISSION_REFERENCE_COLUMN]**.
PURPOSE
Refuse a table in which a required key is blank.
REQUIREMENTS

- Every listed column holds a value on every row; otherwise the table is refused, naming the column and how many rows leave it blank.

RETURNS
pandas.DataFrame. The table, unchanged.

FUNCTION SIGNATURE

```
refuse_repeated_references(dataframe, reference_columns)
```

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by the calling cleaning function or by **load_request_notes**.
reference_columns — **list[str]**. The columns whose values together name a row, passed by the caller, for example **[config.SENTENCE_ID_COLUMN]**.
PURPOSE
Refuse a table in which two rows claim the same identity.
REQUIREMENTS

- Every row carries its own combination of the listed columns; otherwise the table is refused, naming the columns and every repeated value.

RETURNS
pandas.DataFrame. The table, unchanged.

FUNCTION SIGNATURE

```
remove_rows_missing_target(dataframe, target_column)
```

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by the calling cleaning function.
target_column — **str**. The dataset's answer column, passed from config, for example **config.STAY_TARGET_COLUMN**.
PURPOSE
Drop the rows a model cannot learn from.
REQUIREMENTS

- Every row holding an answer is kept, every row without one is removed, and the result is numbered from nought.

RETURNS
pandas.DataFrame. The rows that carry an answer.

FUNCTION SIGNATURE

```
normalize_category_values(dataframe, value_mappings)
```

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by the calling cleaning function.
value_mappings — **dict**. Each category column against its mapping, passed from config, for example **config.HISTORY_VALUE_MAPPINGS**.
PURPOSE
Fold every spelling of a value onto its canonical value.
REQUIREMENTS

- Each listed column holds the value its mapping gives for the written value in lower case; a missing value stays missing.
- A value its mapping does not recognise is refused, naming the column and the values.
- No other column is touched.

RETURNS
pandas.DataFrame. The table with every listed column canonical.

FUNCTION SIGNATURE

```
normalize_name_spacing(dataframe, name_columns)
```

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, passed by the calling cleaning function.
name_columns — **list[str]**. The name columns to tidy, passed from config, for example **config.REQUEST_NAME_COLUMNS**.
PURPOSE
Make one name out of a name written with runs of spaces.
REQUIREMENTS

- In each listed column, every run of whitespace matched by **config.WHITESPACE_RUN_PATTERN** becomes a single space; a missing value stays missing.

RETURNS
pandas.DataFrame. The table with tidy names.

FUNCTION SIGNATURE

`convert_identifiers(identifiers, notation_pattern, canonical_format)`

PARAMETERS
identifiers — **pandas.Series**. One identifier column, named after it, passed by the calling cleaning function, for example the **member_number** column.
notation_pattern — **str**. Every notation the Rules allow for that identifier, passed from config, for example **config.MEMBER_NUMBER_PATTERN**.
canonical_format — **str**. The identifier's canonical form, passed from config, for example **config.CANONICAL_MEMBER_NUMBER_FORMAT**.
PURPOSE
Write every identifier in its one canonical notation.
REQUIREMENTS

- Every value written in an allowed notation is rewritten in the canonical form; a missing value stays missing.
- A value in no allowed notation is refused, naming the column and the values.

RETURNS
pandas.Series. The identifiers, canonical.

FUNCTION SIGNATURE
`convert_whole_numbers(dataframe, number_patterns)`

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run or a detail profile has reached, passed by the caller.
number_patterns — **dict**. Each whole-number column against the pattern that reads it, passed from config, for example **config.HISTORY_WHOLE_NUMBER_PATTERNS**.
PURPOSE
Read numbers written with units or scales as whole numbers.
REQUIREMENTS

- Each listed column holds the whole number its pattern reads out of the written value, as a nullable integer; a missing value stays missing.
- A value its pattern cannot read is refused, naming the column and the values.

RETURNS
pandas.DataFrame. The table with every listed column as whole numbers.

FUNCTION SIGNATURE
`convert_decimal_numbers(dataframe, number_patterns)`

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run or a detail profile has reached, passed by the caller.
number_patterns — **dict**. Each decimal column against the pattern that reads it, passed from config, for example **config.STAY_DECIMAL_NUMBER_PATTERNS**.
PURPOSE
Read numbers written with units or signs as decimals.
REQUIREMENTS

- Each listed column holds the number its pattern reads out of the written value; a missing value stays missing.
- A value its pattern cannot read is refused, naming the column and the values.

RETURNS
pandas.DataFrame. The table with every listed column as decimals.

FUNCTION SIGNATURE
`convert_durations_to_whole_weeks(durations)`

PARAMETERS
durations — **pandas.Series**. The **conservative_treatment_weeks** column, passed by **clean_request_records**.
PURPOSE
Turn a treatment length written in weeks or days into whole weeks.
REQUIREMENTS

- A length written as weeks, with or without a unit, is that number; a length written in days is the whole weeks in them, the days divided by **config.DAYS_PER_WEEK** with the remainder dropped. **config.TREATMENT_DURATION_PATTERN** reads both, as its **config.TREATMENT_WEEKS_PART** and **config.TREATMENT_DAYS_PART** parts.
- A missing value stays missing, and any other writing is refused, naming the column and the values.

RETURNS
pandas.Series. Whole weeks, as nullable integers.

FUNCTION SIGNATURE
`convert_asa_classes(asa_classes)`

PARAMETERS
asa_classes — **pandas.Series**. The **asa_physical_status** column, passed by the calling cleaning function.
PURPOSE
Read the ASA physical status class however it is written.
REQUIREMENTS

- A class written as a number, a Roman numeral or with **ASA** before it, as **config.ASA_CLASS_PATTERN** reads it, is the number **config.ASA_CLASS_NUMBERS** gives it; a missing value stays missing.
- A class with no number there, such as **ASA V**, is refused, naming the column and the values.

RETURNS
pandas.Series. The classes, as nullable integers.

FUNCTION SIGNATURE
`convert_measurements(measurements, notations, units)`

PARAMETERS
measurements — **pandas.Series**. The **height_cm** or **weight_kg** column, passed by **clean_request_records**.
notations — **dict**. Each allowed notation against the pattern that reads its parts, passed from config, **config.HEIGHT_NOTATIONS** or **config.WEIGHT_NOTATIONS**.
units — **dict**. Each part's size in the canonical unit, passed from config, **config.CENTIMETRES_PER_HEIGHT_UNIT** or **config.KILOGRAMS_PER_WEIGHT_UNIT**.
PURPOSE
Turn a height or weight written in any allowed notation into one canonical measurement.
REQUIREMENTS

- The measurement is worked out exactly from every part of the written value, and rounded once, half up, to **config.MEASUREMENT_STEP**.
- A missing value stays missing. A value in no allowed notation, and a measurement of nought, are refused, naming the column and the values.

RETURNS
pandas.Series. The canonical measurements.

FUNCTION SIGNATURE
`convert_condition_lists(condition_lists)`

PARAMETERS
condition_lists — **pandas.Series**. The **other_conditions** column, passed by **clean_request_records** or by **profile_history_details**.
PURPOSE
Turn a list of conditions written freely into the plan's canonical list.
REQUIREMENTS

- The result holds the canonical name, from **config.CONDITION_MAPPING** in any case, of each condition the list names, each once, sorted and joined with **config.CONDITION_SEPARATOR**; the names of a list are those **config.CONDITION_LIST_SEPARATOR_PATTERN** separates.
- One of **config.NO_OTHER_CONDITIONS_SPELLINGS**, in any case, becomes **config.NO_OTHER_CONDITIONS**, and a missing value stays missing.
- A list naming a condition with no canonical name is refused, naming the column and the lists.

RETURNS
pandas.Series. The canonical lists, as text.

FUNCTION SIGNATURE
`treat_values_below_minimum_as_missing(dataframe, minimum_values)`

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, its numbers already read, passed by the calling cleaning function.
minimum_values — **dict**. Each column against the lowest value it can truly hold, passed from config, for example **config.STAY_MINIMUM_VALUES**.
PURPOSE
Empty a reading that cannot be true, keeping the row.
REQUIREMENTS

- A value below its column's minimum becomes missing; every other value, and every row, is kept.

RETURNS
pandas.DataFrame. The table with impossible low readings missing.

FUNCTION SIGNATURE
`treat_values_above_maximum_as_missing(dataframe, maximum_values)`

PARAMETERS
dataframe — **pandas.DataFrame**. The table a cleaning run has reached, its numbers already read, passed by the calling cleaning function.
maximum_values — **dict**. Each column against the highest value it can truly hold, passed from config, for example **config.REQUEST_MAXIMUM_VALUES**.
PURPOSE
Empty a reading beyond its scale, keeping the row.
REQUIREMENTS

- A value above its column's maximum becomes missing; every other value, and every row, is kept.

RETURNS
pandas.DataFrame. The table with impossible high readings missing.

FUNCTION SIGNATURE
`keep_latest_submission_of_each_request(dataframe)`

PARAMETERS
dataframe — **pandas.DataFrame**. Request rows with canonical references and whole-number submission sequences, passed by **clean_request_records** or by **profile_history_details**.
PURPOSE
Keep each request once, as its latest submission.
REQUIREMENTS

- Each request reference appears once, as its row with the highest **submission_sequence**, wherever that row stands; the kept rows stay in their original order and are numbered from nought.
- A row with no reference or no sequence, and two rows sharing a reference and a sequence, are refused, naming the requests.

RETURNS
pandas.DataFrame. One row per request.

FUNCTION SIGNATURE
`convert_markup_to_plain_text(texts)`

PARAMETERS
texts — **pandas.Series**. Free text as it arrived, passed by **clean_clinical_text** or by **profile_sentence_details**.
PURPOSE
Leave the words the writer meant and nothing around them.
REQUIREMENTS

- The result holds no markup matched by **config.MARKUP_PATTERN**, each piece counting as a space, and no escaped character, every HTML character reference, named or numeric, standing as the character it encodes; every run of whitespace, no-break spaces included, is one space, with none at either end.
- A missing value stays missing.

RETURNS
pandas.Series. Plain text.

FUNCTION SIGNATURE
`clean_clinical_text(texts)`

PARAMETERS
texts — **pandas.Series**. A sentence or document column as it arrived, passed by **clean_note_records** or by **profile_sentence_details**.
PURPOSE
Clean clinical text the way the evidence model reads it.
REQUIREMENTS

- The text is plain text, as **convert_markup_to_plain_text** leaves it.
- A text written wholly in capitals is lowered; any other text keeps its case exactly.

RETURNS
pandas.Series. Clean text.

MODULE: data_pipeline/request_cleaning.py
The repairs a request's row needs, shared by the authorisation history and the live queue.

FUNCTION SIGNATURE
`clean_request_records(frame, value_mappings, whole_number_patterns)`

PARAMETERS
frame — **pandas.DataFrame**. Request rows as **load_raw_dataset** read them, passed by **clean_history_dataset** or by **load_open_requests**.
value_mappings — **dict**. The category columns and their mappings, passed from config: **config.HISTORY_VALUE_MAPPINGS** for the history, **config.REQUEST_VALUE_MAPPINGS** for the live queue.
whole_number_patterns — **dict**. The whole-number columns and their patterns, passed from config: **config.HISTORY_WHOLE_NUMBER_PATTERNS** or **config.REQUEST_WHOLE_NUMBER_PATTERNS**.
PURPOSE
Read every answer of a request as the Prior Authorisation Rules read it.
REQUIREMENTS

- Exact repeats appear once; every column of **value_mappings** is canonical; every name column of **config.REQUEST_NAME_COLUMNS** is tidy.
- The request reference, the member number and the diagnosis code are canonical, through **config.REQUEST_REFERENCE_PATTERN**, **config.MEMBER_NUMBER_PATTERN** and **config.DIAGNOSIS_CODE_PATTERN**, written as **config.CANONICAL_REQUEST_REFERENCE_FORMAT**, **config.CANONICAL_MEMBER_NUMBER_FORMAT** and **config.CANONICAL_DIAGNOSIS_CODE_FORMAT**.
- The whole-number columns, **config.REQUEST_DECIMAL_NUMBER_PATTERNS**, the treatment weeks, the ASA class, the height, the weight and the conditions are read as the functions above read them.
- A reading above **config.REQUEST_MAXIMUM_VALUES** is missing, and each request appears once, as its latest submission.

RETURNS
pandas.DataFrame. One clean row per request, every column kept.

MODULE: data_pipeline/note_cleaning.py
The repairs a document's row needs, shared by the clinical note sentences and the live documents.

FUNCTION SIGNATURE
`clean_note_records(frame, value_mappings, whole_number_patterns, id_column, text_column)`

PARAMETERS
frame — **pandas.DataFrame**. Sentence or document rows as **load_raw_dataset** read them, passed by **clean_sentence_dataset** or by **load_request_notes**.
value_mappings — **dict**. The category columns and their mappings, passed from config: **config.SENTENCE_VALUE_MAPPINGS** or **config.NOTE_VALUE_MAPPINGS**.
whole_number_patterns — **dict**. The whole-number columns and their patterns, passed from config: **config.SENTENCE_WHOLE_NUMBER_PATTERNS** or **config.NOTE_WHOLE_NUMBER_PATTERNS**.
id_column — **str**. The row's own identifier, passed from config: **config.SENTENCE_ID_COLUMN** or **config.NOTE_ID_COLUMN**.
text_column — **str**. The text column, passed from config: **config.SENTENCE_TEXT_COLUMN** or **config.NOTE_TEXT_COLUMN**.
PURPOSE
Clean every sentence or document row the same way.
REQUIREMENTS

- Exact repeats appear once; among the rows left, **id_column** and the request reference are given on every row and no **id_column** is shared, as the refusing functions above require.
- Every column of **value_mappings** is canonical, the request reference is canonical, and the whole-number columns are read.
- **text_column** is clean clinical text.

RETURNS
pandas.DataFrame. The clean rows.

MODULE: data_pipeline/history_cleaning.py

FUNCTION SIGNATURE
`clean_history_dataset(frame)`

PARAMETERS
frame — **pandas.DataFrame**. An authorisation history extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py**.
PURPOSE
Clean one authorisation history extract.
REQUIREMENTS

- Every request is clean, as **clean_request_records** leaves it with **config.HISTORY_VALUE_MAPPINGS** and **config.HISTORY_WHOLE_NUMBER_PATTERNS**.
- A reading below **config.HISTORY_MINIMUM_VALUES** is missing.
- A request whose latest submission gives no **config.HISTORY_TARGET_COLUMN** is removed; a decision is never taken from an earlier submission.

RETURNS
pandas.DataFrame. One row per decided request, all **44** columns kept.

MODULE: data_pipeline/stay_cleaning.py

FUNCTION SIGNATURE
`clean_stay_dataset(frame)`

PARAMETERS
frame — **pandas.DataFrame**. A surgical stay extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py**.
PURPOSE
Clean one surgical stay extract.
REQUIREMENTS

- Exact repeats appear once; among the rows left, a blank **config.ADMISSION_REFERENCE_COLUMN**, or one shared by two rows, is refused, naming the column and the shared values.
- Every column of **config.STAY_VALUE_MAPPINGS** is canonical, **config.STAY_NAME_COLUMNS** are tidy, and the member number is canonical.
- **config.STAY_WHOLE_NUMBER_PATTERNS**, **config.STAY_DECIMAL_NUMBER_PATTERNS** and the ASA class are read, and a reading below **config.STAY_MINIMUM_VALUES** or above **config.STAY_MAXIMUM_VALUES** is missing.
- A stay with no length, including one whose length could not be true, is removed.

RETURNS
pandas.DataFrame. One row per admission with a length of stay, all **21** columns kept.

MODULE: data_pipeline/sentence_cleaning.py

FUNCTION SIGNATURE
`clean_sentence_dataset(frame)`

PARAMETERS
frame — **pandas.DataFrame**. The clinical note sentences as **load_raw_dataset** read them, passed by **run_data_pipeline.py**.
PURPOSE
Clean the labelled sentences the evidence model learns from.
REQUIREMENTS

- Every row is clean, as **clean_note_records** leaves it with the sentence settings of config.
- **config.SENTENCE_LENGTH_COLUMN** is measured again on the clean text.
- A row with no evidence type, and a sentence of fewer than **config.MINIMUM_SENTENCE_WORDS** words, is removed, and the result is numbered from nought.

RETURNS
pandas.DataFrame. The clean labelled sentences, all **8** columns kept.

MODULE: data_pipeline/feature_builder.py
The criteria counts the approval model reads, the model inputs in their saved order, and the preparation every tabular model is fitted with.

FUNCTION SIGNATURE
`add_criteria_counts(dataframe, policies)`

PARAMETERS
dataframe — **pandas.DataFrame**. Clean history requests, read by **load_processed_dataset**, passed by **run_model_training.py**.
policies — **dict**. Each service code of **config.SERVICE_POLICY_DOCUMENTS** against its policy, as **read_medical_policy** returned it, passed by **run_model_training.py**.
PURPOSE
Give every history request the criteria counts its own policy's screening gives it, as a live request is screened.
REQUIREMENTS

- The same rows come back in the same order, with the three columns **config.CRITERIA_COUNT_COLUMNS** names added.
- Each row's counts are those **screen_request** gives that request under its own service's policy, and they add up to that policy's number of criteria.
- A request with no service code is refused with **config.NO_SERVICE_CODE_MESSAGE**; a request whose service has no policy handed over is refused, naming the request and the service; a policy handed over under another service's code is refused by the screening, naming the request.

RETURNS
pandas.DataFrame. The requests with their criteria counts.

FUNCTION SIGNATURE
`build_feature_frame(dataframe, numeric_columns, categorical_columns)`

PARAMETERS
dataframe — **pandas.DataFrame**. Clean rows holding at least the named columns, in any order, passed by **run_model_training.py** or by **tools/prediction_tools.py**.
numeric_columns — **list[str]**. The number inputs, passed from config, for example **config.APPROVAL_NUMERIC_FEATURES**.
categorical_columns — **list[str]**. The word inputs, passed from config, for example **config.STAY_CATEGORICAL_FEATURES**.
PURPOSE
Lay a table out exactly as a model is fitted and scored on it.
REQUIREMENTS

- The columns are **numeric_columns** and then **categorical_columns**, in that order, whatever order the table holds them in, with the same rows and index.
- A number goes in as it was cleaned, as a float, and a gap as NaN.
- An answer in words goes in as cleaned, and a gap stays a missing value, never the text **nan** or an empty string.

RETURNS
pandas.DataFrame. The model inputs.

FUNCTION SIGNATURE
`build_model_pipeline(estimator, numeric_columns, categorical_columns)`

PARAMETERS
estimator — **sklearn estimator**. The unfitted estimator, passed by **train_approval_model** or **train_stay_model**.
numeric_columns — **list[str]**. The number inputs, passed from config.
categorical_columns — **list[str]**. The word inputs, passed from config.
PURPOSE
Join the input preparation and the estimator into one object that can be fitted, saved and used on a single row.
REQUIREMENTS

- Fitted, the pipeline fills a blank number with the training data's **config.NUMBER_FILL_STRATEGY** and a blank answer in words with its **config.CATEGORY_FILL_STRATEGY** value, and the estimator receives each answer in words as indicator columns.
- The estimator is the pipeline's last step, named **config.MODEL_STEP_NAME**.

- Fitted on a table, the pipeline records the names of the columns it was fitted on, in order, and refuses a category its training never saw.

RETURNS
sklearn.pipeline.Pipeline. The preparation and the estimator, unfitted.

MODULE: data_pipeline/quality_profiler.py
Measures how dirty each raw extract was, and writes the evidence out. Every figure is counted on the extract's first copies of each row as they arrived, unless said otherwise.

FUNCTION SIGNATURE
`profile_dataset(raw_frame, clean_frame, target_column)`

PARAMETERS
raw_frame — **pandas.DataFrame**. The extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py**.
clean_frame — **pandas.DataFrame**. The same extract as its cleaning function returned it, passed by **run_data_pipeline.py**.
target_column — **str**. The extract's answer column, passed from config.
PURPOSE
Account for one dataset: what arrived, what survived, and what was missing.
REQUIREMENTS

- The profile carries **rows_received**, **rows_kept**, **rows_removed**, **duplicate_rows**, **rows_missing_target**, **columns**, **missing_cells_as_received** and **missing_cells_after_cleaning**.
- **rows_received** is every row of the raw file, repeats included, and **rows_kept** every row of the cleaned table; **rows_removed** is **rows_received** less **rows_kept**; **duplicate_rows** counts the copies beyond the first; **rows_missing_target** counts the first copies with no answer.
- **columns** is the number of columns of the cleaned table.
- The two missing-cell entries name only the columns with a gap, each with its count.

RETURNS
dict. The eight entries above.

FUNCTION SIGNATURE
`profile_history_details(raw_frame, clean_frame)`

PARAMETERS
raw_frame — **pandas.DataFrame**. An authorisation history extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py**.
clean_frame — **pandas.DataFrame**. The same extract as **clean_history_dataset** returned it.
PURPOSE
Account for the defects peculiar to the history extracts.
REQUIREMENTS

- **answered_earlier_submissions_removed** counts the submissions that gave a decision and were set aside for a later submission, the requests found by canonical reference as the cleaning finds them.
- **numbers_not_written_as_plain_numbers** counts, column by column, the values not written as **config.PLAIN_NUMBER_PATTERN** reads a plain number, in the columns of **config.HISTORY_WHOLE_NUMBER_PATTERNS** and **config.REQUEST_DECIMAL_NUMBER_PATTERNS** and the treatment weeks, the ASA class, the height and the weight; **treatment_durations_written_in_days** counts the treatment lengths given in days.
- **heights_by_notation_as_written** and **weights_by_notation_as_written** count the values written in each notation of config, by the notation's name.
- **values_above_maximum_as_received** and **values_below_minimum_as_received** count, by column, the readings beyond **config.REQUEST_MAXIMUM_VALUES** and **config.HISTORY_MINIMUM_VALUES**.
- **request_references_not_written_canonically**, **member_numbers_not_written_canonically**, **diagnosis_codes_not_written_canonically** and **condition_lists_not_written_canonically** count the values not already canonical, a reference, number or code being canonical when **config.CANONICAL_REQUEST_REFERENCE_PATTERN**, **config.CANONICAL_MEMBER_NUMBER_PATTERN** or **config.CANONICAL_DIAGNOSIS_CODE_PATTERN** matches it whole, and **category_values_not_written_canonically** does the same by column for **config.HISTORY_VALUE_MAPPINGS**.
- **target_shares** gives the share of each decision among the cleaned rows, to **config.TARGET_SHARE_DECIMALS** places.

RETURNS
dict. The entries above.

FUNCTION SIGNATURE
`profile_stay_details(raw_frame, clean_frame)`

PARAMETERS
raw_frame — **pandas.DataFrame**. A surgical stay extract as **load_raw_dataset** read it, passed by **run_data_pipeline.py**.
clean_frame — **pandas.DataFrame**. The same extract as **clean_stay_dataset** returned it.
PURPOSE
Account for the defects peculiar to the stay extracts.
REQUIREMENTS

- **rows_with_impossible_target** counts the stays whose length is below its minimum, and **values_below_minimum_as_received** and **values_above_maximum_as_received** count, by column, the readings beyond **config.STAY_MINIMUM_VALUES** and **config.STAY_MAXIMUM_VALUES**.

- **numbers_not_written_as_plain_numbers**, over the columns of **config.STAY_WHOLE_NUMBER_PATTERNS** and **config.STAY_DECIMAL_NUMBER_PATTERNS** and the ASA class, **member_numbers_not_written_canonically** and **category_values_not_written_canonically** count as they do for the history.
- **median_length_of_stay_days** is the middle length of the cleaned stays.

RETURNS
dict. The entries above.

FUNCTION SIGNATURE
`profile_sentence_details(raw_frame, clean_frame)`

PARAMETERS
raw_frame — **pandas.DataFrame**. The clinical note sentences as **load_raw_dataset** read them, passed by **run_data_pipeline.py**.
clean_frame — **pandas.DataFrame**. The same extract as **clean_sentence_dataset** returned it.
PURPOSE
Account for the defects peculiar to free text.
REQUIREMENTS

- **sentences_with_markup**, **sentences_with_encoding_artifacts** and **sentences_in_capitals** count the sentences that arrived with each defect.
- **sentence_lengths_disagreeing_with_plain_text_as_received** counts the lengths that do not match the plain text.
- **rows_with_too_few_words** counts the answered sentences whose plain text has fewer than **config.MINIMUM_SENTENCE_WORDS** words, and **shortest_sentence_words** is the shortest cleaned sentence in words.
- **request_references_not_written_canonically**, **category_values_not_written_canonically** and **target_shares** count as they do for the history.

RETURNS
dict. The entries above.

FUNCTION SIGNATURE
`write_quality_report(profiles)`

PARAMETERS
profiles — **dict**. Each dataset's profile, **profile_dataset** and its detail profile merged into one flat dictionary, keyed by the raw file's name without **.csv**, passed by **run_data_pipeline.py**.
PURPOSE
Leave the evidence on disk in a form a person and a program can both read.
REQUIREMENTS

- The report is written to **config.DATA_QUALITY_REPORT_FILE** as JSON indented by **config.JSON_INDENT** with its keys sorted, and its folder exists afterwards.
- It holds the five datasets, **authorisation_history_train**, **authorisation_history_eval**, **surgical_stays_train**, **surgical_stays_eval** and **clinical_note_sentences**, and in each one the reasons it carries among **duplicate_rows**, **rows_missing_target**, **answered_earlier_submissions_removed**, **rows_with_impossible_target** and **rows_with_too_few_words** add up to its **rows_removed**.
- Writing the same profiles twice produces the same file.

RETURNS
None.

========== MODELLING ==========

Three models are trained, scored on data they never learned from, and saved with everything needed to score one row. Each model is trained with the settings the configuration holds for it: config.APPROVAL_FOREST_TREES, config.APPROVAL_FOREST_LEAF_MINIMUM and **config.APPROVAL_FOREST_FEATURE_SHARE** for the approval model, **config.STAY_FOREST_TREES** and **config.STAY_FOREST_LEAF_MINIMUM** for the stay model, and **config.EVIDENCE_MODEL_MAXIMUM_ITERATIONS** for the evidence model, every random choice seeded with **config.RANDOM_SEED**. The interface, the scores reported and each model's minimum are fixed.

MODULE: modeling/model_registry.py
Saves and loads the models, their feature schemas and their scores. Nothing else in the project writes to output/models/.

FUNCTION SIGNATURE
`save_model(model, file_path)`

PARAMETERS
model — **sklearn.pipeline.Pipeline**. A fitted model, preparation and estimator together, passed by **run_model_training.py**.
file_path — **str**. Where to save it, passed from config, for example **config.APPROVAL_MODEL_FILE**.
PURPOSE
Put a fitted model on disk so that the service can use it without training anything.
REQUIREMENTS

- The folder the file belongs in exists afterwards, whether or not it existed before.
- The whole fitted object is saved, compressed at **config.MODEL_FILE_COMPRESSION**, so that loading it gives something that can score a raw row; saving again replaces the file.

RETURNS

None.

FUNCTION SIGNATURE
`save_feature_schema(feature_names, file_path)`

PARAMETERS
feature_names — **list[str]**. The columns the model was fitted on, in order, passed by **run_model_training.py**.
file_path — **str**. Where to save them, passed from config, **config.APPROVAL_SCHEMA_FILE** or **config.STAY_SCHEMA_FILE**.
PURPOSE
Record the columns a saved model expects, in the order it expects them.
REQUIREMENTS
- The file holds the column names in their original order, as JSON indented by **config.JSON_INDENT**, and its folder exists afterwards.

RETURNS
None.

FUNCTION SIGNATURE
`load_model(file_path)`

PARAMETERS
file_path — **str**. A model file written by **save_model**, passed from config, for example **config.EVIDENCE_MODEL_FILE**.
PURPOSE
Bring a saved model back into memory, ready to score.
REQUIREMENTS
- What comes back can predict at once, with no fitting and no preparation added by the caller.

RETURNS
sklearn.pipeline.Pipeline. The fitted model as it was saved.

FUNCTION SIGNATURE
`load_feature_schema(file_path)`

PARAMETERS
file_path — **str**. A schema file written by **save_feature_schema**, passed from config.
PURPOSE
Read back the columns a model expects.
REQUIREMENTS
- The list comes back in the order it was saved in.

RETURNS
list[str]. The column names.

FUNCTION SIGNATURE
`save_metrics(metrics)`

PARAMETERS
metrics — **dict**. Every model's record, keyed **approval**, **stay** and **evidence**, passed by **run_model_training.py**.
PURPOSE
Leave the scores on disk for a reader and for the marking suite.
REQUIREMENTS
- The file at **config.MODEL_METRICS_FILE** holds everything it was given, as JSON indented by **config.JSON_INDENT** with its keys sorted, and its folder exists afterwards.

RETURNS
None.

MODULE: modeling/model_evaluation.py
Scores a fitted model on the training data and on held-out data, so the gap between the two shows whether it learned or memorised. Each function scores the model it is given and never fits it again. Every figure is a plain number rounded to **config.METRIC_DECIMALS** places.

FUNCTION SIGNATURE
`evaluate_classifier(model, train_features, train_target, held_out_features, held_out_target)`

PARAMETERS
model — **sklearn.pipeline.Pipeline**. The fitted approval model, passed by **train_approval_model**.
train_features — **pandas.DataFrame**. The inputs it learned from.
train_target — **pandas.Series**. Their answers, **0** or **1**.
held_out_features — **pandas.DataFrame**. Inputs it never saw.
held_out_target — **pandas.Series**. Their answers.
PURPOSE
Say how well the approval model ranks requests it has not seen.
REQUIREMENTS

- The result carries **train_roc_auc**, **held_out_roc_auc**, **accuracy**, **precision**, **recall**, **f1**, **mean_predicted_probability**, **positive_share** and **generalisation_gap**.
- Both ROC AUC figures come from the model's own probabilities; the other figures are measured on the held-out rows.
- **generalisation_gap** is **train_roc_auc** less **held_out_roc_auc**.

RETURNS
dict. The nine figures above.

FUNCTION SIGNATURE
`evaluate_regressor(model, train_features, train_target, held_out_features, held_out_target)`

PARAMETERS
model — **sklearn.pipeline.Pipeline**. The fitted stay model, passed by **train_stay_model**.
train_features — **pandas.DataFrame**. The inputs it learned from.
train_target — **pandas.Series**. The nights those stays lasted.
held_out_features — **pandas.DataFrame**. Inputs it never saw.
held_out_target — **pandas.Series**. The nights those stays lasted.
PURPOSE
Say how close the stay model's nights are on stays it has not seen, and whether it beats guessing.
REQUIREMENTS

- The result carries **train_r2**, **held_out_r2**, **mean_absolute_error**, **average_estimate_mean_absolute_error**, **lowest_prediction**, **highest_prediction** and **generalisation_gap**.
- **average_estimate_mean_absolute_error** is what always answering the training average would cost on the held-out rows; the two predictions bound the model's held-out forecasts.
- **generalisation_gap** is **train_r2** less **held_out_r2**.

RETURNS
dict. The seven figures above.

FUNCTION SIGNATURE
`evaluate_text_classifier(model, train_text, train_target, held_out_text, held_out_target)`

PARAMETERS
model — **sklearn.pipeline.Pipeline**. The fitted evidence model, passed by **train_evidence_model**.
train_text — **pandas.Series**. The sentences it learned from.
train_target — **pandas.Series**. Their evidence types.
held_out_text — **pandas.Series**. Sentences it never saw.
held_out_target — **pandas.Series**. Their evidence types.
PURPOSE
Say how well the evidence model names every kind of evidence, not only the common ones.
REQUIREMENTS

- The result carries **train_macro_f1**, **held_out_macro_f1**, **accuracy**, **weighted_f1**, **generalisation_gap** and **per_class_f1**.
- **per_class_f1** is keyed by every class the model knows.
- **generalisation_gap** is **train_macro_f1** less **held_out_macro_f1**.

RETURNS
dict. The six entries above.

MODULE: modeling/approval_model.py

FUNCTION SIGNATURE
`train_approval_model(train_features, train_target, held_out_features, held_out_target)`

PARAMETERS
train_features — **pandas.DataFrame**. The training requests with their criteria counts, laid out by **build_feature_frame** from **config.APPROVAL_NUMERIC_FEATURES** and **config.APPROVAL_CATEGORICAL_FEATURES**, passed by **run_model_training.py**.
train_target — **pandas.Series**. Their decisions as integers, through **config.APPROVAL_TARGET_ENCODING**.
held_out_features — **pandas.DataFrame**. The held-out requests, built the same way.
held_out_target — **pandas.Series**. Their decisions, encoded the same way.
PURPOSE
Estimate the share of comparable past requests that were approved.
REQUIREMENTS

- The model is a pipeline from **build_model_pipeline**, fitted on the training rows only; it gives probabilities, and its classes are **0** and **1** in that order.
- Its held-out ROC AUC reaches **config.MINIMUM_APPROVAL_ROC_AUC**, and its scores are calibrated: across the held-out requests the mean score is within **0.03** of the share that were approved.
- It learns from the screening. Turning one criterion from Not met to Met raises the score for at least **95%** of the held-out requests that have at least one Not met, and from Not documented to Met for at least **80%** of those that have at least one Not documented; either way the average score rises by at least **0.1**.
- Across the live queue its likelihoods fall in all three bands.
- Its scores come from **evaluate_classifier**.

RETURNS
tuple[sklearn.pipeline.Pipeline, dict]. The fitted model and its scores.

MODULE: modeling/stay_model.py

FUNCTION SIGNATURE

```
train_stay_model(train_features, train_target, held_out_features, held_out_target)
```

PARAMETERS

train_features — **pandas.DataFrame**. The training stays, laid out by **build_feature_frame** from **config.STAY_NUMERIC_FEATURES** and **config.STAY_CATEGORICAL_FEATURES**, passed by **run_model_training.py**.
train_target — **pandas.Series**. The nights those stays lasted.
held_out_features — **pandas.DataFrame**. The held-out stays, built the same way.
held_out_target — **pandas.Series**. The nights those stays lasted.

PURPOSE

Estimate the nights a member stays in hospital after an operation.

REQUIREMENTS

- The model is a pipeline from **build_model_pipeline**, fitted on the training rows only.
- Its held-out R squared reaches **config.MINIMUM_STAY_R2**, its average miss is below **0.75** times that of always answering the training average, and no held-out forecast is below **config.STAY_NIGHTS_MINIMUM**.
- Each of these lengthens the forecast for at least **90%** of the held-out stays it is applied to: an ASA class of **4** rather than **2**, by at least **1** night on average; both knees rather than one, for the knee replacements, by at least **1** night; a hospital without an enhanced recovery programme rather than with one, by at least **0.5** night.
- Its scores come from **evaluate_regressor**.

RETURNS

tuple[sklearn.pipeline.Pipeline, dict]. The fitted model and its scores.

MODULE: modeling/evidence_model.py

FUNCTION SIGNATURE

```
train_evidence_model(train_text, train_target, held_out_text, held_out_target)
```

PARAMETERS

train_text — **pandas.Series**. The training share of the clean sentences, from the split **run_model_training.py** makes.
train_target — **pandas.Series**. Their evidence types.
held_out_text — **pandas.Series**. The held-out share, from the same split.
held_out_target — **pandas.Series**. Their evidence types.

PURPOSE

Name the kind of evidence a sentence gives.

REQUIREMENTS

- The model is one pipeline that takes raw sentences, as a list or a Series of strings, with the turning of words into numbers inside it and the estimator as its last step, named **config.MODEL_STEP_NAME**; it is fitted on the training sentences only.
- Its classes are the nine distinct values of **config.EVIDENCE_TYPE_MAPPING**, no more and no fewer, and any text, however strange, is read as one of them.
- Its held-out macro F1 reaches **config.MINIMUM_EVIDENCE_MACRO_F1**; every evidence type is predicted for some held-out sentence, and at least **75%** of the held-out sentences of each type are read as that type.
- Its scores come from **evaluate_text_classifier**.

RETURNS

tuple[sklearn.pipeline.Pipeline, dict]. The fitted model and its scores.

========== THE KNOWLEDGE BASE ==========

The sixteen documents are the only source of any criterion, rule, check or answer the system gives. They are read once, divided into their numbered sections, cut into passages and stored so that they can be searched by meaning. A service's criteria are read from its own policy alone.

MODULE: knowledge/pdf_extraction.py

Gets the text out of the PDFs and does nothing else with it.

FUNCTION SIGNATURE

```
extract_document_text(file_path)
```

PARAMETERS

file_path — **str**. The path of one PDF, passed by **extract_text_from_directory** or by **read_medical_policy**.

PURPOSE

Read the whole text of one document.

REQUIREMENTS

- Every page's text is present, in page order, one page to a line break.
- A PDF with no pages, and a PDF with a page that gives up no text, are refused, naming the file.

RETURNS

str. The document's text.

FUNCTION SIGNATURE
`extract_text_from_directory(docs_directory)`
PARAMETERS
docs_directory — **str**. The folder holding the documents, passed by **run_knowledge_indexing.py** from **config.DOCS_DIRECTORY**.
PURPOSE
Read every document in a folder once, for indexing.
REQUIREMENTS

- Every file whose name ends in **config.PDF_FILE_SUFFIX**, whatever its case, is read, and nothing else in the folder is.
- Each entry carries **document_name**, the file name as it is on disk, and **document_text**, the whole document; the entries come back in name order.
- A folder with no PDF is refused, naming the folder.

RETURNS
list[dict]. One entry per document.

MODULE: knowledge/chunk_splitting.py
Cuts the documents into passages small enough for a search to return alone and whole enough that a criterion is never cut in half.

FUNCTION SIGNATURE
`split_documents_into_sections(document_entries)`
PARAMETERS
document_entries — **list[dict]**. The documents as **extract_text_from_directory** returned them, passed by **run_knowledge_indexing.py**.
PURPOSE
Divide each document at its numbered headings, so that a passage never spans two subjects.
REQUIREMENTS

- No section carries the document's one header line, recognised by **config.DOCUMENT_HEADER_PATTERN**; a document with no header line or more than one is refused, naming the document.
- The sections follow the headings **config.SECTION_HEADING_PATTERN** recognises, numbered from **1** in order; a document laid out otherwise is refused, naming the document and the numbers found.
- Each entry carries **document_name** and **section_text**, every run of whitespace a single space; each section after the first opens with its heading, and the first holds everything before the second heading.

RETURNS
list[dict]. One entry per section, in document order.

FUNCTION SIGNATURE
`build_chunk_records(section_entries)`
PARAMETERS
section_entries — **list[dict]**. The sections as **split_documents_into_sections** returned them, passed by **run_knowledge_indexing.py**.
PURPOSE
Cut each section into passages a search can return, overlapping enough that a sentence near a boundary is still whole in one of them.
REQUIREMENTS

- A passage is made of whole sentences of one section, divided by **config.SENTENCE_BOUNDARY_PATTERN**, and is at most **config.CHUNK_SIZE_CHARACTERS** long; it never draws text from two sections or two documents.
- Each passage after the first of its section opens with the last sentences of the passage before it, up to **config.CHUNK_OVERLAP_CHARACTERS**.
- Every section gives at least one passage.
- Each record carries **chunk_id**, **chunk_text** and **document_name**; the identifier is unique across the set and the same when the same documents are cut again.

RETURNS
list[dict]. One record per passage.

MODULE: knowledge/vector_index.py
Turns passages into vectors, stores them, and searches them. This is the only module that talks to the vector store.

FUNCTION SIGNATURE
`build_vector_index(chunk_records, store_path)`
PARAMETERS
chunk_records — **list[dict]**. The passages as **build_chunk_records** returned them, passed by **run_knowledge_indexing.py**.
store_path — **str**. The folder the store is kept in, passed from **config.VECTOR_STORE_DIRECTORY**.
PURPOSE
Build the searchable store the whole system reads from.
REQUIREMENTS

- A file given as the folder, and a folder that holds files but no **config.VECTOR_STORE_FILE_NAME**, are refused, naming the path.
- The store is persistent, holds the collection **config.VECTOR_COLLECTION_NAME**, embeds with **config.EMBEDDING_MODEL_NAME** on **config.EMBEDDING_DEVICE**, and compares by **config.VECTOR_DISTANCE_METRIC** distance.
- Every passage is stored once, under its **chunk_id**, with its text and with **document_name** as its metadata.
- Building again replaces the store rather than adding to it; if the embedding model cannot be loaded, the existing store is left as it was.

RETURNS
int. How many passages the collection holds.

FUNCTION SIGNATURE
`open_vector_index(store_path)`

PARAMETERS
store_path — **str**. The folder **build_vector_index** wrote to, passed from **config.VECTOR_STORE_DIRECTORY**.
PURPOSE
Open the store for searching, without changing it.
REQUIREMENTS
- A folder with no **config.VECTOR_STORE_FILE_NAME**, and a store with no **config.VECTOR_COLLECTION_NAME** collection, are refused, naming the path.
- The collection comes back with the embedding it was built with; nothing is rebuilt or written.

RETURNS
chromadb.Collection. The knowledge base, ready to search.

FUNCTION SIGNATURE
`find_section_passage(collection, question, document_name, section_heading)`

PARAMETERS
collection — **chromadb.Collection**. The knowledge base, as **open_vector_index** returned it, passed by a function of **tools/retrieval_tools.py**.
question — **str**. The question to ask, passed from config, for example **config.DOCUMENTATION_QUESTION**.
document_name — **str**. The one document to search, passed from config.
section_heading — **str**. The heading the section opens with, passed from config, for example **config.DOCUMENTATION_SECTION_HEADING**.
PURPOSE
Read one section of one document, whole.
REQUIREMENTS
- The nearest passage of that document alone is taken.
- A document with no passage is refused with **config.INDEX_HOLDS_NO_CHUNKS_MESSAGE**, and a passage that does not open with the heading is refused with **config.PASSAGE_OF_ANOTHER_SECTION_MESSAGE**.
- The passage comes back without its heading.

RETURNS
str. The section's text.

FUNCTION SIGNATURE
`search_documents(collection, query_texts)`

PARAMETERS
collection — **chromadb.Collection**. The knowledge base, passed by **answer_from_documents**.
query_texts — **list[str]**. One or more questions asked together.
PURPOSE
Ask the store several questions at once for the chat, and get back the nearest passages with how far each one is.
REQUIREMENTS
- One list of passages comes back for each question, in the order the questions were asked, each holding **config.CHAT_RETRIEVAL_RESULT_COUNT** passages, nearest first.
- No passage of **config.CHAT_EXCLUDED_DOCUMENTS** is ever returned.
- Each passage carries **chunk_id**, **chunk_text**, **document_name** and **distance**, a plain number.

RETURNS
list[list[dict]]. The passages, question by question.

MODULE: knowledge/policy_reading.py
Reads a service's criteria out of that service's own medical policy. This is the module that keeps the criteria out of the code.

FUNCTION SIGNATURE
`read_medical_policy(service_code)`

PARAMETERS
service_code — **str**. The service whose policy is wanted, one of the keys of **config.SERVICE_POLICY_DOCUMENTS**, passed by **run_model_training.py**, by the workflow's context or by the chat.
PURPOSE

Establish every criterion of one service, with its own figures, from its own policy.
REQUIREMENTS

- A service with no configured policy is refused with **config.NO_POLICY_CONFIGURED_MESSAGE**. The policy is read from its configured document under **config.DOCS_DIRECTORY**, and from no other.
- The document's text is read with every run of **config.WHITESPACE_RUN_PATTERN** as one space. It must state its own service code once and no other, as **config.POLICY_SERVICE_CODE_PATTERN** finds them; must hold its coverage criteria section, as **config.CRITERIA_SECTION_PATTERN** finds it; and must number its criteria from **1** in order, as **config.CRITERION_NUMBER_PATTERN** divides them. A document that fails any of these is refused, naming it.
- Each criterion is read whole with the one sentence form of **config.CRITERION_SENTENCE_PATTERNS** that fits it exactly, its name and its sentence being the parts **config.CRITERION_NAME_PART** and **config.CRITERION_SENTENCE_PART**. A criterion no form fits, a kind stated twice, and a figure config gives no type, are refused.
- Each criterion carries **criterion_kind**, **criterion_name** as the policy writes it, **criterion_sentence** word for word in that text, and **rules**: the whole-number figures of **config.WHOLE_NUMBER_RULES** as integers, the body mass index figures of **config.BODY_MASS_INDEX_RULES** as exact decimals, and the listed names of **config.LISTED_NAME_MAPPINGS**, divided by **config.LISTED_NAME_SEPARATOR_PATTERN**, as canonical names in the policy's order; a listed name the plan does not know is refused.
- For the criterion of **config.CONSERVATIVE_TREATMENT_OR_RED_FLAG_CRITERION**, the red flags that waive the treatment are read from the policy's one list of them, as **config.RED_FLAG_LIST_PATTERN** finds it, and kept under **config.LISTED_RED_FLAGS_RULE**; a list whose count word, read through **config.NUMBER_WORD_VALUES**, is not the number of names listed is refused.
- A policy reads the same every time.

RETURNS
dict. **service_code**, **document_name** and **criteria**, in the policy's own order.

========== AGENT TOOLS ==========

The seven modules below are the boundary between what a policy, a model or a document says and what the review records. Everything a language model is later told is assembled here, as plain facts, by ordinary code that can be read and reproduced. No agent works out any of these results or figures for itself.

MODULE: tools/request_data_tools.py
Reads the live queue and its documents through the same cleaning as the extracts, and serves one request at a time.

FUNCTION SIGNATURE
`load_open_requests()`

PARAMETERS
None.
PURPOSE
Read the live queue once, as the Prior Authorisation Rules read it.
REQUIREMENTS

- **config.OPEN_REQUESTS_FILE** is read and cleaned as **clean_request_records** cleans it, with **config.REQUEST_VALUE_MAPPINGS** and **config.REQUEST_WHOLE_NUMBER_PATTERNS**.
- Each open request appears once, as its latest submission.

RETURNS
pandas.DataFrame. The open requests.

FUNCTION SIGNATURE
`load_request_notes()`

PARAMETERS
None.
PURPOSE
Read the documents sent with the open requests once.
REQUIREMENTS

- **config.REQUEST_NOTES_FILE** is read and cleaned as **clean_note_records** cleans it, with **config.NOTE_VALUE_MAPPINGS**, **config.NOTE_WHOLE_NUMBER_PATTERNS**, **config.NOTE_ID_COLUMN** and **config.NOTE_TEXT_COLUMN**.
- Every document carries a note number, and no request has two documents with the same number; otherwise the file is refused.

RETURNS
pandas.DataFrame. The documents.

FUNCTION SIGNATURE
`fetch_request(open_requests, request_reference)`

PARAMETERS
open_requests — **pandas.DataFrame**. The live queue as **load_open_requests** returned it, held in memory by the caller.
request_reference — **str**. The canonical reference of the request wanted.
PURPOSE
Give one request's own answers to whatever is about to reason about it.
REQUIREMENTS

- The record holds every column the queue carries for that request.
- A reference that is not open is refused with **config.UNKNOWN_REQUEST_MESSAGE**, and a request that gives no service code with **config.NO_SERVICE_CODE_MESSAGE**.

RETURNS
dict. The request, column by column.

FUNCTION SIGNATURE
`fetch_request_documents(request_notes, request)`

PARAMETERS
request_notes — **pandas.DataFrame**. The documents as **load_request_notes** returned them, held in memory by the caller.
request — **dict**. The request, as **fetch_request** returned it.
PURPOSE
Give the documents sent with one request, in the order the office numbered them.
REQUIREMENTS

- Only that request's documents come back, all of them, in **note_number** order whatever order they stand in the file.
- Where the request gives a **supporting_document_count**, the documents that arrived must number exactly that; otherwise the request is refused, naming it and both counts.

RETURNS
list[dict]. One record per document, every column of the file.

FUNCTION SIGNATURE
`rank_review_queue(open_requests)`

PARAMETERS
open_requests — **pandas.DataFrame**. The live queue, or any table of requests with canonical references, passed by **api/routes.py**.
PURPOSE
Put the queue in the order the Utilisation Management Policy sets.
REQUIREMENTS

- Requests whose urgency is **config.URGENT** come first, and then every other request, a blank urgency waiting as standard; each group is in request-reference order.
- Every request appears once.

RETURNS
list[str]. The request references in queue order.

FUNCTION SIGNATURE
`canonical_request_reference(written_reference)`

PARAMETERS
written_reference — **str**. A request reference as a person or a model wrote it, passed by **answer_from_documents**.
PURPOSE
Write a request reference canonically where it can be read.
REQUIREMENTS

- A reference in a notation **config.REQUEST_REFERENCE_PATTERN** allows comes back as **config.CANONICAL_REQUEST_REFERENCE_FORMAT** writes it, **PA-** and six digits; any other text comes back as written.

RETURNS
str. The reference.

FUNCTION SIGNATURE
`describe_request(request)`

PARAMETERS
request — **dict**. A request's columns, passed by an agent, a letter tool or the service.
PURPOSE
Give a request in the words a page and a prompt show.
REQUIREMENTS

- Every column comes back as text, and a missing value as **config.NOT_GIVEN_TEXT**.

RETURNS
dict. The request, column by column, as text.

MODULE: tools/screening_tools.py

FUNCTION SIGNATURE
`screen_request(request, policy)`

PARAMETERS
request — **dict**. One clean request, from the live queue or the history, passed by the workflow's context, by **add_criteria_counts** or by the service.
policy — **dict**. A policy as **read_medical_policy** returns it, or any policy of the same shape.
PURPOSE
Screen a request's questionnaire against every criterion of its policy, as the Prior Authorisation Rules set out.

REQUIREMENTS

- A policy whose **service_code** is not the request's is refused with **config.POLICY_OF_ANOTHER_SERVICE_MESSAGE**.
- Every criterion of the policy appears once, in the policy's order, with **criterion_kind**, **criterion_name**, **criterion_sentence**, **result** and **missing_answers**.
- Every result follows **MET, NOT MET AND NOT DOCUMENTED** above, judged against the figures of the policy handed over and never a figure written into the code. Each kind of **config.CRITERION_SENTENCE_PATTERNS** is judged on the answers **config.CRITERION_ANSWER_COLUMNS** gives it, read from **config.SCREENING_ANSWER_COLUMNS**; a psychological assessment answered **config.PSYCHOLOGY_COMPLETED** is Met, and a home test result among **config.HOME_TESTS_SHOWING_LABORATORY_NEED** shows a need for the laboratory. A criterion kind the screening does not know is refused, naming the document and the kind.
- **body_mass_index** is worked out exactly, the height turned to metres by **config.CENTIMETRES_PER_METRE**, and kept half up to **config.MEASUREMENT_STEP**, as an exact decimal; it is nothing when the height or the weight is blank.
- **screening** and **decisions_open** follow **THE SCREENING AND THE DECISIONS OPEN** above, and the three counts are given under **config.CRITERIA_COUNT_COLUMNS**.

RETURNS

dict. **request_reference**, **service_code**, **policy_document**, **body_mass_index**, **criteria**, the three counts, **screening** and **decisions_open**.

MODULE: tools/evidence_tools.py

FUNCTION SIGNATURE

```
read_document_evidence(evidence_model, documents)
```

PARAMETERS

evidence_model — **sklearn.pipeline.Pipeline**. The fitted evidence model, loaded from **config.EVIDENCE_MODEL_FILE**.
documents — **list[dict]**. A request's documents as **fetch_request_documents** returned them.

PURPOSE

Read every sentence of a request's documents for the evidence it gives.

REQUIREMENTS

- Each document is split into its sentences with **config.DOCUMENT_SENTENCE_PATTERN**, in the order written, and the documents in the order given; each sentence is kept whole, as written.
- Each finding carries **note_id**, **note_author_role**, **sentence** and **evidence_type**, the evidence model's reading, as text.
- A document with no text is refused, naming it; no documents give no findings.

RETURNS

list[dict]. One finding per sentence.

FUNCTION SIGNATURE

```
flag_document_disagreements(evidence_findings, request, policy)
```

PARAMETERS

evidence_findings — **list[dict]**. The request's findings, as **link_evidence_to_criteria** returned them.
request — **dict**. The request, as **fetch_request** returned it.
policy — **dict**. The request's own policy.

PURPOSE

Flag every sentence of conservative treatment whose weeks are not the questionnaire's.

REQUIREMENTS

- A policy of another service is refused with **config.POLICY_OF_ANOTHER_SERVICE_MESSAGE**, and a policy that asks no weeks of treatment gives no flags.
- The weeks a **config.CONSERVATIVE_TREATMENT_EVIDENCE** sentence states are those **config.STATED_WEEKS_PATTERN** finds in it, in any case, as **READING A CLINICAL DOCUMENT** above describes: in digits, a decimal giving its whole weeks, or in words. Of the words found, separated as **config.NUMBER_WORD_SEPARATOR_PATTERN** finds, a word of **config.TENS_WORDS** followed by one of **config.UNIT_WORDS** gives their sum, and otherwise the last word gives its value, each through **config.NUMBER_WORD_VALUES**; a word found before the number, as in "completed eight weeks", is no part of it, and words ending in no number state none. A sentence that states none is not flagged.
- A sentence is flagged **config.DESCRIBES_BLANK_ANSWER** when the questionnaire's weeks are blank, and **config.DISAGREES_WITH_QUESTIONNAIRE** when they are not among the weeks it states.
- Each disagreement is the finding with **stated_weeks**, **questionnaire_weeks** (nothing when blank), **flag** and **disagreement_text**, written with **config.DISAGREEMENT_WITH_BLANK_ANSWER_TEMPLATE** or **config.DISAGREEMENT_TEMPLATE**, several stated weeks joined by **config.DISAGREEMENT_WEEKS_SEPARATOR**.

RETURNS

list[dict]. One entry per flagged sentence.

FUNCTION SIGNATURE

```
link_evidence_to_criteria(evidence_findings, screening)
```

PARAMETERS

evidence_findings — **list[dict]**. The findings as **read_document_evidence** returned them.
screening — **dict**. The request's screening, as **screen_request** returned it.

PURPOSE

Record, for every sentence, the criteria of the request's own policy its evidence bears on.
REQUIREMENTS

- Every finding comes back, in order, with **criteria_borne_on** added: the names of the screening's criteria whose kind **config.EVIDENCE_CRITERION_KINDS** lists for its evidence type, in the policy's order.
- An evidence type that is not a key of **config.EVIDENCE_CRITERION_KINDS** is refused with **config.NO_EVIDENCE_CRITERIA_MESSAGE**; a type listed with no kinds bears on none.

RETURNS
list[dict]. The findings, each with the criteria it bears on.

MODULE: tools/prediction_tools.py

FUNCTION SIGNATURE
`estimate_approval_likelihood(approval_model, feature_names, request, screening)`

PARAMETERS
approval_model — **sklearn.pipeline.Pipeline**. The fitted approval model, loaded from **config.APPROVAL_MODEL_FILE**.
feature_names — **list[str]**. Its saved schema, loaded from **config.APPROVAL_SCHEMA_FILE**.
request — **dict**. The request, as **fetch_request** returned it.
screening — **dict**. That request's screening.
PURPOSE
Give the request's approval likelihood and its band.
REQUIREMENTS

- A screening of another request is refused with **config.SCREENING_OF_ANOTHER_REQUEST_MESSAGE**.
- The model is given the request with the screening's three counts, laid out by **build_feature_frame**; inputs that are not **feature_names**, in order, are refused, naming both.
- **approval_likelihood** is the model's probability of approval rounded to **config.LIKELIHOOD_DECIMALS** places, and **approval_band** is cut on that rounded figure.

RETURNS
dict. **approval_likelihood** and **approval_band**.

FUNCTION SIGNATURE
`estimate_expected_stay(stay_model, feature_names, request, screening)`

PARAMETERS
stay_model — **sklearn.pipeline.Pipeline**. The fitted stay model, loaded from **config.STAY_MODEL_FILE**.
feature_names — **list[str]**. Its saved schema, loaded from **config.STAY_SCHEMA_FILE**.
request — **dict**. An inpatient request, as **fetch_request** returned it.
screening — **dict**. That request's screening.
PURPOSE
Give the expected stay of an inpatient operation, and whether it is longer than the days requested.
REQUIREMENTS

- A screening of another request is refused with **config.SCREENING_OF_ANOTHER_REQUEST_MESSAGE**; a request with no site of service with **config.NO_SITE_OF_SERVICE_MESSAGE**; a request not at **config.INPATIENT_SITE** with **config.NOT_AN_INPATIENT_OPERATION_MESSAGE**.
- The model is given the request with the screening's body mass index, the number of its other conditions as **comorbidity_count**, **config.NO_OTHER_CONDITIONS** counting as none, and **diabetes_recorded** as **config.BOOLEAN_YES** when **config.DIABETES_CONDITION** is among them and **config.BOOLEAN_NO** otherwise; with the conditions blank, both are blank. Inputs that are not **feature_names**, in order, are refused.
- **expected_stay_days** is rounded to **config.STAY_DECIMALS** place; **requested_inpatient_days** is a whole number or nothing; **longer_stay_expected** is true when that rounded figure is more than **config.LONGER_STAY_MARGIN_DAYS** above the days requested, and nothing when none were requested.

RETURNS
dict. **expected_stay_days**, **requested_inpatient_days** and **longer_stay_expected**.

FUNCTION SIGNATURE
`describe_expected_stay(stay, site_of_service)`

PARAMETERS
stay — **dict**. The stay, as **estimate_expected_stay** returned it, or empty where none was worked out.
site_of_service — **str** | **None**. The request's site of service.
PURPOSE
Write the stay in the one wording every agent and page uses.
REQUIREMENTS

- A blank site gives **config.STAY_WITHOUT_SITE_TEXT**, and an empty stay **config.STAY_NOT_INPATIENT_TEXT**.
- Otherwise the stay, to **config.STAY_DECIMALS** place, is written with **config.STAY_WITHOUT_REQUESTED_DAYS_TEMPLATE**, **config.LONGER_STAY_TEMPLATE** or **config.NO_LONGER_STAY_TEMPLATE**, as the stay requires.

RETURNS
str. The stay in words.

MODULE: tools/retrieval_tools.py
Each function reads one section of one document, whole, through find_section_passage.

FUNCTION SIGNATURE
`read_documentation_to_send(collection, service_code)`
PARAMETERS
collection — **chromadb.Collection**. The knowledge base, as **open_vector_index** returned it.
service_code — **str**. A service code of **config.SERVICE_POLICY_DOCUMENTS**.
PURPOSE
Read what a service's policy asks the requesting office to send.
REQUIREMENTS
- A service with no configured policy is refused with **config.NO_POLICY_CONFIGURED_MESSAGE**.
- **config.DOCUMENTATION_QUESTION** is asked of that policy alone, and its section under **config.DOCUMENTATION_SECTION_HEADING** comes back whole, without its heading.

RETURNS
str. The documentation to send.

FUNCTION SIGNATURE
`read_reviewer_checks(collection, evidence_type)`
PARAMETERS
collection — **chromadb.Collection**. The knowledge base.
evidence_type — **str**. An evidence type of **config.EVIDENCE_GUIDE_DOCUMENTS**.
PURPOSE
Read the checks an evidence guide sets the reviewer.
REQUIREMENTS
- An evidence type with no guide is refused with **config.NO_EVIDENCE_GUIDE_MESSAGE**.
- **config.REVIEWER_CHECKS_QUESTION** is asked of that guide alone, and its section under **config.REVIEWER_CHECKS_SECTION_HEADING** comes back whole, without its heading.

RETURNS
str. The reviewer's checks.

FUNCTION SIGNATURE
`read_office_guidance(collection, evidence_type)`
PARAMETERS
collection — **chromadb.Collection**. The knowledge base.
evidence_type — **str**. An evidence type of **config.EVIDENCE_GUIDE_DOCUMENTS**.
PURPOSE
Read what an evidence guide says to tell the requesting office.
REQUIREMENTS
- An evidence type with no guide is refused with **config.NO_EVIDENCE_GUIDE_MESSAGE**.
- **config.OFFICE_MESSAGE_QUESTION** is asked of that guide alone, and its section under **config.OFFICE_MESSAGE_SECTION_HEADING** comes back whole, without its heading.

RETURNS
str. The message to the office.

MODULE: tools/letter_tools.py
The details a letter shows, added from the request itself so that no model writes them.

FUNCTION SIGNATURE
`build_authorisation_details(request, policy_document)`
PARAMETERS
request — **dict**. The approved request.
policy_document — **str**. The policy it was reviewed under, from its screening.
PURPOSE
Give the details a provider authorisation letter shows.
REQUIREMENTS
- The details are keyed **config.REQUEST_REFERENCE_DETAIL**, **config.MEMBER_NUMBER_DETAIL**, **config.SERVICE_DETAIL**, **config.SITE_OF_SERVICE_DETAIL** and **config.MEDICAL_POLICY_DETAIL**, and, for a request at **config.INPATIENT_SITE** only, **config.INPATIENT_DAYS_DETAIL**.
- Each value is the request's own, as **describe_request** writes it, except the medical policy, which is **policy_document**, and the service, written with **config.SERVICE_WITH_CODE_FORMAT** from its name in **config.SERVICE_NAMES** and its code.

RETURNS
dict. The letter's details.

FUNCTION SIGNATURE

`build_information_request_details(request, screening, documentation_to_send)`

PARAMETERS
request — **dict**. The request information is asked for.
screening — **dict**. Its screening.
documentation_to_send — **str**. Its policy's documentation to send, as **read_documentation_to_send** returned it.
PURPOSE
Give the details an information request letter shows.
REQUIREMENTS

- The details are keyed **config.REQUEST_REFERENCE_DETAIL**, **config.SERVICE_DETAIL**, **config.CRITERIA_NOT_DOCUMENTED_DETAIL**, **config.DOCUMENTATION_TO_SEND_DETAIL**, **config.REPLY_WITHIN_DETAIL** and **config.UNANSWERED_REQUEST_DETAIL**.
- The reference is the request's own and the service is written as for an authorisation. **config.CRITERIA_NOT_DOCUMENTED_DETAIL** is a list with one entry for each criterion Not documented: its name, a colon and a space, and the labels **config.QUESTIONNAIRE_ANSWER_LABELS** gives its blank answers, joined with a semicolon and a space.
- The time to reply is **config.URGENT_REPLY_TEXT** for an urgent request and **config.STANDARD_REPLY_TEXT** otherwise, and the last detail is **config.UNANSWERED_REQUEST_TEXT**.

RETURNS
dict. The letter's details.

MODULE: tools/decision_tools.py
FUNCTION SIGNATURE
`find_decision_refusal(review_decision, screening, request, coordinator_id)`
PARAMETERS
review_decision — **dict**. The decision as it would be recorded, with **decision**, **decided_by**, **decided_by_role** and **decision_reason**, passed by the gate or by the service.
screening — **dict**. The request's screening, as the run holds it.
request — **dict**. The request, as the run holds it.
coordinator_id — **str**. The username of the person who opened the review.
PURPOSE
Say why a decision may not be recorded, before anything is recorded or spent.
REQUIREMENTS

- The reasons are tried in this order, and the first that applies is given: a field of **config.DECISION_RECORD_FIELDS** missing, not text or blank; a decision none of **config.DECISIONS**; a role other than **config.ROLE_NURSE**; the person who opened the review, compared without regard to case or surrounding space; a decision the screening does not leave open; an approval whose letter would lack the site of service, or the inpatient days of an inpatient operation.
- Their messages are, in the same order, **config.DECISION_FIELDS_MISSING_MESSAGE**, **config.UNKNOWN_DECISION_MESSAGE**, **config.DECIDING_ROLE_MESSAGE**, **config.OWN_REVIEW_MESSAGE**, **config.DECISION_NOT_OPEN_MESSAGE** with the reason **config.SCREENING_CLOSED_REASONS** gives for the screening, and **config.APPROVAL_LETTER_INCOMPLETE_MESSAGE** with the blank answers named by **config.AUTHORISATION_ANSWER_LABELS**; each is filled with the request's reference and what it refuses, fields and decisions listed with a comma and a space and blank answers joined with **or**.
- A decision that may be recorded gives nothing.

RETURNS
str | None. Why the decision may not be recorded, or nothing.

========== PERSISTENCE ==========

Six tables hold everything the system is later asked to account for: who signed in, what ran, what was reviewed, what a UM Nurse Reviewer decided, what every language model call cost, and every question the chat was asked. Nothing is written anywhere else.

MODULE: persistence/database.py
The connection and the session factory. It defines no function: two module-level objects are built once, when the module is first imported, and everything else in the project uses them.
MODULE-LEVEL OBJECTS

- **database_engine** — the connection to the database at **config.APPLICATION_DATABASE_URL**, usable from more than one thread, because the web framework answers on threads of its own.
- **session_factory** — bound to that engine, and the only way the rest of the project opens a session.

REQUIREMENTS

- Both are created when the module is imported, and no other engine is created anywhere in the project.
- Importing this module neither creates tables nor writes anything; creating the tables is the work of **run_database_setup.py**.

MODULE: persistence/models.py

Declares the six tables. Table and column names are read by the marking suite and by the service's replies, and must be reproduced exactly, in lower case.

CLASS

`Base`

The declarative base every table below inherits from, and the object **run_database_setup.py** uses to create them all.

TABLE

user_accounts

CLASS

`UserAccount`

COLUMNS

Column	Type	Notes
username	**str**	The key
display_name	**str**	The name a page shows
role	**str**	One of the three roles
password_hash	**str**	Never the password itself
password_salt	**str**	This account's own salt
session_token	**str | None**	The current session, or nothing when signed out
created_at	**datetime**	Set by the database

TABLE

agent_runs

CLASS

`AgentRun`

COLUMNS

Column	Type	Notes
run_id	**str**	The key
thread_id	**str**	The identifier the paused workflow is stored under
request_reference	**str**	The request being reviewed
coordinator_id	**str**	The username of the person who opened the review
coordinator_request	**str**	What they asked to be looked at, as long text
status	**str**	One of **config.AGENT_RUN_STATUS_RUNNING**, **config.AGENT_RUN_STATUS_AWAITING_DECISION**, **config.AGENT_RUN_STATUS_COMPLETE** and **config.AGENT_RUN_STATUS_FAILED**
llm_call_count	**int**	Calls the run has made, nought until it makes one
started_at	**datetime**	Set by the database
finished_at	**datetime | None**	Set by the database when the run completes or fails

TABLE

authorisation_reviews

CLASS

`AuthorisationReview`

COLUMNS

Column	Type	Notes
review_id	**str**	The key
run_id	**str**	The run that produced it
request_reference	**str**	The request
service_code	**str**	Its service
service_name	**str**	The service's name
policy_document	**str**	The policy it was screened under
urgency	**str | None**	Its urgency, nothing when the office left it blank
site_of_service	**str | None**	Its site of service, nothing when blank
coordinator_request	**str**	What the Intake Coordinator asked, as long text
criteria	**list**	Every criterion with its result, as JSON
screening	**str**	The screening
decisions_open	**list**	The decisions open, as JSON
body_mass_index	**float | None**	The index the screening worked out
evidence	**list**	Every finding with the criteria it bears on, as JSON
disagreements	**list**	Every disagreement, as JSON
approval_likelihood	**float**	The likelihood
approval_band	**str**	Its band
expected_stay_days	**float | None**	The expected stay, for an inpatient operation
requested_inpatient_days	**int | None**	The days requested
longer_stay_expected	**bool | None**	Whether a longer stay is expected
stay_text	**str**	The stay in words, as long text
review_plan_text	**str**	The planner's write-up, as long text
criteria_analysis_text	**str**	The criteria analyst's write-up, as long text
evidence_review_text	**str**	The evidence analyst's write-up, as long text
prediction_analysis_text	**str**	The likelihood analyst's write-up, as long text
recommendation_text	**str**	The writer's recommendation, as long text
recommended_decision	**str**	One of the decisions open
status	**str**	One of **config.REVIEW_STATUS_OPTIONS**
letter_text	**str | None**	The letter, where one was written, as long text
letter_details	**dict | None**	The letter's details, as JSON, and nothing where no letter was written
notice_text	**str | None**	The member notice, where one was written, as long text
referral_summary	**dict | None**	The referral summary, as JSON, and nothing unless referred
created_at	**datetime**	Set by the database
status_changed_at	**datetime**	Set by the database, and again when the outcome is recorded

TABLE
review_decisions
CLASS
`ReviewDecision`
COLUMNS

Column	Type	Notes
decision_id	**int**	The key, assigned by the database
review_id	**str**	The review decided
run_id	**str**	The run it belongs to
decision	**str**	One of **config.DECISIONS**
decided_by	**str**	The deciding nurse's username
decided_by_role	**str**	Their role
decision_reason	**str**	Their own words, as long text
decided_at	**datetime**	Set by the database

TABLE
llm_calls
CLASS

`LlmCall`

COLUMNS

Column	Type	Notes
call_id	**int**	The key, assigned by the database
run_id	**str**	The run that made the call
agent_name	**str**	Which agent made it
model_name	**str**	Which model answered
prompt_characters	**int**	How much was sent
response_characters	**int**	How much came back
latency_milliseconds	**int**	How long it took
called_at	**datetime**	Set by the database

TABLE
chat_turns
CLASS
`ChatTurn`

COLUMNS

Column	Type	Notes
turn_id	**int**	The key, assigned by the database
username	**str**	Who asked
user_role	**str**	Their role at the time
question_text	**str**	What they asked, with every member number withheld, as long text
question_type	**str**	One of **config.CHAT_TYPE_OPTIONS**
request_reference	**str | None**	The request the question is about, if any
answer_text	**str**	What they were told, refusal included, as long text
source_documents	**str**	The documents the answer rested on, written as text
closest_distance	**float**	How far the nearest passage was
was_refused	**bool**	Whether the answer was a refusal
asked_at	**datetime**	Set by the database

REQUIREMENTS

- Every column above exists under exactly that name, and only a column marked as able to be empty accepts nothing.
- Timestamps marked as set by the database are set by the database itself.

MODULE: persistence/repository.py
Every read and write against the six tables. Nothing outside this module builds a query. Each function works inside the session its caller opened from **session_factory**: a write joins the caller's transaction, which the caller commits as one unit, so that what belongs together is saved together. A read returns plain records, never database objects.

FUNCTION SIGNATURE
`insert_user_account(session, account_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
account_record — **dict**. One account's columns, built by **run_database_setup.py**, with the password already in its stored form.
PURPOSE
Create one sign-in account.
REQUIREMENTS

- The row is written into the caller's transaction; only the columns handed in are set.

RETURNS
None.

FUNCTION SIGNATURE
`fetch_user_account(session, condition)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.

condition — **sqlalchemy** condition. A comparison on the **user_accounts** columns, written by the caller, for example on **username** when signing in or on **session_token** when guarding a request.
PURPOSE
Find one account.
REQUIREMENTS
- The record carries every column of the account, so a password can be checked or a role read without a second query.
- Nothing is returned when no account matches.

RETURNS
dict | None. The account as a plain record, or nothing.

FUNCTION SIGNATURE
```
update_session_token(session, username, session_token)
```
PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
username — **str**. Whose token to set.
session_token — **str | None**. The new token, or nothing to end the session.
PURPOSE
Start or end one person's session.
REQUIREMENTS
- The account's token becomes exactly what was handed in, and nothing else about it changes.
- An unknown username is refused, naming it.

RETURNS
None.

FUNCTION SIGNATURE
```
insert_agent_run(session, run_record)
```
PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
run_record — **dict**. The run's columns, built by **load_request_context_node**.
PURPOSE
Record that a run has started, before any agent is called.
REQUIREMENTS
- The row is added and flushed.

RETURNS
None.

FUNCTION SIGNATURE
```
update_agent_run(session, run_id, status_value, llm_call_count)
```
PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
run_id — **str**. Which run to update.
status_value — **str**. One of the four run statuses, passed from config.
llm_call_count — **int**. How many calls the run has made so far.
PURPOSE
Move a run on: to waiting at the gate, and later to complete or failed.
REQUIREMENTS
- The status and the call count become what was handed in.
- **finished_at** is set by the database when, and only when, the status is one of **config.AGENT_RUN_FINISHED_STATUSES**.
- An unknown run is refused, naming it.

RETURNS
None.

FUNCTION SIGNATURE
```
insert_llm_call(session, run_id, call_record)
```
PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
run_id — **str**. The run the call belongs to.
call_record — **dict**. The **call_record** that **call_language_model** returned.
PURPOSE
Record one language model call.
REQUIREMENTS
- One row per call, filed under **run_id**; the record itself carries no run identifier.

RETURNS
None.

FUNCTION SIGNATURE
```
fetch_llm_calls(session, run_id)
```

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
run_id — **str**. The run whose calls are wanted.
PURPOSE
Read back everything one run spent.
REQUIREMENTS

- Every call of that run, and no other run's, in the order they were made, lowest **call_id** first, each with **agent_name**, **model_name**, **prompt_characters**, **response_characters** and **latency_milliseconds**.

RETURNS
list[dict]. One record per call.

FUNCTION SIGNATURE
`insert_authorisation_review(session, review_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
review_record — **dict**. The review at the gate, built by **compose_review_recommendation_node**.
PURPOSE
Write the review a UM Nurse Reviewer will decide.
REQUIREMENTS

- The row is added and flushed.

RETURNS
None.

FUNCTION SIGNATURE
`fetch_review_records(session, condition)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
condition — **sqlalchemy** condition. A comparison on the **authorisation_reviews** columns, written by the caller, for example on **review_id** or **status**.
PURPOSE
Read reviews back with everything a page or a reply needs, in one query.
REQUIREMENTS

- Each record carries every column of the review, the run's **thread_id**, **coordinator_id** and **llm_call_count**, and the decision's **decision**, **decided_by**, **decided_by_role** and **decision_reason**, which are nothing until a decision is recorded.
- The newest review comes first, by **created_at** and then by **review_id**, highest first.

RETURNS
list[dict]. One record per review the condition matched.

FUNCTION SIGNATURE
`insert_review_decision(session, decision_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
decision_record — **dict**. The decision, built by **record_review_outcome_node**.
PURPOSE
Record the decision as its own row, because it is what an auditor asks about.
REQUIREMENTS

- One row per decision taken, written into the caller's transaction.

RETURNS
None.

FUNCTION SIGNATURE
`record_review_outcome(session, review_id, outcome_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
review_id — **str**. The review to complete.
outcome_record — **dict**. The fields the decision settled, built by **record_review_outcome_node**.
PURPOSE
Complete a review once its decision is taken and its letters are written.
REQUIREMENTS

- Exactly the fields handed in are set, and **status_changed_at** is set again by the database; everything else the review held is left as it was.
- An unknown review is refused, naming it.

RETURNS
None.

FUNCTION SIGNATURE
`insert_chat_turn(session, turn_record)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
turn_record — **dict**. One chat turn, built by the chat endpoint.
PURPOSE
Record one exchange with the authorisation assistant.
REQUIREMENTS
- Every turn is written, refusals included.

RETURNS
None.

FUNCTION SIGNATURE
`fetch_chat_turns(session, condition, turn_limit)`

PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by the caller.
condition — **sqlalchemy** condition. A comparison on the **chat_turns** columns: on **username** for one person's own turns, or one that holds for every row for the audit log.
turn_limit — **int**. At most how many turns to read.
PURPOSE
Read back a conversation, for the chat page, for the assistant's memory and for the audit log.
REQUIREMENTS
- The most recent turns come first, highest **turn_id** first, at most **turn_limit** of them, each with every column of the turn.

RETURNS
list[dict]. Newest first.

========== AGENTIC WORKFLOW ==========

Five agents review a request, and up to two more write what the decision calls for. Each agent is exactly one language model call. None of them works anything out: every criterion result, figure and passage an agent is shown was established by the policies, the models, the documents and the tools, and each prompt tells its agent to use only what it is given. A run makes **config.LLM_CALLS_BEFORE_DECISION** calls before the gate and, in all, the number **config.LLM_CALLS_BY_DECISION** gives for its decision.
The workflow stops before any decision is recorded and before any letter is written, and the pause is written to disk so that it survives a restart.

MODULE: agentic/state.py

Declares what the workflow carries from one node to the next, as a **typing.TypedDict**, so every node and the router read the state as a dictionary.

CLASS
`AuthorisationReviewState`

STATE KEYS

Key	Type	Written by	What it carries
request_reference	**str**	the caller	The request being reviewed
coordinator_request	**str**	the caller	What the person who opened the review asked to be looked at
coordinator_id	**str**	the caller	The username of whoever opened the review
thread_id	**str**	the caller	The identifier the pause is stored under
run_id	**str**	load_request_context_node	The run, as recorded in **agent_runs**
review_id	**str**	load_request_context_node	The review this run will write
request_profile	**dict**	load_request_context_node	The request, as **fetch_request** returned it
request_documents	**list**	load_request_context_node	Its documents, as **fetch_request_documents** returned them
screening	**dict**	load_request_context_node	The screening under the request's own policy
evidence_findings	**list**	load_request_context_node	Every sentence read, with the criteria it bears on
disagreements	**list**	load_request_context_node	Every disagreement flagged
likelihood	**dict**	load_request_context_node	The approval likelihood and its band
stay	**dict**	load_request_context_node	The expected stay, empty unless the request is for an inpatient operation
review_plan	**dict**	plan_review_node	The planner's reply
criteria_analysis	**dict**	assess_criteria_node	The criteria analyst's reply
evidence_review	**dict**	review_evidence_node	The evidence analyst's reply
prediction_analysis	**dict**	weigh_predictions_node	The likelihood analyst's reply
review_decision	**dict**	the gate	**decision**, one of **config.DECISIONS**; **decided_by**, the deciding nurse's username; **decided_by_role**, their role; **decision_reason**, their own words
office_letter	**dict**	the gate and draft_office_letter_node	The letter's fields, empty until one is written
letter_details	**dict**	the gate and draft_office_letter_node	The details the letter carries, empty until one is written
member_notice	**dict**	the gate and draft_member_notice_node	The notice's fields, empty until one is written
llm_call_count	**int**	every agent node	The calls made so far

REQUIREMENTS

- Every key above is declared, and nothing else is.
- **llm_call_count** is added to by each node that returns it rather than replaced; every other key is replaced by whichever node writes it.

MODULE: agentic/agents.py

The language model client, the reply parser, and the seven agents. Each agent function hands the model facts that are already established and asks for prose in a fixed layout of labelled fields, so that the reply can be read back reliably. Every prompt speaks for **config.ORGANISATION_NAME**.

The key and the model name are read from the environment, through **config.GEMINI_API_KEY_VARIABLE** and **config.GEMINI_MODEL_VARIABLE**, which are set in **.env** and loaded from **config.ENV_FILE** when the module is imported. No key or model name is written into the code.

MODULE-LEVEL OBJECTS

- **generation_settings** — **google.genai.types.GenerateContentConfig**. The one generation configuration every call uses, holding **config.GEMINI_MAX_OUTPUT_TOKENS** and **config.GEMINI_TEMPERATURE**, with automatic function calling turned off.

FUNCTION SIGNATURE

```
call_language_model(prompt_text, agent_name)
```

PARAMETERS

prompt_text — **str**. The whole prompt, assembled by an agent function or by **answer_from_documents**.

agent_name — **str**. The calling agent's name constant from config, for example **config.AGENT_NAME_PLANNER**.

PURPOSE

Make one language model call and measure what it cost.

REQUIREMENTS

- The key and the model name are read at the moment of the call; when either is missing the call is refused, naming both settings and **.env**.
- The reply is generated under **generation_settings**, and the client is open for the call alone.
- An empty reply, and a reply that did not finish normally, are refused, naming the agent and the model.
- **call_record** carries **agent_name**, **model_name**, **prompt_characters**, **response_characters** and **latency_milliseconds**, whole milliseconds measured around the call itself through **config.MILLISECONDS_PER_SECOND**.

RETURNS

dict. **response_text**, the reply with surrounding whitespace removed, and **call_record**.

FUNCTION SIGNATURE

```
parse_labelled_response(response_text, field_labels, agent_name)
```

PARAMETERS

response_text — **str**. What the model replied, as **call_language_model** returned it, passed by an agent function or by **answer_from_documents**.

field_labels — **list[str]**. The labels the prompt asked for, in order, in upper case.

agent_name — **str**. Which agent replied, used in the refusal.

PURPOSE

Read a labelled reply back into its fields, tolerantly enough to survive the way models decorate their answers.

REQUIREMENTS

- A line whose text before its first colon is one of **field_labels** starts that field, whatever its case, bold markers, the decoration **config.RESPONSE_DECORATION_EDGE_PATTERN** finds at its edges, or hyphens written for spaces.
- No field keeps a bold marker or surrounding whitespace, and the text on a label's own line carries none of **config.RESPONSE_DECORATION_CHARACTERS** at either end.
- The lines after a label, up to the next label, belong to that field, each with its surrounding whitespace removed and on a new line, and a line of decoration alone adds nothing; text before the first label is ignored; a label written again adds its text to the field on a new line rather than replacing it.
- A label that is missing, or answered with nothing, is refused, naming the agent and every such label.

RETURNS

dict. Exactly the labels asked for, in the order asked.

FUNCTION SIGNATURE

```
join_labelled_response(labelled_fields)
```

PARAMETERS

labelled_fields — **dict**. An agent's fields, as **parse_labelled_response** returned them, passed by **compose_review_recommendation_node** or **record_review_outcome_node**.

PURPOSE

Write an agent's fields back as one text for the record.

REQUIREMENTS

- Each field is written as its label, a colon, a space and its text, in the order given, with one blank line between fields.

RETURNS

str. The fields as one text.

FUNCTION SIGNATURE

```
plan_review(request_profile, request_documents, policy_document, coordinator_request)
```

PARAMETERS

request_profile — **dict**. The request, passed by **plan_review_node** from the workflow state.

request_documents — **list[dict]**. The request's documents, passed by the same node.

policy_document — **str**. The medical policy the request is screened under, taken by the same node from the screening.

coordinator_request — **str**. What the person who opened the review asked, passed by the same node.

PURPOSE

Decide what the review has to establish about this request.

REQUIREMENTS

- The prompt states the request's own facts as **describe_request** words them, the policy, each document in its order or that none was sent, and the coordinator's request.
- The agent is told to name no criterion, threshold or figure it is not given.
- The reply carries **PRIORITY**, **CHECKS** and **RATIONALE**, and the call is made as **config.AGENT_NAME_PLANNER**.

RETURNS

dict. **review_plan**, holding the three fields, and **call_record**.

FUNCTION SIGNATURE

```
analyse_criteria(screening, request_profile, review_plan)
```

PARAMETERS

screening — **dict**. The request's screening, passed by **assess_criteria_node** from the workflow state.

request_profile — **dict**. The request, passed by the same node.

review_plan — **dict**. The planner's three fields, passed by the same node.

PURPOSE

Explain which criteria are met, which are not and which are not documented, and why.

REQUIREMENTS

- The prompt states every criterion in the policy's order, with its sentence, the answers it reads (**config.CRITERION_ANSWER_COLUMNS**, labelled by **config.QUESTIONNAIRE_ANSWER_LABELS**), the body mass index for the criteria of **config.BODY_MASS_INDEX_CRITERIA** or why none was worked out, its result and its blank answers; then the screening, the decisions open and the plan's **CHECKS**.
- The agent is told to quote every result as given, never to fill a blank answer from a document, and never to write that anything has been decided.
- The reply carries **CRITERIA MET**, **CRITERIA NOT MET**, **NOT DOCUMENTED** and **CRITERIA SUMMARY**, and the call is made as **config.AGENT_NAME_CRITERIA**.

RETURNS

dict. **criteria_analysis** and **call_record**.

FUNCTION SIGNATURE

```
review_evidence(evidence_findings, disagreements, reviewer_checks, request_profile, screening,
```

PARAMETERS

evidence_findings — **list[dict]**. Every sentence read, with the criteria it bears on, passed by **review_evidence_node** from the workflow state.
disagreements — **list[dict]**. Every disagreement flagged, passed by the same node.
reviewer_checks — **dict**. Each evidence type found against the reviewer checks its guide sets, read by the same node.
request_profile — **dict**. The request, passed by the same node.
screening — **dict**. The request's screening, passed by the same node.
review_plan — **dict**. The planner's three fields, passed by the same node.

PURPOSE

Say what the documents show, what disagrees with the questionnaire, and what the evidence guides say to check.

REQUIREMENTS

- The prompt states the request's urgency and site of service; each answer the policy's criteria read, together with those of **config.EVIDENCE_EXTRA_ANSWER_COLUMNS**, each once; every sentence, document by document, with its evidence type and the criteria it bears on; every disagreement with its sentence; each guide passage with the guide it came from; and the plan's **CHECKS**. An empty list is stated as none.
- The agent is told that a document never fills or replaces an answer and that a disagreement is flagged for the reviewer, never settled.
- The reply carries **EVIDENCE FOUND**, **DISAGREEMENTS**, **CHECKS TO MAKE** and **EVIDENCE SUMMARY**, and the call is made as **config.AGENT_NAME_EVIDENCE**.

RETURNS

dict. **evidence_review** and **call_record**.

FUNCTION SIGNATURE

```
analyse_predictions(likelihood, stay, request_profile, screening, review_plan)
```

PARAMETERS

likelihood — **dict**. The approval likelihood and its band, passed by **weigh_predictions_node** from the workflow state.
stay — **dict**. The expected stay, empty unless the request is for an inpatient operation, passed by the same node.
request_profile — **dict**. The request, passed by the same node.
screening — **dict**. The request's screening, passed by the same node.
review_plan — **dict**. The planner's three fields, passed by the same node.

PURPOSE

Say what the two model figures show and what they rest on.

REQUIREMENTS

- The prompt states the likelihood to **config.LIKELIHOOD_DECIMALS** places with its band and the band thresholds, and the inputs of **config.APPROVAL_INPUT_LABELS**, the screening's counts among them; the stay as **describe_expected_stay** words it and, for an inpatient operation, **config.LONGER_STAY_MARGIN_DAYS** and the inputs of **config.STAY_INPUT_LABELS**, the screening's body mass index among them; **config.BLANK_INPUT_RULE_TEXT**; and the plan's **CHECKS**.
- The agent is told that neither figure decides anything or is ever told to an office or a member, and never to recalculate or invent a figure.
- The reply carries **LIKELIHOOD**, **EXPECTED STAY** and **PREDICTION SUMMARY**, and the call is made as **config.AGENT_NAME_PREDICTIONS**.

RETURNS

dict. **prediction_analysis** and **call_record**.

FUNCTION SIGNATURE

```
write_review_recommendation(request_profile, coordinator_request, screening, likelihood, stay,
```

PARAMETERS

request_profile — **dict**. The request, passed by **compose_review_recommendation_node** from the workflow state.
coordinator_request — **str**. What the review was opened to look at, passed by the same node.
screening — **dict**. The request's screening, passed by the same node.
likelihood — **dict**. The approval likelihood and its band, passed by the same node.
stay — **dict**. The expected stay, empty unless the request is for an inpatient operation, passed by the same node.

review_plan, **criteria_analysis**, **evidence_review**, **prediction_analysis** — **dict**. The four earlier agents' fields, passed by the same node.
PURPOSE
Write the review a UM Nurse Reviewer decides from, and recommend one decision.
REQUIREMENTS

- The prompt states the request and its policy, the coordinator's request, the plan's **PRIORITY** and **CHECKS**, every field the other three agents returned, each criterion's result, the screening, the likelihood with its band, the stay in words, each of **config.DECISIONS** with **config.DECISION_DESCRIPTIONS** and **config.DECISION_OPEN_CONDITIONS**, and the decisions open.
- The agent is told that it recommends and does not decide, that neither model figure is a reason for a decision, and to quote every figure exactly.
- The reply carries **RECOMMENDATION** and **RECOMMENDED DECISION**, and the call is made as **config.AGENT_NAME_WRITER**.
- **RECOMMENDED DECISION** is returned as one of the decisions open, read in upper case without the decoration at its edges; a decision written over more than one line, and one that is not open, are refused, naming the agent.

RETURNS
dict. **review_recommendation**, holding the two fields, and **call_record**.

FUNCTION SIGNATURE
`draft_office_letter(decision, request_profile, letter_details)`
PARAMETERS
decision — **str**. **config.DECISION_APPROVE** or **config.DECISION_REQUEST_INFORMATION**, passed by **draft_office_letter_node** from the review decision.
request_profile — **dict**. The request, passed by the same node.
letter_details — **dict**. The details the letter carries, built by the same node with **build_authorisation_details** or **build_information_request_details**.
PURPOSE
Write the letter the requesting office receives.
REQUIREMENTS

- The prompt states the letter **config.LETTER_KINDS** names for the decision, the requesting office, the decision with its description, every detail (a list item by item), **config.LETTER_GUIDANCE** for the decision and **config.LETTER_SIGN_OFF**.
- The reply carries **SUBJECT** and **LETTER**, and the call is made as **config.AGENT_NAME_LETTER**.
- Refused, naming the agent: a subject over more than one line, or one that does not name the request reference and the service, by its name or its code, in any case; a subject or letter holding any of **config.LETTER_FORBIDDEN_TERMS**, in any case; a letter holding any of **config.TEMPLATE_SLOT_CHARACTERS**; a letter that does not end with **config.LETTER_SIGN_OFF**, compared without regard to case, each run of the characters **config.SIGN_OFF_SPACING_PATTERN** finds read as one space.

RETURNS
dict. **office_letter**, holding the two fields, and **call_record**.

FUNCTION SIGNATURE
`draft_member_notice(request_profile, screening)`
PARAMETERS
request_profile — **dict**. The approved request, passed by **draft_member_notice_node** from the workflow state.
screening — **dict**. The request's screening, passed by the same node.
PURPOSE
Write the notice the member receives after an approval.
REQUIREMENTS

- A request with no site of service is refused before any call, naming the request.
- The prompt states the service's name, where it is given (the site of service, and the planned facility when the request names one), **config.NOTICE_GUIDANCE** and **config.LETTER_SIGN_OFF**.
- The reply carries **HEADLINE** and **NOTICE**, and the call is made as **config.AGENT_NAME_NOTICE**.
- Refused, naming the agent: a headline over more than one line; a headline and notice that, read together in any case, do not name the service, or the most specific place given, the planned facility where there is one and otherwise the site of service; a headline or notice naming, in any case, any criterion of the screening, a value of **config.NOTICE_WITHHELD_COLUMNS** or any of **config.LETTER_FORBIDDEN_TERMS**; a notice holding any of **config.TEMPLATE_SLOT_CHARACTERS**; a notice that does not end with the sign-off.

RETURNS
dict. **member_notice**, holding the two fields, and **call_record**.

MODULE: agentic/nodes.py
The ten steps of the workflow, the router out of the gate, and the context every node and the service share. Each node takes the state, does one thing and returns only the keys it wrote. A node that calls an agent records the call in **llm_calls** against the run as soon as the agent function returns it, and returns **llm_call_count** as **1**.

MODULE-LEVEL RESOURCES
Everything below is loaded once, when the module is first imported, from the paths in config.

Name	Type	What it holds
approval_model, **stay_model**, **evidence_model**	**sklearn.pipeline.Pipeline**	The saved models, loaded by **load_model**
approval_feature_names, **stay_feature_names**	**list[str]**	Their saved schemas, loaded by **load_feature_schema**
open_requests	**pandas.DataFrame**	The live queue, loaded by **load_open_requests**
request_notes	**pandas.DataFrame**	Its documents, loaded by **load_request_notes**
knowledge_collection	**chromadb.Collection**	The knowledge base, opened by **open_vector_index**

FUNCTION SIGNATURE

```
gather_request_context(request_reference)
```

PARAMETERS

request_reference — **str**. A request in the live queue, passed by **load_request_context_node**, and by **api/routes.py** for every request it ranks.

PURPOSE

Establish everything about one request that needs no language model.

REQUIREMENTS

- The screening is under the request's own medical policy; the findings are its documents' sentences as **read_document_evidence** reads them, linked by **link_evidence_to_criteria**; the disagreements and the likelihood are those the tools give for that request and policy.
- **stay** is worked out only for a request whose site of service is **config.INPATIENT_SITE**, and is empty otherwise.
- No language model is called and nothing is written.

RETURNS

dict. **request_profile**, **request_documents**, **screening**, **evidence_findings**, **disagreements**, **likelihood** and **stay**.

FUNCTION SIGNATURE

```
load_request_context_node(state)
```

PARAMETERS

state — **AuthorisationReviewState**. The workflow's state, handed to the node by LangGraph. This node reads **request_reference**, **coordinator_request**, **coordinator_id** and **thread_id**.

PURPOSE

Gather the request's context and open the run.

REQUIREMENTS

- The run identifier is **config.RUN_ID_PREFIX** and the review identifier **config.REVIEW_ID_PREFIX**, each followed by **config.IDENTIFIER_HEX_LENGTH** random upper-case hexadecimal characters.
- The run is written to **agent_runs** with the four keys read and the status **config.AGENT_RUN_STATUS_RUNNING**; no language model is called.

RETURNS

dict. **run_id**, **review_id** and the seven keys of **gather_request_context**.

FUNCTION SIGNATURE

```
plan_review_node(state)
```

PARAMETERS

state — **AuthorisationReviewState**. This node reads **request_profile**, **request_documents**, **screening**, **coordinator_request** and **run_id**.

PURPOSE

Ask the planner what the review has to establish.

REQUIREMENTS

- **plan_review** is called once and its call recorded.

RETURNS

dict. **review_plan** and **llm_call_count**.

FUNCTION SIGNATURE

```
assess_criteria_node(state)
```

PARAMETERS

state — **AuthorisationReviewState**. This node reads **screening**, **request_profile**, **review_plan** and **run_id**.

PURPOSE

Ask the criteria analyst to explain the screening.

REQUIREMENTS

- **analyse_criteria** is called once and its call recorded.

RETURNS

dict. **criteria_analysis** and **llm_call_count**.

FUNCTION SIGNATURE

`review_evidence_node(state)`

PARAMETERS
state — **AuthorisationReviewState**. This node reads **evidence_findings**, **disagreements**, **request_profile**, **screening**, **review_plan** and **run_id**.
PURPOSE
Ask the evidence analyst to review the documents with the guides' checks in hand.
REQUIREMENTS

- The reviewer checks are read with **read_reviewer_checks** for each evidence type found that **config.EVIDENCE_GUIDE_DOCUMENTS** covers, in that order, and for no other.
- **review_evidence** is called once and its call recorded.

RETURNS
dict. **evidence_review** and **llm_call_count**.

FUNCTION SIGNATURE
`weigh_predictions_node(state)`

PARAMETERS
state — **AuthorisationReviewState**. This node reads **likelihood**, **stay**, **request_profile**, **screening**, **review_plan** and **run_id**.
PURPOSE
Ask the likelihood analyst to explain the two figures.
REQUIREMENTS

- **analyse_predictions** is called once and its call recorded.

RETURNS
dict. **prediction_analysis** and **llm_call_count**.

FUNCTION SIGNATURE
`compose_review_recommendation_node(state)`

PARAMETERS
state — **AuthorisationReviewState**. This node reads every key written so far.
PURPOSE
Store the review a UM Nurse Reviewer will decide.
REQUIREMENTS

- **write_review_recommendation** is called once.
- In one transaction: the call is recorded; the review is written to **authorisation_reviews** with status **config.REVIEW_STATUS_AWAITING**, carrying the tools' own results and figures, never an agent's retelling of them, each earlier agent's fields as one text through **join_labelled_response**, and the writer's two fields; and the run moves to **config.AGENT_RUN_STATUS_AWAITING_DECISION** with every call made so far, this one included.
- A value the request left blank is stored as nothing, and every number is a plain Python number.

RETURNS
dict. **llm_call_count**.

FUNCTION SIGNATURE
`request_review_decision_node(state)`

PARAMETERS
state — **AuthorisationReviewState**. This node reads **review_id**, **screening**, **request_profile** and **coordinator_id**.
PURPOSE
Hold the run until a UM Nurse Reviewer decides, and take in that decision.
REQUIREMENTS

- The run pauses here, offering the review identifier, and resumes with the decision the service passes in.
- On resuming, a decision **find_decision_refusal** refuses stops the run with a **ValueError** giving its reason.
- **office_letter**, **letter_details** and **member_notice** start empty.

RETURNS
dict. **review_decision**, **office_letter**, **letter_details** and **member_notice**.

FUNCTION SIGNATURE
`route_review_decision(state)`

PARAMETERS
state — **AuthorisationReviewState**. This function reads **review_decision**.
PURPOSE
Choose the path out of the gate, and out of the letter.
REQUIREMENTS

- The path is chosen from the decision and from nothing else.

RETURNS
str. The decision, one of **config.DECISIONS**.

FUNCTION SIGNATURE
`draft_office_letter_node(state)`

PARAMETERS

state — **AuthorisationReviewState**. This node reads **review_decision**, **request_profile**, **screening** and **run_id**.
PURPOSE
Have the letter to the requesting office written.
REQUIREMENTS

- For an approval the details are **build_authorisation_details**'s; for a request for information they are **build_information_request_details**'s, with the documentation to send read by **read_documentation_to_send** for the request's service.
- **draft_office_letter** is called once and its call recorded.

RETURNS
dict. **office_letter**, **letter_details** and **llm_call_count**.

FUNCTION SIGNATURE
`draft_member_notice_node(state)`

PARAMETERS
state — **AuthorisationReviewState**. This node reads **request_profile**, **screening** and **run_id**.
PURPOSE
Have the member notice written.
REQUIREMENTS

- **draft_member_notice** is called once and its call recorded.

RETURNS
dict. **member_notice** and **llm_call_count**.

FUNCTION SIGNATURE
`record_review_outcome_node(state)`

PARAMETERS
state — **AuthorisationReviewState**. This node reads **review_decision**, **office_letter**, **letter_details**, **member_notice**, **screening**, **evidence_findings**, **disagreements**, **review_id**, **run_id** and **llm_call_count**.
PURPOSE
Record the decision and everything it led to, and close the run.
REQUIREMENTS

- In one transaction: the decision is written to **review_decisions**; the review takes the status **config.REVIEW_STATUS_BY_DECISION** gives, with **letter_text** and **notice_text**, each written by **join_labelled_response**, and **letter_details** for what was written, and nothing for what was not; and the run moves to **config.AGENT_RUN_STATUS_COMPLETE** with the total calls made.
- **referral_summary** holds **criteria**, **evidence** and **disagreements** for a referral, and is nothing for any other decision.

RETURNS
dict. An empty dictionary: the work is in the database.

MODULE: agentic/graph.py
The order the nodes run in, the gate, the branches and the checkpoint.

FUNCTION SIGNATURE
`build_workflow(checkpoint_path)`

PARAMETERS
checkpoint_path — **str**. The file paused runs are written to, passed by **api/routes.py** from **config.CHECKPOINT_DATABASE_FILE**.
PURPOSE
Assemble the ten nodes into a workflow that can be run, paused and resumed.
REQUIREMENTS

- The ten nodes are registered under exactly the names above, and the first seven run in the order shown, from the start.
- **draft_office_letter** is one node. There are exactly two conditional branches, both chosen by **route_review_decision**: out of the gate, for each of **config.DECISIONS**, and out of **draft_office_letter**, for **config.DECISION_APPROVE** and **config.DECISION_REQUEST_INFORMATION**.
- Every path ends at **record_review_outcome**, the last node, and every node is reachable from the start.
- The workflow is compiled with a checkpointer writing to **checkpoint_path** through a connection usable from more than one thread, so a run paused at the gate can be resumed by another process, after a restart.
- Building the workflow runs nothing and calls no model.

RETURNS
langgraph.graph.state.CompiledStateGraph. The compiled workflow.

============== THE REST SERVICE ==============
Every figure about a request that the dashboard shows comes from this service. The dashboard holds no business rule and reads no file of its own, so the same request gives the same answer whether it is reached through a browser, a script or the marking suite.

MODULE: api/schemas.py

The shape of every request the service accepts and every reply it returns, declared once. A request that does not meet the conditions stated is refused with **status 422** before any work is done, and a reply carries exactly the fields listed. Every dictionary a reply carries holds only plain Python values, never a NumPy value. Field names are read by the dashboard and by the marking suite and must be reproduced exactly.

MODULE-LEVEL TYPES

- **ServiceFilter** — **typing.Literal**. The values the queue may be filtered by, from **config.QUEUE_SERVICE_OPTIONS**: **config.QUEUE_SERVICE_ALL**, or one of the five service codes.
- **ReviewStatusValue** — **typing.Literal**. The statuses a review may take, from **config.REVIEW_STATUS_OPTIONS**.
- **DecisionValue** — **typing.Literal**. The decisions of **config.DECISIONS**.
- **CoordinatorRequestText** — **typing.Annotated[str, ...]**. Text with surrounding whitespace removed, from **config.COORDINATOR_REQUEST_MINIMUM_CHARACTERS** to **config.COORDINATOR_REQUEST_MAXIMUM_CHARACTERS** characters.
- **DecisionReasonText** — **typing.Annotated[str, ...]**. The same, from **config.DECISION_REASON_MINIMUM_CHARACTERS** to **config.DECISION_REASON_MAXIMUM_CHARACTERS** characters.
- **QuestionText** — **typing.Annotated[str, ...]**. The same, from **config.CHAT_QUESTION_MINIMUM_CHARACTERS** to **config.CHAT_QUESTION_MAXIMUM_CHARACTERS** characters.

REQUEST SHAPE

`SignInRequest`

FIELDS

Field	Type	What it carries
username	**str**	The account signing in
password	**str**	The password as typed

VALIDATION

- Both fields are required.

RESPONSE SHAPE

`SignedInUserResponse`

FIELDS

Field	Type	What it carries
username	**str**	The account that signed in
display_name	**str**	The person's name as the dashboard shows it
role	**str**	The account's role
session_token	**str**	The token every later request carries in **config.SESSION_TOKEN_HEADER**
pages	**list[str]**	The pages the role is offered, from **config.ROLE_PAGE_ACCESS**
areas	**list[str]**	The areas the role reaches, from **config.ROLE_AREA_ACCESS**

VALIDATION

None beyond the field types above.

RESPONSE SHAPE

`SignedOutResponse`

FIELDS

Field	Type	What it carries
username	**str**	The account whose session ended
signed_out	**bool**	That the session is over

VALIDATION

None beyond the field types above.

RESPONSE SHAPE

`QueueRow`

FIELDS

Field	Type	What it carries
queue_position	**int**	The request's place in the queue, counting from **1**
request_reference	**str**	The request
service_code	**str**	Its service
service_name	**str**	The service's name, from **config.SERVICE_NAMES**
urgency	**str**	Its urgency, in the words **describe_request** gives
site_of_service	**str**	Its site of service, in the same words
document_count	**int**	How many documents arrived with it
criteria_met_count	**int**	Criteria met
criteria_not_met_count	**int**	Criteria not met
criteria_not_documented_count	**int**	Criteria not documented
screening	**str**	The screening
decisions_open	**list[str]**	The decisions open
approval_likelihood	**float**	The approval likelihood
approval_band	**str**	Its band
disagreement_count	**int**	How many disagreements were flagged

VALIDATION

None beyond the field types above.

RESPONSE SHAPE

`RequestDetailResponse`

FIELDS

Field	Type	What it carries
queue_position	**int**	The request's place in the queue
request	**dict**	Every column of the request, as **describe_request** words it
screening	**dict**	The screening, its **body_mass_index** a plain number or **null**
evidence_findings	**list[dict]**	Every sentence read, with the criteria it bears on
disagreements	**list[dict]**	Every disagreement flagged
likelihood	**dict**	The approval likelihood and its band
stay	**dict**	The expected stay, empty unless the request is for an inpatient operation
stay_text	**str**	The stay as **describe_expected_stay** words it

VALIDATION

None beyond the field types above.

REQUEST SHAPE

`ReviewRequest`

FIELDS

Field	Type	What it carries
request_reference	**str**	The request to review
coordinator_request	**CoordinatorRequestText**	What the person opening the review wants looked at

VALIDATION

- **coordinator_request** meets **CoordinatorRequestText**.

REQUEST SHAPE

`DecisionRequest`

FIELDS

Field	Type	What it carries
decision	**DecisionValue**	One of **config.DECISIONS**
decision_reason	**DecisionReasonText**	The UM Nurse Reviewer's reason, in their own words

VALIDATION

- **decision** is one of **config.DECISIONS**, and **decision_reason** meets **DecisionReasonText**.

RESPONSE SHAPE

`ReviewRecordResponse`

FIELDS

Field	Type	What it carries
review_id	**str**	The review
run_id	**str**	The run that produced it
thread_id	**str**	The thread its run is checkpointed under
coordinator_id	**str**	Who opened the review
request_reference	**str**	The request
service_code	**str**	Its service
service_name	**str**	The service's name
policy_document	**str**	The policy it was screened under
urgency	**str | None**	Its urgency, **null** when blank
site_of_service	**str | None**	Its site of service, **null** when blank
coordinator_request	**str**	What the review was opened to look at
criteria	**list[dict]**	Every criterion with its result
screening	**str**	The screening
decisions_open	**list[str]**	The decisions open
body_mass_index	**float | None**	The index the screening worked out
evidence	**list[dict]**	Every finding with the criteria it bears on
disagreements	**list[dict]**	Every disagreement
approval_likelihood	**float**	The likelihood
approval_band	**str**	Its band
expected_stay_days	**float | None**	The expected stay, for an inpatient operation
requested_inpatient_days	**int | None**	The days requested
longer_stay_expected	**bool | None**	Whether a longer stay is expected
stay_text	**str**	The stay in words
review_plan_text	**str**	The planner's write-up
criteria_analysis_text	**str**	The criteria analyst's write-up
evidence_review_text	**str**	The evidence analyst's write-up
prediction_analysis_text	**str**	The likelihood analyst's write-up
recommendation_text	**str**	The writer's recommendation
recommended_decision	**str**	One of the decisions open
status	**str**	One of **config.REVIEW_STATUS_OPTIONS**
letter_text	**str | None**	The letter, once written
letter_details	**dict | None**	The letter's details, once written
notice_text	**str | None**	The member notice, once written
referral_summary	**dict | None**	The referral summary, for a referral
llm_call_count	**int**	The run's call count, as **agent_runs** holds it
decision	**str | None**	The decision, once taken
decided_by	**str | None**	Who took it
decided_by_role	**str | None**	Their role
decision_reason	**str | None**	Their reason

VALIDATION

None beyond the field types above.

RESPONSE SHAPE

`CallBudgetResponse`

FIELDS

Field	Type	What it carries
review_id	**str**	The review
run_id	**str**	Its run
call_count	**int**	The calls the run has made
expected_call_count	**int**	The calls the workflow allows at this point
total_latency_milliseconds	**int**	The calls' latencies added together
calls	**list[dict]**	Every call, as **fetch_llm_calls** returns them

VALIDATION
None beyond the field types above.

REQUEST SHAPE
`ChatRequest`

FIELDS

Field	Type	What it carries
question	**QuestionText**	The question

VALIDATION

- **question** meets **QuestionText**.

RESPONSE SHAPE
`ChatResponse`

FIELDS

Field	Type	What it carries
question_type	**str**	One of **config.CHAT_TYPE_OPTIONS**
request_reference	**str | None**	The request the question is about, if any
answer	**str**	The answer, or the refusal
source_documents	**list[str]**	The documents the answer rested on
was_refused	**bool**	Whether it was a refusal

VALIDATION
None beyond the field types above.

RESPONSE SHAPE
`ChatTurnResponse`

FIELDS

Field	Type	What it carries
turn_id	**int**	The turn
username	**str**	Who asked
user_role	**str**	Their role at the time
question_text	**str**	What they asked, member numbers withheld
question_type	**str**	How it was classified
request_reference	**str | None**	The request it was about, if any
answer_text	**str**	What they were told
source_documents	**str**	The documents, written as text
closest_distance	**float**	How far the nearest passage was
was_refused	**bool**	Whether it was a refusal

VALIDATION
None beyond the field types above.
REQUIREMENTS

- Only a field that may legitimately be empty is declared as able to be empty.

MODULE: api/routes.py

PURPOSE
Expose the queue, the request detail, the review workflow, the decision, the call budget, the chat and the audit trail over HTTP, behind one sign-in and access guard and one failure handler.

MODULE-LEVEL OBJECTS

- **review_workflow** — **langgraph.graph.state.CompiledStateGraph**. The workflow, built once by **build_workflow** from **config.CHECKPOINT_DATABASE_FILE**.
- **request_details** — **dict[str, dict]**. Every queued request's detail, keyed by its reference, built once with **gather_request_context**.
- **review_queue** — **list[dict]**. One queue row per request, in the order **rank_review_queue** gives, built at the same time.
- **request_locks** — **dict[str, threading.Lock]**. One lock per queued request, so that no request is reviewed twice at once and no review is decided twice at once.
- **public_router** — **fastapi.APIRouter**. Signing in and signing out, reachable without a token.
- **guarded_router** — **fastapi.APIRouter**. Every other endpoint, included behind **authorize_request** so that the guard cannot be forgotten on a new endpoint.
- **SessionTokenHeader** — a **typing.Annotated** alias for **str | None**. The optional header **config.SESSION_TOKEN_HEADER**, optional so that a request without it is answered **401** rather than **422**.
- **SignedInUser** — a **typing.Annotated** alias for **dict**. The parameter type that receives the account **authorize_request** returned.
- **service_application** — **fastapi.FastAPI**. The application, titled **config.API_TITLE**, holding both routers and **report_failed_request** as its failure handler.

REQUIREMENTS

- The models, the live files and the knowledge base are not loaded again here: the service uses those **agentic/nodes.py** holds.
- The queue and every detail are worked out once, when the module is imported, from the same tools the workflow uses, so the queue, the detail and the stored review agree; no language model is called.
- Every endpoint answers **200** when it succeeds, except **POST /reviews**, which answers **201**.
- No endpoint carries a default parameter value except **session_token** of **sign_out_user**.
- Only **POST /reviews**, **POST /reviews/{review_id}/decision** and **POST /chat/ask** may cost language model calls.

sign_in_user
ENDPOINT
POST /auth/signin
FUNCTION SIGNATURE
`sign_in_user(sign_in_request)`

PARAMETERS
sign_in_request — **SignInRequest**. The username and password, read from the request body.
PURPOSE
Exchange a username and password for a session token.
REQUIREMENTS

- An unknown username and a wrong password are answered identically, with **401** and **config.SIGN_IN_FAILED_MESSAGE**.
- The password is checked with **hash_password** and the account's own salt, compared in constant time.
- A correct password issues a fresh token of **config.SESSION_TOKEN_BYTES** and stores it against the account, replacing any token it held, so an account holds one session.

RETURNS
dict. A **SignedInUserResponse**.

sign_out_user
ENDPOINT
POST /auth/signout
FUNCTION SIGNATURE
`sign_out_user(session_token=None)`

PARAMETERS
session_token — **str | None**. The token to end, declared as **SessionTokenHeader** with a default of **None**, which it takes when the header is absent.
PURPOSE
End a session.
REQUIREMENTS

- A request with no token, or a token no account holds, is answered **401** with **config.SIGN_IN_REQUIRED_MESSAGE**.
- The account's token is cleared, so the same token opens nothing afterwards.

RETURNS
dict. A **SignedOutResponse**, with **signed_out true**.

list_review_queue
ENDPOINT
GET /queue
FUNCTION SIGNATURE
`list_review_queue(service_code)`

PARAMETERS
service_code — **str**. Required, declared as **ServiceFilter** and taken from the query string.
PURPOSE
Give the desk its ranked list of open requests.
REQUIREMENTS

- **config.QUEUE_SERVICE_ALL** gives every row; a service code gives exactly that service's rows, in the same order and with the same positions.

RETURNS
list[dict]. One **QueueRow** per request.

get_request_detail
ENDPOINT
GET /queue/{request_reference}
FUNCTION SIGNATURE
`get_request_detail(request_reference)`

PARAMETERS
request_reference — **str**. The request, taken from the path.
PURPOSE
Show one request with everything the review would work from, before any language model call is spent on it.
REQUIREMENTS

- A request not in the queue is answered **404** with **config.UNKNOWN_REQUEST_MESSAGE** filled with the reference.

RETURNS
dict. A **RequestDetailResponse**.

start_authorisation_review
ENDPOINT
POST /reviews
FUNCTION SIGNATURE
`start_authorisation_review(review_request, signed_in_user, request)`

PARAMETERS
review_request — **ReviewRequest**. The request and what is to be looked at, read from the request body.
signed_in_user — **dict**. The account of whoever is opening the review, declared as **SignedInUser**.
request — **fastapi.Request**. The HTTP request itself, through which **report_failed_request** learns the thread of a run that fails.
PURPOSE
Run the five reviewing agents on one request and leave it waiting for a UM Nurse Reviewer.
REQUIREMENTS

- A request not in the queue is answered **404** with **config.UNKNOWN_REQUEST_MESSAGE** filled with the reference, before any call.
- Under the request's lock, a request that already has a stored review is answered **409** with **config.REQUEST_ALREADY_REVIEWED_MESSAGE** filled with the reference.
- The run's thread is the reference, a hyphen and **config.IDENTIFIER_HEX_LENGTH** random upper-case hexadecimal characters, kept on the request's state as **opened_thread_id**.
- The workflow runs to the gate with the signed-in username as **coordinator_id**.

RETURNS
dict. A **ReviewRecordResponse**, the stored review, with status **201**.

list_authorisation_reviews
ENDPOINT
GET /reviews
FUNCTION SIGNATURE
`list_authorisation_reviews(status_value)`

PARAMETERS
status_value — **list[str]**. Required, a list of **ReviewStatusValue** taken from the query string, each status given as its own **status_value**.
PURPOSE
List reviews by status, for the decision queue and the audit trail.
REQUIREMENTS

- Every review whose status is one of those asked for is returned once, newest first.

RETURNS
list[dict]. One **ReviewRecordResponse** per review.

submit_review_decision
ENDPOINT
POST /reviews/{review_id}/decision
FUNCTION SIGNATURE
`submit_review_decision(review_id, decision_request, signed_in_user)`

PARAMETERS
review_id — **str**. The review, taken from the path.

decision_request — **DecisionRequest**. The decision and the reason, read from the request body.
signed_in_user — **dict**. The account of the UM Nurse Reviewer deciding, declared as **SignedInUser**.
PURPOSE
Check the decision, then resume the paused run with it.
REQUIREMENTS

- An unknown review is answered **404** with **config.REVIEW_NOT_FOUND_MESSAGE**; a review whose request is not in the queue, or whose checkpoint holds nothing, is refused, naming the review.
- Under the request's lock: a review no longer at **config.REVIEW_STATUS_AWAITING** is answered **409** with **config.ALREADY_DECIDED_MESSAGE**; where the checkpoint holds a different decision still being recorded, **409** with **config.DECISION_IN_PROGRESS_MESSAGE** filled from it, and the same decision sent again finishes recording it.
- The decision, with the signed-in username as **decided_by** and their role as **decided_by_role**, is checked by **find_decision_refusal** against the checkpoint's screening, request and opener before the run resumes; a refusal is answered **409** with its reason, and no call is spent.
- The run resumes from its checkpoint with the decision.

RETURNS
dict. A **ReviewRecordResponse**, the decided review.

get_call_budget
ENDPOINT
GET /reviews/{review_id}/calls
FUNCTION SIGNATURE
`get_call_budget(review_id)`
PARAMETERS
review_id — **str**. The review, taken from the path.
PURPOSE
Show every language model call the review's run made, against the number the workflow allows.
REQUIREMENTS

- An unknown review is answered **404** with **config.REVIEW_NOT_FOUND_MESSAGE**.
- **expected_call_count** is the figure **config.LLM_CALLS_BY_DECISION** gives for the decision recorded, or for the decision the checkpoint holds while it is being recorded, and **config.LLM_CALLS_BEFORE_DECISION** otherwise; a review whose checkpoint holds nothing and has no decision is refused, naming the review.

RETURNS
dict. A **CallBudgetResponse**.

ask_authorisation_assistant
ENDPOINT
POST /chat/ask
FUNCTION SIGNATURE
`ask_authorisation_assistant(chat_request, signed_in_user)`
PARAMETERS
chat_request — **ChatRequest**. The question, read from the request body.
signed_in_user — **dict**. The account of whoever is asking, declared as **SignedInUser**.
PURPOSE
Answer one question from the documents or the records, or refuse, and record the exchange.
REQUIREMENTS

- Every member number in the question, found by **config.MEMBER_NUMBER_IN_TEXT_PATTERN**, is replaced by **config.CHAT_MEMBER_NUMBER_WITHHELD_TEXT** before the question goes anywhere, so no member number reaches the language model or the record.
- The asker's own conversation is recalled with **recall_conversation**, and the question is answered by **answer_from_documents** in exactly one call.
- A question of **config.CHAT_RECORD_TYPES** is answered by **answer_request_status**, with the member number, made canonical by **config.CANONICAL_MEMBER_NUMBER_FORMAT**, when the question gave exactly one; a number written in two groups of digits is withheld like any other but is the member number of no request.
- Every turn is recorded, refusals included, with the documents joined by a comma and a space.

RETURNS
dict. A **ChatResponse**.

list_own_chat_turns
ENDPOINT
GET /chat/turns
FUNCTION SIGNATURE
`list_own_chat_turns(signed_in_user, turn_limit)`
PARAMETERS
signed_in_user — **dict**. The account of whoever is asking, declared as **SignedInUser**.
turn_limit — **int**. Required, taken from the query string, from **1** to **config.CHAT_AUDIT_MAX_ROWS**.
PURPOSE
Give a person their own conversation back.
REQUIREMENTS

- Only the signed-in person's own turns are returned, newest first, at most **turn_limit** of them.

RETURNS
list[dict]. One **ChatTurnResponse** per turn.

list_recent_chat_turns
ENDPOINT
GET /audit/chat-turns
FUNCTION SIGNATURE
`list_recent_chat_turns(turn_limit)`

PARAMETERS
turn_limit — **int**. Required, taken from the query string, from **1** to **config.CHAT_AUDIT_MAX_ROWS**.
PURPOSE
Give the audit trail every recent question, whoever asked it.
REQUIREMENTS

- Turns from every person are returned, newest first, at most **turn_limit** of them.

RETURNS
list[dict]. One **ChatTurnResponse** per turn.

report_failed_request
FUNCTION SIGNATURE
`report_failed_request(request, failure)`

PARAMETERS
request — **fastapi.Request**. The HTTP request that failed.
failure — **Exception**. A **ValueError** from any part of the system, a **google.genai.errors.APIError**, or an **httpx.HTTPError**.
PURPOSE
Turn every failure into one reply the dashboard can show, and never leave a failed run looking as though it is still running.
REQUIREMENTS

- The reply has status **502** and a **detail** naming the method, the path and the reason.
- Where the failing request opened a run, that run is recorded as **config.AGENT_RUN_STATUS_FAILED** with the number of calls recorded against it.

RETURNS
fastapi.responses.JSONResponse. The failure reply.

=============== ACCESS CONTROL ===============
Everyone signs in, and the service, not the dashboard, decides what each role may reach.

MODULE: access/password_security.py

FUNCTION SIGNATURE
`hash_password(plain_password, password_salt)`

PARAMETERS
plain_password — **str**. The password as typed, passed by **sign_in_user**, or by **run_database_setup.py** from **config.SEED_USER_ACCOUNTS**.
password_salt — **str**. The account's own salt, read by **sign_in_user** from **user_accounts**, or made by **run_database_setup.py** from **config.PASSWORD_SALT_BYTES**.
PURPOSE
Turn a password into the form the database stores, so that the database never holds a password.
REQUIREMENTS

- The stored form is a key derivation with **config.PASSWORD_HASH_ALGORITHM** over **config.PASSWORD_HASH_ROUNDS** rounds, salted with **password_salt**, written as hexadecimal text.
- The same password and salt always give the same result, and a different salt gives a different result.

RETURNS
str. The stored form.

MODULE: access/permissions.py

FUNCTION SIGNATURE
`authorize_request(request, session_token=None)`

PARAMETERS
request — **fastapi.Request**. The incoming request, read for its method and its route's path template.
session_token — **str | None**. The token sent under **config.SESSION_TOKEN_HEADER**, with a default of **None**, which it takes when the header is absent.
PURPOSE
Decide, for every guarded request, who is asking and whether their role reaches that area.
REQUIREMENTS

- A request with no token, or a token no account holds, is refused **401** with **config.SIGN_IN_REQUIRED_MESSAGE**.
- The area is the one **config.AREA_BY_ROUTE** gives for the method and the route's path template, for example **/reviews/{review_id}/decision**.
- A role whose **config.ROLE_AREA_ACCESS** does not hold that area is refused **403** with **config.AREA_REFUSED_MESSAGE**, naming the role and the area, before the endpoint runs and whether or not what it asks for exists.

RETURNS
dict. The signed-in person's account record.

========== THE AUTHORISATION CHAT ==========
One question, one language model call, answered from the documents and from nothing else. The assistant classifies what it was asked, answers the kinds it may answer, and refuses the rest. It knows nothing about any request: a question about one is answered by the service from the review records, as the asker's role allows.

Kind	What it covers	Answered from
config.CHAT_TYPE_MEDICAL_POLICY	What a service's medical policy says	The policies and the passages found
config.CHAT_TYPE_REVIEW_RULES	How prior authorisation works: who opens and who decides, the decisions, the letters, the timeframes and the Rules' readings	The passages found
config.CHAT_TYPE_EVIDENCE_GUIDANCE	What an evidence guide says	The guides and the passages found
config.CHAT_TYPE_REQUEST_STATUS	Where one request has got to	The review records, by **answer_request_status**
config.CHAT_TYPE_OUTCOME_PREDICTION	Whether a request will be approved, or how likely it is	The review records, by **answer_request_status**
config.CHAT_TYPE_OUT_OF_SCOPE	Anything the documents do not answer	Nothing: it is refused

MODULE: chat/conversation_memory.py

FUNCTION SIGNATURE
```
recall_conversation(session, username, question_text)
```
PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **ask_authorisation_assistant**.
username — **str**. Whose conversation to recall, the signed-in person's.
question_text — **str**. The question just asked, its member numbers already withheld.
PURPOSE
Give the assistant the last few exchanges of this person's own conversation, as history to read and as words to search with.
REQUIREMENTS
- At most **config.CHAT_HISTORY_TURN_LIMIT** of that person's own most recent turns are recalled, and nobody else's.
- **history_text** shows them oldest first, each with what was asked and what was answered, an answer to a question of **config.CHAT_RECORD_TYPES** replaced by **config.CHAT_RECORD_ANSWER_IN_HISTORY** so that nothing from the review records reaches a prompt; with no earlier turn it says this is the first question.
- **search_text** is the question asked just before, where there is one, and the new question together.

RETURNS
dict. **history_text** and **search_text**.

MODULE: chat/authorisation_assistant.py

FUNCTION SIGNATURE
```
answer_from_documents(collection, question_text, conversation)
```
PARAMETERS
collection — **chromadb.Collection**. The knowledge base, passed by **ask_authorisation_assistant** from **agentic/nodes.py**.
question_text — **str**. The question, its member numbers already withheld.
conversation — **dict**. **history_text** and **search_text**, as **recall_conversation** returned them.
PURPOSE
Settle what kind of question was asked and answer it from the documents in one language model call, refusing when they do not ground an answer.
REQUIREMENTS
- Two searches are made together with **search_documents**, one for the question and one for **search_text**, and their passages are offered once each, nearest first.

- The prompt also carries the question, the conversation's **history_text**, every service's criteria, red flags listed and documentation to send, each read from that service's own policy, and what each evidence guide says to tell the requesting office; it names the six kinds of **config.CHAT_TYPE_OPTIONS**, and tells the model to answer only from what it is given, never to write a member number, and that it knows nothing about any request.
- The reply carries **QUESTION TYPE**, **REQUEST** and **ANSWER**, with **config.CHAT_NO_VALUE_REPLY** in a field that does not apply, and the call is made as **config.AGENT_NAME_AUTHORISATION_ASSISTANT**.
- Refused, naming the agent: a **QUESTION TYPE** or **REQUEST** over more than one line; a type, read in upper case without the decoration at its edges, that is not among **config.CHAT_TYPE_OPTIONS**; an empty **REQUEST**; a type of **config.CHAT_ANSWERED_TYPES** whose answer is **config.CHAT_NO_VALUE_REPLY**.
- A question of **config.CHAT_ANSWERED_TYPES** becomes **config.CHAT_TYPE_REQUEST_STATUS** when the question names a request reference, as **config.REQUEST_REFERENCE_IN_TEXT_PATTERN** finds one, or when the reply names one the documents do not hold.
- For a question of **config.CHAT_RECORD_TYPES**, **request_reference** is the first reference the question names, made canonical; failing that, nothing when the model wrote **config.CHAT_NO_VALUE_REPLY**; failing that, the model's reference when the question or the conversation names it, made canonical where it is written as a reference. A reference neither names is refused. For any other type it is nothing.
- A question is answered when its type is one of **config.CHAT_ANSWERED_TYPES** and the nearest passage is within **config.CHAT_MAX_RELEVANCE_DISTANCE**. Any other question, except one of **config.CHAT_RECORD_TYPES**, is answered as a refusal, **was_refused true**: its type becomes **config.CHAT_TYPE_OUT_OF_SCOPE** and its answer **config.CHAT_REFUSAL_MESSAGE**.
- **source_documents** names, once each, every policy for a **config.CHAT_TYPE_MEDICAL_POLICY** answer, every guide for a **config.CHAT_TYPE_EVIDENCE_GUIDANCE** answer, and the documents of the passages found; it is empty unless the question was answered. **closest_distance** is the nearest passage's distance.

RETURNS
dict. **question_type**, **request_reference**, **answer**, **source_documents**, **closest_distance** and **was_refused**.

MODULE: chat/request_status_answer.py

FUNCTION SIGNATURE
```
answer_request_status(session, signed_in_user, open_requests, question_type, request_reference,
```
PARAMETERS
session — **sqlalchemy.orm.Session**. An open session, opened by **ask_authorisation_assistant**.
signed_in_user — **dict**. The asker's account.
open_requests — **pandas.DataFrame**. The live queue, passed from **agentic/nodes.py**.
question_type — **str**. One of **config.CHAT_RECORD_TYPES**, as **answer_from_documents** classified the question.
request_reference — **str | None**. The request the question is about, as **answer_from_documents** returned it.
member_number — **str | None**. The one member number the question gave, in canonical form, or nothing.
PURPOSE
Answer a question about one request from the review records, in the way the asker's role allows.
REQUIREMENTS
- Any other question type is refused, naming it, and no language model is called.
- A **config.ROLE_REPRESENTATIVE** hears what **THE CONFIRMATION LINE** sets out. The confirmation is **config.CHAT_REQUEST_CONFIRMED_TEMPLATE**, with the service as **config.SERVICE_WITH_CODE_FORMAT** and the stored review's status label from **config.REVIEW_STATUS_LABELS** in lower case, **config.REVIEW_STATUS_AWAITING**'s when none is stored. It is given only when **member_number** is the member number the open request holds, and is followed, where the status is **config.REVIEW_STATUS_INFORMATION_REQUESTED**, by **config.CHAT_ANSWERS_ASKED_FOR_TEXT** and the entries of the letter's **config.CRITERIA_NOT_DOCUMENTED_DETAIL**, one to a line. Every other named request is answered **config.CHAT_NO_CONFIRMATION_TEMPLATE**, filled with the reference asked about.
- A role of **config.ROLES_SEEING_EVERY_REVIEW** hears **config.CHAT_REVIEW_NOT_NAMED_MESSAGE** when no request is named and **config.CHAT_NO_REVIEW_TEMPLATE** when no review is stored. Otherwise it hears five lines for each review: **config.CHAT_REVIEW_TEMPLATE**, with the service as **config.SERVICE_WITH_CODE_FORMAT** and the status label as written; **config.CHAT_SCREENING_TEMPLATE**, each criterion as its name and result, joined with a semicolon and a space, and the decisions open joined with a comma and a space; **config.CHAT_FIGURES_TEMPLATE**, with the likelihood to **config.LIKELIHOOD_DECIMALS** places and the stay in words; **config.CHAT_EVIDENCE_TEMPLATE**, with the sentences counted by evidence type, each as the type and its count, in name order and joined with a comma and a space, or **config.CHAT_NO_DOCUMENTS_TEXT**; and **config.CHAT_AWAITING_DECISION_TEMPLATE** or **config.CHAT_DECIDED_TEMPLATE**, the reason with every member number withheld.
- Any other role is refused, naming it.

RETURNS
str. The answer, its lines joined by line breaks.

============== THE DASHBOARD ==============
A sign-in page and five pages over the service, named by config.PAGE_QUEUE, config.PAGE_DETAIL, config.PAGE_DECISION, config.PAGE_AUDIT and **config.PAGE_CHAT**. The dashboard holds no business rule: every figure on every page was produced by the service. The only checks a page makes itself are the service's own input limits and the refusal of a decision on one's own

review, repeated so that a form the service would refuse is never sent. Each page is a function that draws itself and returns nothing.
A decision, a status, a band, a question type, an urgent request and an evidence type are badged with **config.DECISION_BADGE_COLOURS**, **config.STATUS_BADGE_COLOURS**, **config.APPROVAL_BAND_BADGE_COLOURS**, **config.CHAT_TYPE_BADGE_COLOURS**, **config.URGENT_BADGE_COLOUR** and **config.EVIDENCE_BADGE_COLOUR**, and everything else with **config.NEUTRAL_BADGE_COLOUR**. The theme and the port are pre-loaded in **.streamlit/config.toml**.

MODULE: dashboard/api_client.py
Every request the dashboard sends to the service passes through this module, so no page holds an address, a timeout or a session token.

FUNCTION SIGNATURE
`call_service(method, path, query=None, payload=None, waiting_message=None)`

PARAMETERS
method — **str**. The HTTP method, for example **GET** or **POST**.
path — **str**. The path beneath **config.API_BASE_URL**, for example **/queue**.
query — **dict | None**. Query parameters, defaulting to **None** for an endpoint that takes none.
payload — **dict | None**. The request body, defaulting to **None** for an endpoint that takes none.
waiting_message — **str | None**. What the reader is told while a call that costs language model calls runs, defaulting to **None** for a call that returns at once.
PURPOSE
Make one call to the service and give the page back the reply.
REQUIREMENTS

- The call goes to **config.API_BASE_URL** with **config.API_REQUEST_TIMEOUT_SECONDS**, and the signed-in person's token travels under **config.SESSION_TOKEN_HEADER** whenever there is a session.
- While the call runs, and only then, a spinner shows **waiting_message** with the time elapsed; with no message nothing is shown.
- A **401** while signed in ends the session in the browser and returns to signing in, where **config.SESSION_ENDED_MESSAGE** is shown.
- Any other reply with status **400** or more is shown as the service's own **detail**, or as the status and the text when the reply is not JSON, and the page stops there.

RETURNS
dict | list. The service's reply read from JSON.

MODULE: dashboard/sign_in.py

FUNCTION SIGNATURE
`render_sign_in_page()`

PARAMETERS
None.
PURPOSE
Let a person sign in.
REQUIREMENTS

- The page shows **config.DASHBOARD_TITLE**, **config.DASHBOARD_SUBTITLE** and a form for the username and the password, and, once after a session has ended, **config.SESSION_ENDED_MESSAGE**.
- Signing in calls the service once, with the username stripped of surrounding space, and keeps the reply in the browser session; a refused sign-in shows the service's own message.

RETURNS
None. The page draws itself.

MODULE: dashboard/review_queue.py

FUNCTION SIGNATURE
`render_review_queue_page()`

PARAMETERS
None.
PURPOSE
Show the open queue in the order the desk works it, and let a person pick one request.
REQUIREMENTS

- Counts along the top, from the whole queue, say how many requests there are, how many are urgent, how many have every criterion met, how many have answers missing and how many have a criterion not met.
- The reader filters by one of **config.QUEUE_SERVICE_OPTIONS**, labelled by **config.QUEUE_SERVICE_LABELS**; the service does the filtering and the ordering.
- The table shows each request's position, reference, service, urgency, documents, the three criteria counts, disagreements, screening, likelihood to **config.LIKELIHOOD_DECIMALS** places and band.

- Selecting a row remembers that request, so that the detail page opens on it.

RETURNS
None. The page draws itself.

MODULE: dashboard/request_detail.py

FUNCTION SIGNATURE
`render_request_detail_page()`

PARAMETERS
None.
PURPOSE
Show everything established about one request, and let a person open its review.
REQUIREMENTS

- The request is chosen in queue order, opening on the one the queue page remembered; everything shown comes from its detail and its stored reviews.
- The headline shows the service with its code, the urgency where urgent, the screening, the likelihood band, the queue position and any review's status, with the requesting practice (**config.PROVIDER_NAME_COLUMN**, **config.PROVIDER_SPECIALTY_COLUMN**, **config.NETWORK_STATUS_COLUMN**), the channel (**config.RECEIVED_CHANNEL_COLUMN**), the documents, the site of service and the policy; counts show the criteria met of all, not met and not documented, with the likelihood and the expected stay, or **config.STAY_METRIC_NOT_WORKED_OUT**, and the stay in words.
- A screening tab draws each criterion's result leading to the screening and the decisions open, coloured by **config.SCREENING_FILL_COLOURS** and **config.SCREENING_LINE_COLOURS**, keyed by result, **config.SCREENING_NODE** and **config.DECISION_NODE**, and drawn with **config.SCREENING_TEXT_COLOUR**, **config.SCREENING_EDGE_COLOUR** and **config.SCREENING_GRAPH_FONT**, with the body mass index where worked out and each criterion's result, sentence and answers missing.
- An evidence tab shows each disagreement, how many sentences bear on each criterion and on none, charted in **config.EVIDENCE_CHART_COLOUR** and **config.EVIDENCE_NO_CRITERION_COLOUR**, and every sentence, document by document, with its evidence type and the criteria it bears on, or that no document was sent; a questionnaire tab shows each criterion's answers as the Rules read them.
- A review tab shows the stored review's recommendation, or a form that takes what the reader wants looked at, of at least **config.COORDINATOR_REQUEST_MINIMUM_CHARACTERS**; starting it calls the service once, saying **config.LLM_CALLS_BEFORE_DECISION** model calls are being made, and the page reports the review waiting with its recommendation.

RETURNS
None. The page draws itself.

MODULE: dashboard/decision_queue.py

FUNCTION SIGNATURE
`render_decision_queue_page()`

PARAMETERS
None.
PURPOSE
Show the reviews waiting at the gate and let a UM Nurse Reviewer decide one.
REQUIREMENTS

- Only reviews at **config.REVIEW_STATUS_AWAITING** are listed, each by its request, service and urgency; with none, the page says nothing is waiting.
- The chosen review shows its recommendation, policy and band, the stay in words, the count of each result, the likelihood, the recommendation text, each criterion with its answers missing, each disagreement, each decision not open with the reason **config.SCREENING_CLOSED_REASONS** gives, and, on request, what was asked and each agent's write-up.
- Where the signed-in person opened the review, the page says another UM Nurse Reviewer must decide it, in place of the form.
- The form offers exactly the review's decisions open and requires a reason of at least **config.DECISION_REASON_MINIMUM_CHARACTERS**; a decision sent but not recorded is filled in again.
- Recording calls the service once, with a waiting message, and the page reports the status reached with **config.DECISION_OUTCOME_TEXTS** for the decision.

RETURNS
None. The page draws itself.

MODULE: dashboard/audit_trail.py

FUNCTION SIGNATURE
`render_audit_trail_page()`

PARAMETERS

None.
PURPOSE
Show what has been reviewed and decided, the record behind each review, and every question the chat has been asked.
REQUIREMENTS

- Reviews are listed by the statuses the reader chooses from **config.REVIEW_STATUS_OPTIONS**, **config.DECIDED_REVIEW_STATUSES** chosen at first, with the request, service, status, who opened it, the decision and who took it, and the model calls.
- Selecting one shows its screening, recommendation and decision with who took it, their role and their reason; the recommendation and each agent's write-up; and the letter, titled by **config.LETTER_KINDS**, with its details, the member notice and the referral summary, where each exists.
- Its call budget shows the calls made against the calls expected, the total latency in seconds to **config.LATENCY_SECONDS_DECIMALS** place, and every call with what it cost.
- A second tab shows the most recent **config.CHAT_AUDIT_MAX_ROWS** chat turns from everyone, newest first, with who asked, their role, the type, the request, whether it was refused, the question, the answer and the sources.

RETURNS
None. The page draws itself.

MODULE: dashboard/authorisation_chat.py

FUNCTION SIGNATURE
`render_authorisation_chat_page()`
PARAMETERS
None.
PURPOSE
Let anyone signed in ask the authorisation assistant a question, and show the conversation.
REQUIREMENTS

- The conversation is the signed-in person's own turns, oldest at the top, at most **config.CHAT_PAGE_TURN_LIMIT** of them, each answer badged with its type and shown with its sources where it has any.
- A question shorter than **config.CHAT_QUESTION_MINIMUM_CHARACTERS** is not sent, and one longer than **config.CHAT_QUESTION_MAXIMUM_CHARACTERS** cannot be typed.
- Asking calls the service once, with a waiting message, and the conversation is shown again with the new turn.

RETURNS
None. The page draws itself.

========== THE SIX ENTRY POINTS ==========

Six files sit at the top of the project, and each is run from the Project folder. The first four build a stage of the system and are run once, in order; the last two start the service and the dashboard. Each prints its progress for the person running it; nothing in the marking suite reads console output.

MODULE: run_database_setup.py
Creates the database, its tables and the accounts people sign in with. It is safe to run again.

FUNCTION SIGNATURE
`main()`
PARAMETERS
None.
PURPOSE
Give the system a database to write to, and the staff the accounts to sign in with.
REQUIREMENTS

- **config.DATABASE_DIRECTORY** and every table exist when the run finishes, and a second run neither fails nor duplicates anything.
- A database holding a table that lacks one of its columns, made by an earlier version, is refused, naming the table and the columns and saying that it and **config.CHECKPOINT_DATABASE_FILE** are to be deleted.
- Each account of **config.SEED_USER_ACCOUNTS** can sign in afterwards with the username, password and role given there, stored with its own random salt of **config.PASSWORD_SALT_BYTES** and the form **hash_password** gives; an account already present is left exactly as it is.
- The console names **config.APPLICATION_DATABASE_FILE**, the tables, each account as it is created, aligned by **config.USERNAME_WIDTH** and **config.DISPLAY_NAME_WIDTH**, and how many accounts were created and how many were already present.

RETURNS
None.
MAIN BLOCK

- Running the file performs the setup once; importing the module performs none of it.

MODULE: run_data_pipeline.py
Cleans the five raw extracts and writes the evidence that each one needed cleaning.

MODULE-LEVEL OBJECTS

- **DATASET_JOBS** — **list[tuple]**. The five cleaning jobs, in order: for each, the raw file, the cleaned file, the cleaning function, the answer column and the detail profiler for that kind of data.

FUNCTION SIGNATURE
`main()`

PARAMETERS
None.
PURPOSE
Turn the five raw extracts into the cleaned tables the modelling stage reads, and produce the quality report.
REQUIREMENTS

- Each extract's profile, filed under the raw file's name without its extension, is its general profile and its detail profile merged into one flat dictionary, worked out from the table **load_raw_dataset** reads and the table its cleaning returns.
- All five extracts are cleaned before any is saved, so a refusal leaves no partial set of cleaned files.
- Each cleaned table is saved to its path, and the report to **config.DATA_QUALITY_REPORT_FILE**.
- One console line per extract, aligned by **config.DATASET_NAME_WIDTH** and **config.ROW_COUNT_WIDTH**, says how many rows were received, kept and removed, and a closing line names the report.

RETURNS
None.
MAIN BLOCK

- Running the file cleans all five extracts and writes the report; importing the module cleans nothing.

MODULE: run_model_training.py
Trains the three models in one run, and saves them only when every one reaches the minimum the configuration sets for it.

FUNCTION SIGNATURE
`main()`

PARAMETERS
None.
PURPOSE
Produce the three saved models, their schemas and their scores.
REQUIREMENTS

- The approval model learns from the cleaned history training extract with each request's criteria counts under its own policy, and is scored on the cleaned held-out extract; its target is encoded by **config.APPROVAL_TARGET_ENCODING**, and a decision the encoding does not know is refused, naming the column and the values.
- The stay model learns from the cleaned stays training extract and is scored on the held-out one; the inputs of both models come from **build_feature_frame**.
- The cleaned sentences are split once, **config.SENTENCE_HELD_OUT_SHARE** held out, stratified by evidence type, with **config.RANDOM_SEED**.
- A model below its minimum, **config.MINIMUM_APPROVAL_ROC_AUC**, **config.MINIMUM_STAY_R2** or **config.MINIMUM_EVIDENCE_MACRO_F1**, stops the run before anything is saved, naming every shortfall.
- Otherwise the three models and the two schemas are saved to their paths in config, each schema exactly the columns its model was fitted on, in order, and **config.MODEL_METRICS_FILE** records under **approval**, **stay** and **evidence** the **algorithm**, **training_rows**, **held_out_rows**, **scores**, **minimum_required** and **scikit_learn_version**.
- The console gives each model's held-out score, training score and gap, then names the metrics file and says every model reaches its minimum.

RETURNS
None.
MAIN BLOCK

- Running the file trains, scores and saves the three models; importing the module trains nothing.

MODULE: run_knowledge_indexing.py
Builds the knowledge base and reads it back.

FUNCTION SIGNATURE
`main()`

PARAMETERS
None.
PURPOSE
Turn the sixteen documents into the store the whole system reads from, and show that reading it works.
REQUIREMENTS

- The store at **config.VECTOR_STORE_DIRECTORY** is rebuilt from every document in **config.DOCS_DIRECTORY**.

- The console says how many documents were read, how many sections were found, how many passages were stored and where the store is.
- Each policy's documentation to send, read back from the reopened store with **read_documentation_to_send**, is printed under its service code and its document; a passage that cannot be read stops the run with its refusal.

RETURNS
None.
MAIN BLOCK

- Running the file builds the store and reads it back; importing the module builds nothing.

MODULE: run_api_service.py
Starts the REST service, after checking that everything it needs is on disk.

MODULE-LEVEL OBJECTS

- **REQUIRED_BEFORE_STARTING** — **list[tuple[str, str]]**. Each path the service needs paired with what produces it: the three models, the two schemas, the two live files, the documents folder, the store's **config.VECTOR_STORE_FILE_NAME** and the database.

FUNCTION SIGNATURE
`main()`
PARAMETERS
None.
PURPOSE
Start the service where the earlier stages have been run, and explain itself where they have not.
REQUIREMENTS

- Every path in **REQUIRED_BEFORE_STARTING** is checked before anything is imported from the service.
- Where something is missing, the console says the service cannot start and names each missing path with what produces it.
- Otherwise the console says where the interactive documentation is, and the service starts on **config.API_HOST** and **config.API_PORT**, **8081**.

RETURNS
int. **0** once the service has run, **1** when it could not start.
MAIN BLOCK

- Running the file makes **main**'s result the process's exit status; importing the module starts nothing.

MODULE: run_dashboard.py
The dashboard's front door: the service check, signing in, and the navigation.

MODULE-LEVEL OBJECTS

- **PAGE_RENDERERS** — **dict[str, Callable]**. Each page name in config paired with the function that draws it.
- **PAGE_ICONS** — **dict[str, str]**. Each page name paired with its icon.
- **PAGE_URL_PATHS** — **dict[str, str]**. Each page name paired with its address path.

FUNCTION SIGNATURE
`main()`
PARAMETERS
None.
PURPOSE
Give the staff one screen to work from, with only the pages the signed-in person's role is offered.
REQUIREMENTS

- The page is titled **config.DASHBOARD_TITLE**; a service not answering at **config.API_BASE_URL** is reported, naming the address and how to start it, and nothing else is drawn.
- Until someone signs in, only the sign-in page is drawn, whichever page address was opened.
- Once signed in, the sidebar shows the person's name and role and a way to sign out that ends the session at the service; the pages offered are exactly those the service returned for the role, in that order, each with its renderer, icon and address path.

RETURNS
None. The dashboard draws itself.
MAIN BLOCK

- Running the file through Streamlit draws the dashboard on port **8080**; importing the module draws nothing.

OUTPUT
Running the six entry points in order produces the following, all inside the Project folder.

FILES CREATED

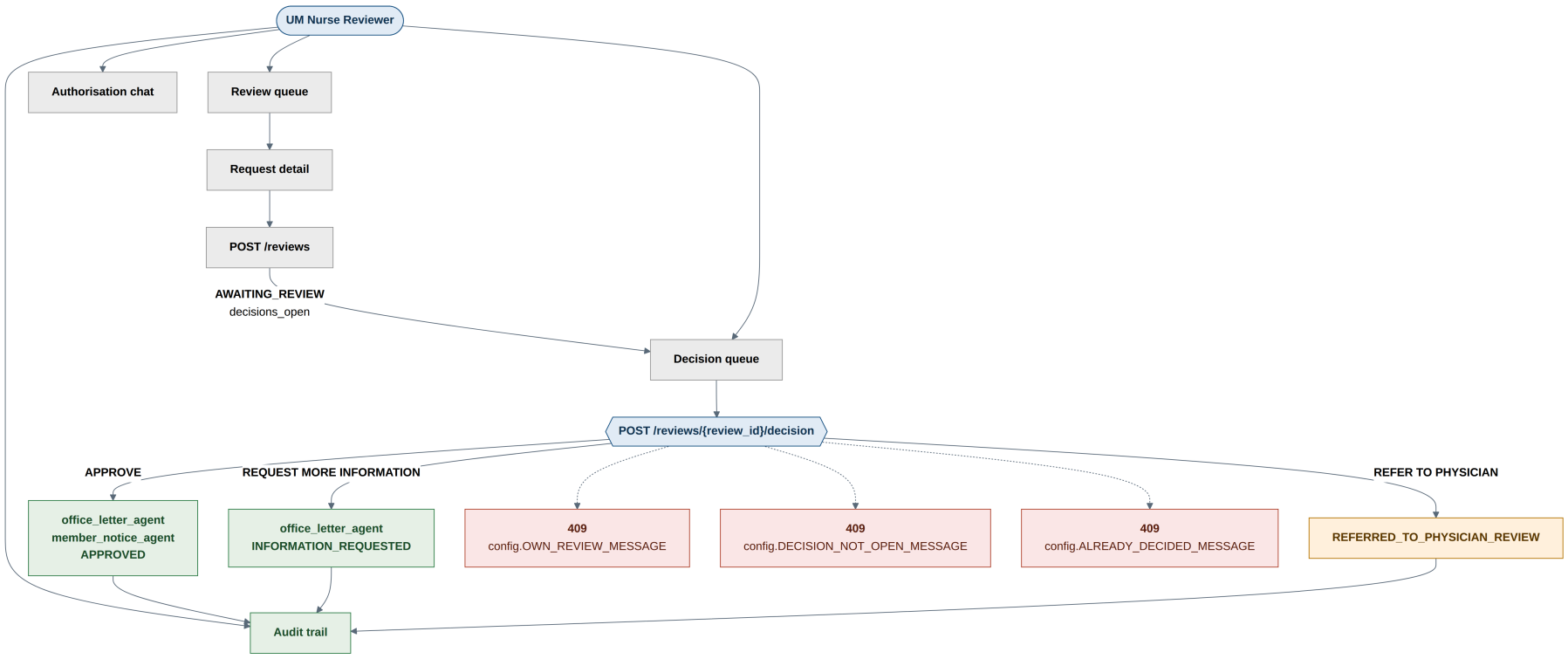
Path	Created by	Contents
database/clearauth.db	**run_database_setup.py**	The six tables the application writes to, and the fourteen seed accounts
processed/authorisation_history_train_clean.csv	**run_data_pipeline.py**	Cleaned approval training data
processed/authorisation_history_eval_clean.csv	**run_data_pipeline.py**	Cleaned approval held-out data
processed/surgical_stays_train_clean.csv	**run_data_pipeline.py**	Cleaned stay training data
processed/surgical_stays_eval_clean.csv	**run_data_pipeline.py**	Cleaned stay held-out data
processed/clinical_note_sentences_clean.csv	**run_data_pipeline.py**	Cleaned clinical sentences
output/data_quality_report.json	**run_data_pipeline.py**	How dirty each raw extract was, and what cleaning did to it
output/models/approval_model.pkl	**run_model_training.py**	The saved approval model
output/models/stay_model.pkl	**run_model_training.py**	The saved stay model
output/models/evidence_model.pkl	**run_model_training.py**	The saved evidence model, text handling included
output/models/approval_features.json	**run_model_training.py**	The columns the approval model expects, in order
output/models/stay_features.json	**run_model_training.py**	The columns the stay model expects, in order
output/model_metrics.json	**run_model_training.py**	Every score of every saved model
database/chroma_store/	**run_knowledge_indexing.py**	The searchable knowledge base
database/checkpoint.db	**run_api_service.py**	Every run's workflow state, checkpointed after each node, so that a run paused at the UM Nurse Reviewer's gate can be resumed

INFORMATION EACH COMMAND MUST REPORT

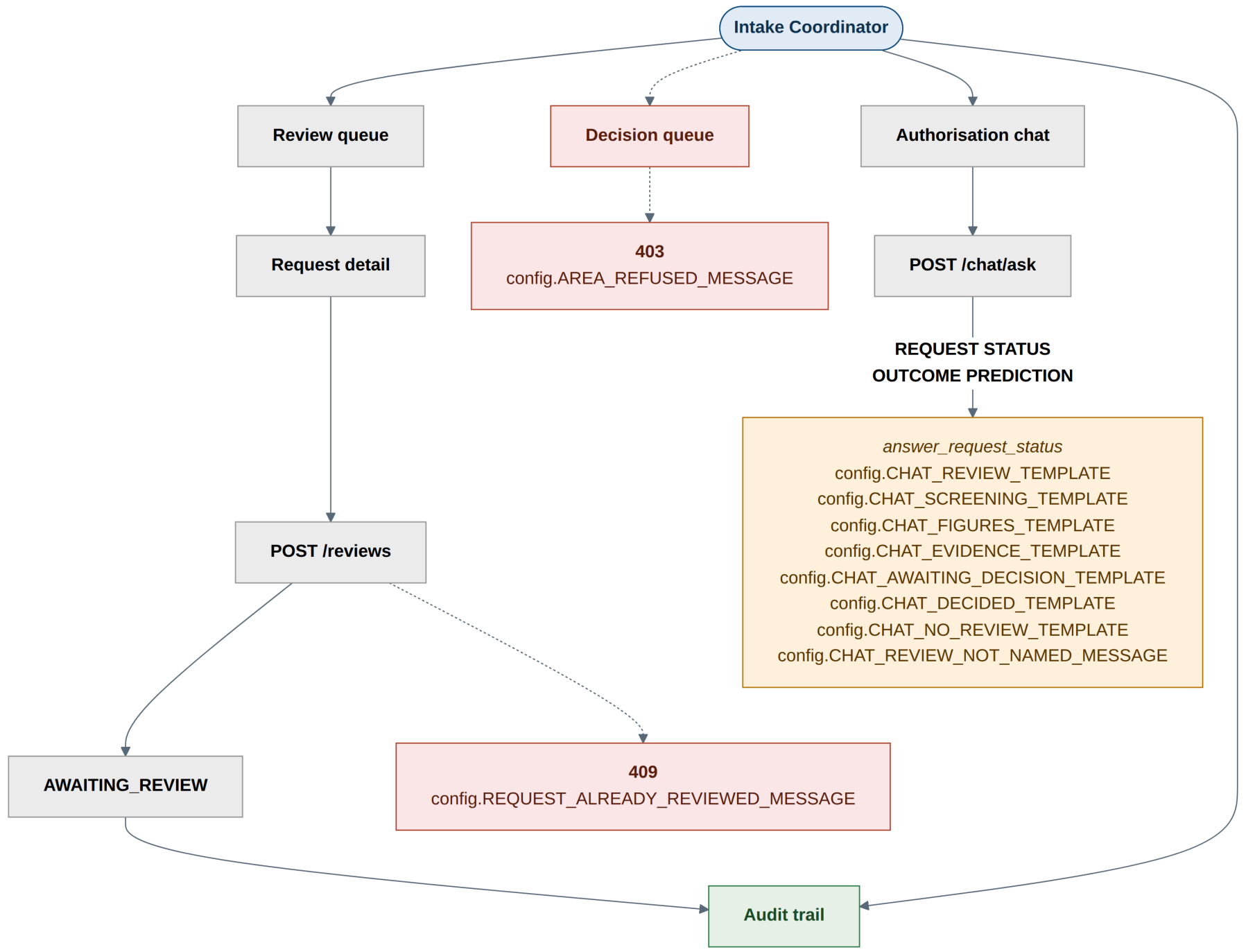
Command	What it must make clear
run_database_setup.py	Where the database is, which tables exist, and how many accounts were created and were already present
run_data_pipeline.py	For each of the five extracts, how many rows arrived and how many survived cleaning, and where the quality report was written
run_model_training.py	For each of the three models, the score on data it did not learn from, the score on data it did and the gap, and that every model reaches its minimum
run_knowledge_indexing.py	How many documents were read, how many sections and passages were stored, and each policy's documentation to send read back from the store
run_api_service.py	That the service is starting on port **8081**, and where its interactive documentation can be opened
run_dashboard.py	That the dashboard is listening on port **8080**, and the address at which it can be opened

FLOW DIAGRAM:

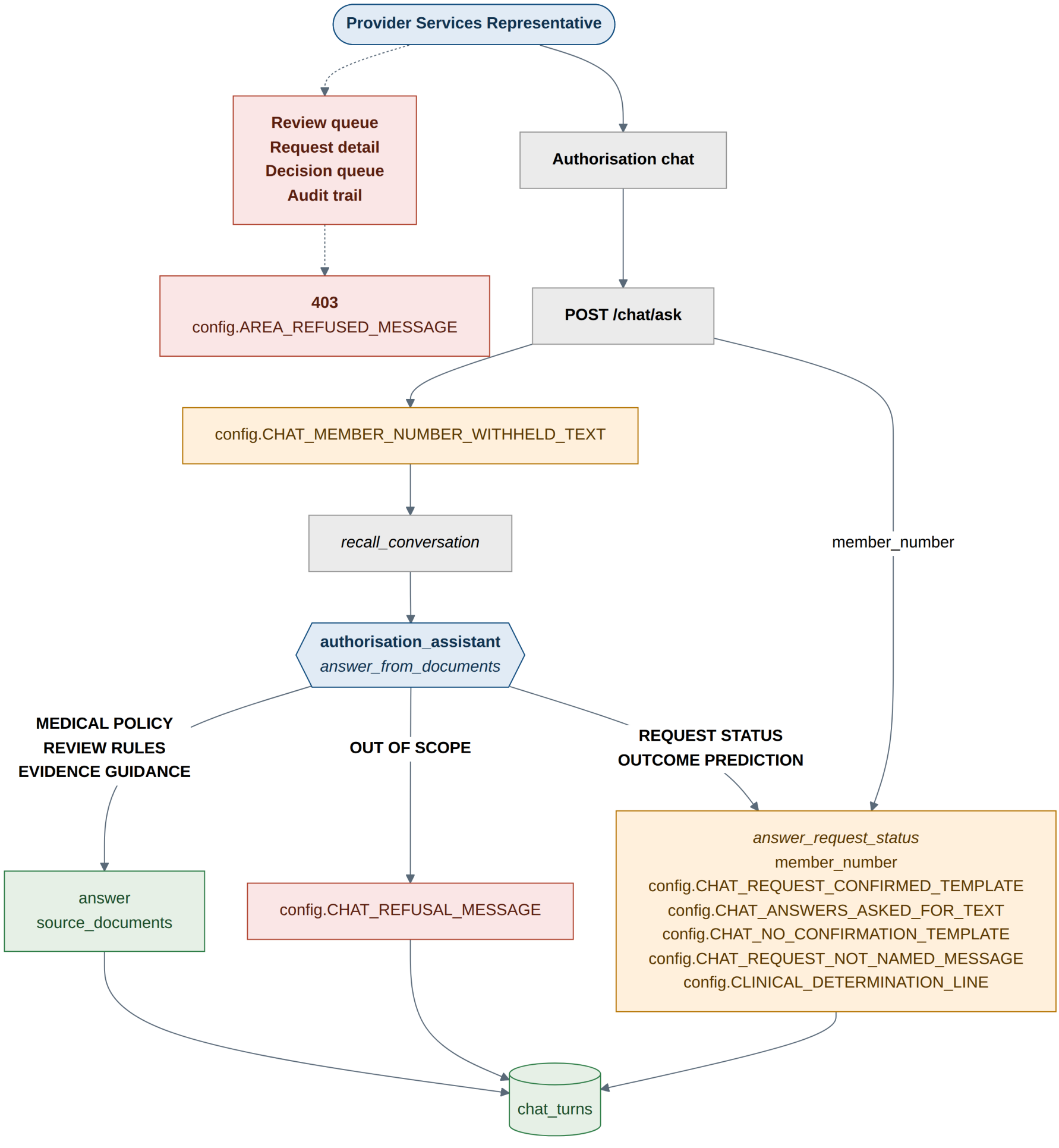
Role UM Nurse Reviewer:



Role Intake Coordinator:



Role Provider Services Representative:



Agent to Agent Communication Flow:

load_request_context

gather_request_context

fetch_request
fetch_request_documents
read_medical_policy
screen_request

read_document_evidence
link_evidence_to_criteria
flag_document_disagreements
estimate_approval_likelihood
estimate_expected_stay

Authorisation chat

review_planner
plan_review

authorisation_assistant
answer_from_documents

review_plan

assess_criteria

criteria_analyst
analyse_criteria

read_reviewer_checks

chat_turns

reviewer_checks

evidence_analyst
review_evidence

criteria_analysis

evidence_review

weigh_predictions

likelihood_analyst
analyse_predictions

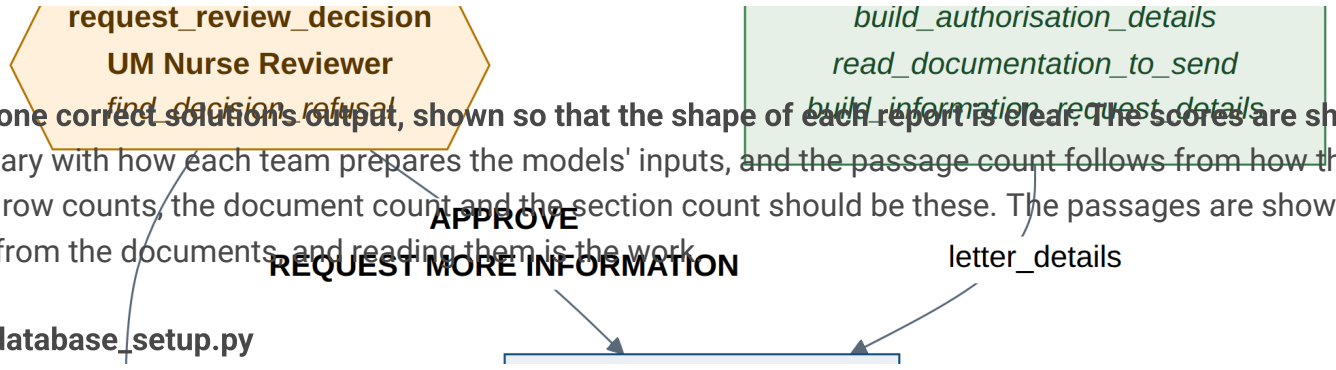
prediction_analysis

compose_review_recommendation

review_writer
write_review_recommendation

describe_expected_stay
join_labelled_response

AWAITING_REVIEW



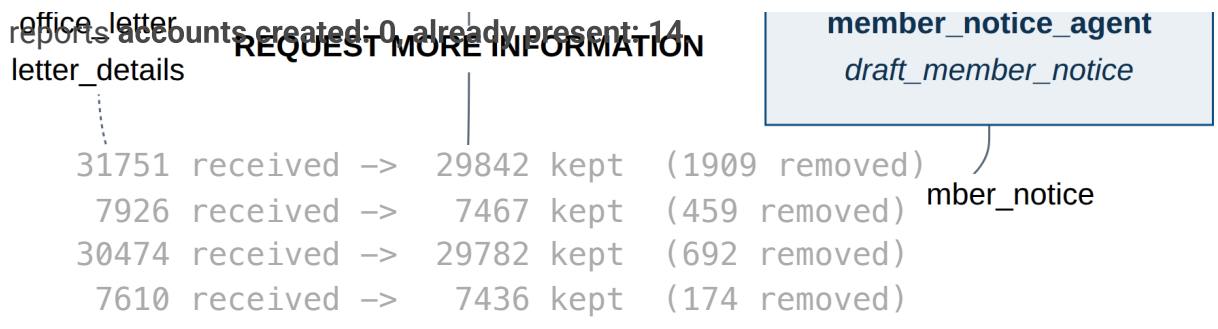
EXPECTED OUTPUT

The transcript below is one correct solution's output, shown so that the shape of each report is clear. The scores are shown as N.NNNN because they vary with how each team prepares the models' inputs, and the passage count follows from how the documents are cut. The row counts, the document count and the section count should be these. The passages are shown in outline only: they come from the documents, and reading them is the work.

ENTRY POINT 1 — run_database_setup.py

```
database: database/clearauth.db
  tables: agent_runs, authorisation_reviews, chat_turns, llm_calls, review_decisions, user_acco
  account created  maria    Maria Okafor         UM Nurse Reviewer
  account created  james    James Whitfield      UM Nurse Reviewer
  ...
  account created  dylan    Dylan Hughes         Provider Services Representative
  accounts created: 14, already present: 0
```

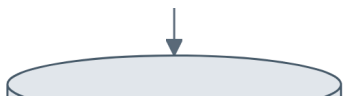
Run a second time it creates nothing and reports **accounts created: 0, already present: 14**.



ENTRY POINT 2 — run_data_pipeline.py

```
authorisation_history_train      31751 received ->  29842 kept  (1909 removed)
authorisation_history_eval        7926 received ->   7467 kept  (459 removed)
surgical_stays_train             30474 received ->  29782 kept  (692 removed)
surgical_stays_eval               7610 received ->   7436 kept  (174 removed)
clinical_note_sentences          36651 received ->  35695 kept  (956 removed)

quality report written to output/data_quality_report.json
```



ENTRY POINT 3 — run_model_training.py

```
training three models
  approval        held-out ROC AUC N.NNNN  (train N.NNNN, gap +N.NNNN)
  stay            held-out R2 N.NNNN  MAE N.NNNN nights  (train N.NNNN, gap +N.NNNN)
  evidence        held-out macro F1 N.NNNN  (train N.NNNN, gap +N.NNNN)

metrics written to output/model_metrics.json
every model reaches the minimum the configuration sets for it
```

A model below its minimum stops the run with a refusal naming it, and nothing is saved.

ENTRY POINT 4 — run_knowledge_indexing.py

```
building the knowledge index
    documents read:  16
    sections found:  145
    chunks stored:   NNN
    store location:  database/chroma_store

what each medical policy asks the requesting office to send
  RAD-LSM  Medical-Policy-Lumbar-Spine-MRI.pdf
      The requesting office should send with the request ...
  RAD-CSM  Medical-Policy-Cervical-Spine-MRI.pdf
      ...
  ORT-TKR  Medical-Policy-Total-Knee-Replacement.pdf
      ...
  BAR-WLS  Medical-Policy-Bariatric-Surgery.pdf
      ...
  SLP-PSG  Medical-Policy-In-Laboratory-Sleep-Study.pdf
      ...
```

Each passage is its own policy's section, whole and without its heading. Two services read back with the same passage, or a passage from another document, is the failure this stage exists to catch.

ENTRY POINT 5 — run_api_service.py

```
Starting ClearAuth AI API on port 8081. Interactive documentation: http://localhost:8081/docs
INFO:     Started server process [NNNN]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8081 (Press CTRL+C to quit)
```

Run with the saved approval model missing, it names what is missing and stops:

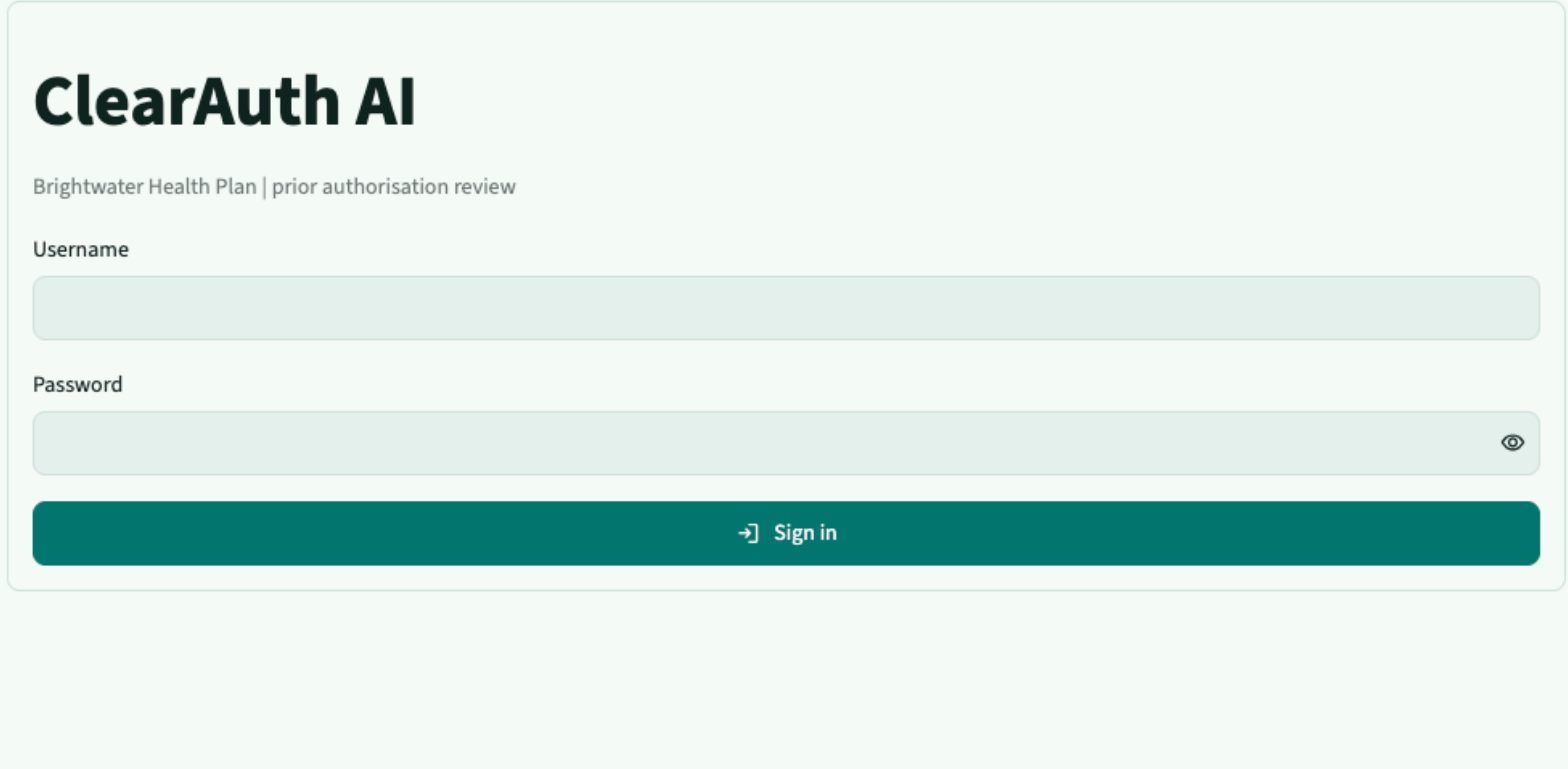
```
The service cannot start.
  output/models/approval_model.pkl is missing; it comes from run_model_training.py
```

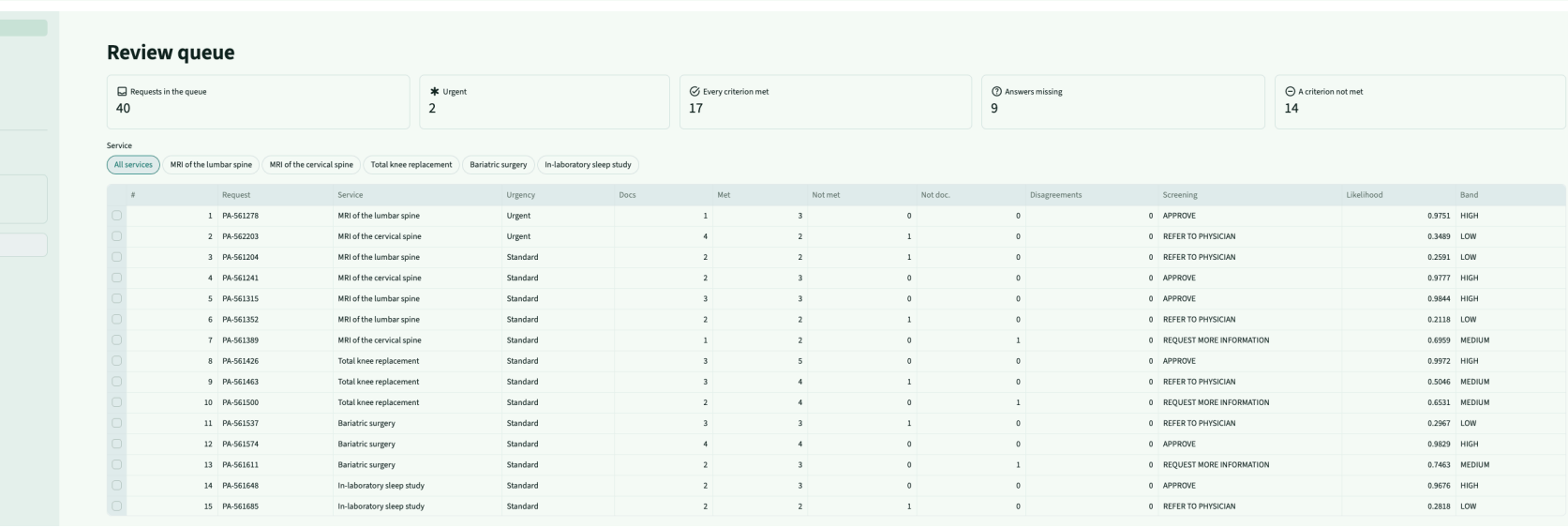
ENTRY POINT 6 — run_dashboard.py

```
  You can now view your Streamlit app in your browser.

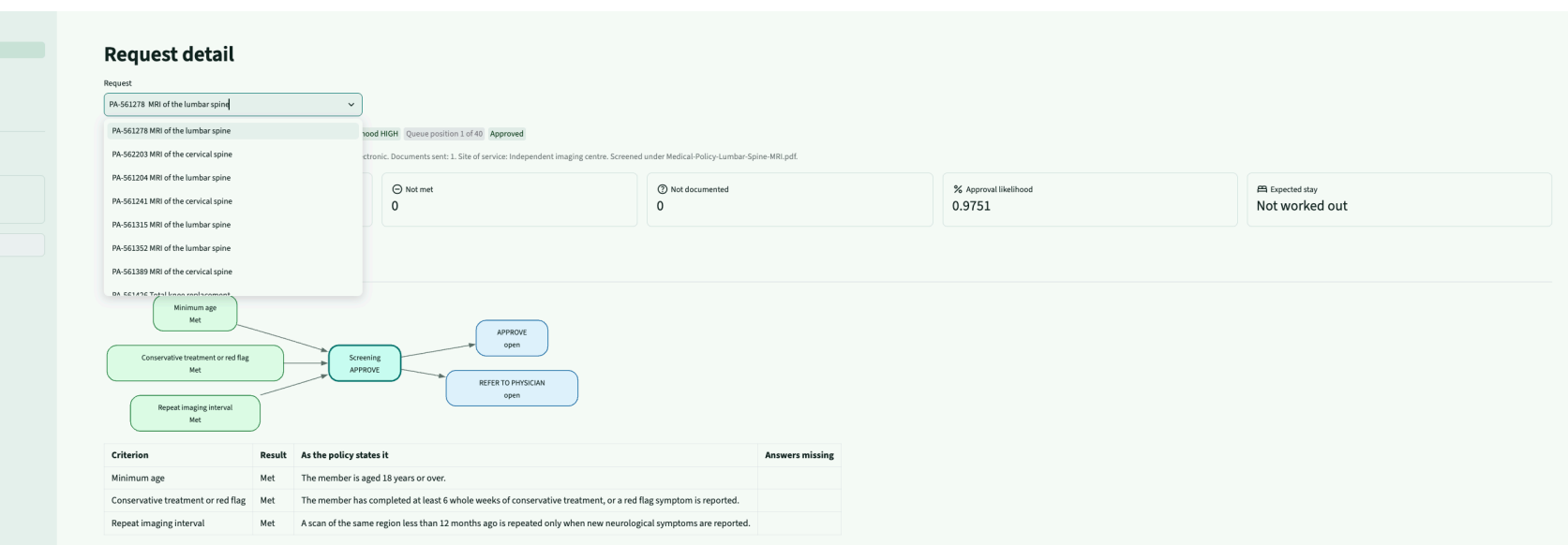
  Local URL: http://localhost:8080
  Network URL: http://N.N.N.N:8080
```

DASHBOARD OUTPUT:

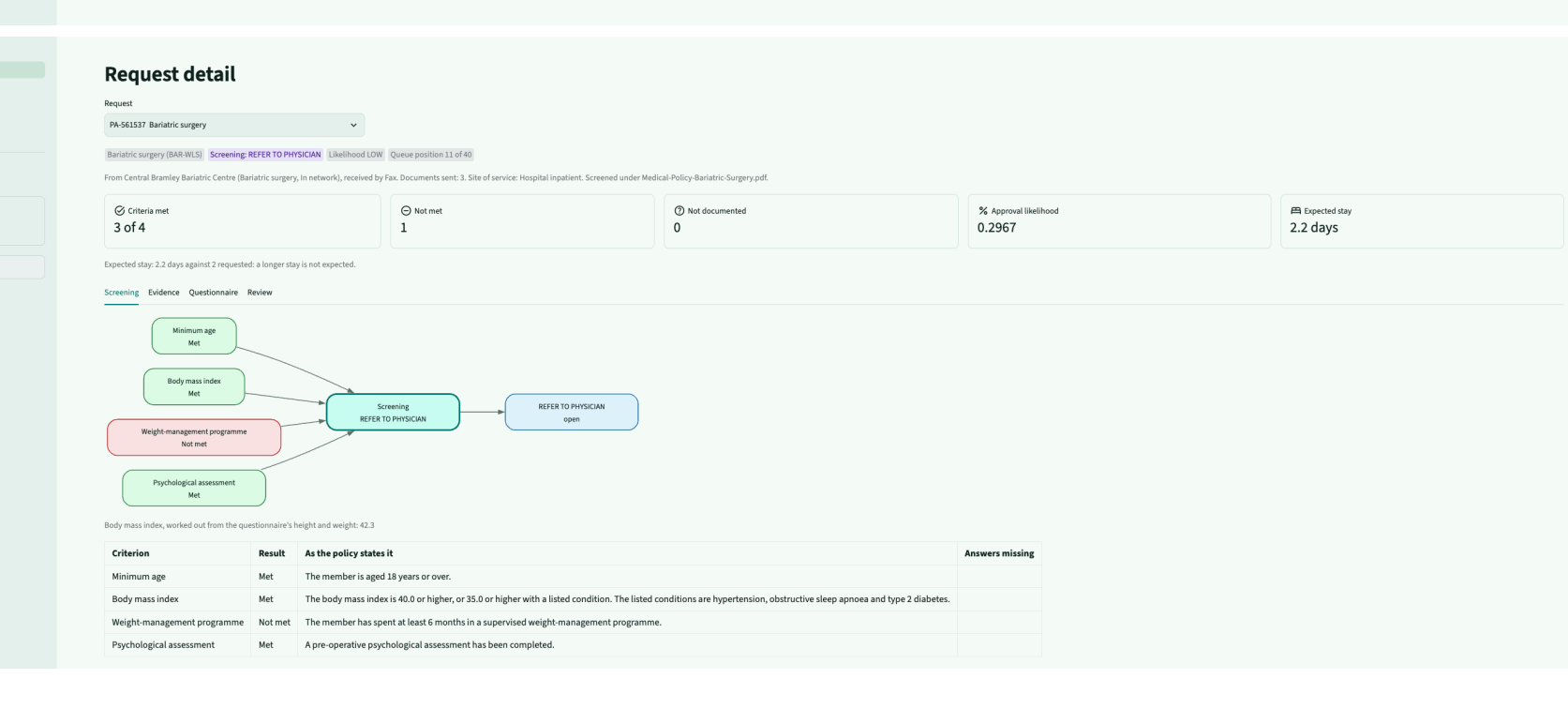




#	Request	Service	Urgency	Docs	Met	Not met	Not doc.	Disagreements	Screening	Likelihood	Band
1	PA-561278	MRI of the lumbar spine	Urgent	1	3	0	0	0	APPROVE	0.9751	HIGH
2	PA-562203	MRI of the cervical spine	Urgent	4	2	1	0	0	REFER TO PHYSICIAN	0.3489	LOW
3	PA-561204	MRI of the lumbar spine	Standard	2	2	1	0	0	REFER TO PHYSICIAN	0.2591	LOW
4	PA-561241	MRI of the cervical spine	Standard	2	3	0	0	0	APPROVE	0.9777	HIGH
5	PA-561315	MRI of the lumbar spine	Standard	3	3	0	0	0	APPROVE	0.9844	HIGH
6	PA-561352	MRI of the lumbar spine	Standard	2	2	1	0	0	REFER TO PHYSICIAN	0.2118	LOW
7	PA-561389	MRI of the cervical spine	Standard	1	2	0	1	0	REQUEST MORE INFORMATION	0.6959	MEDIUM
8	PA-561426	Total knee replacement	Standard	3	5	0	0	0	APPROVE	0.9972	HIGH
9	PA-561463	Total knee replacement	Standard	3	4	1	0	0	REFER TO PHYSICIAN	0.5046	MEDIUM
10	PA-561500	Total knee replacement	Standard	2	4	0	1	0	REQUEST MORE INFORMATION	0.6531	MEDIUM
11	PA-561537	Bariatric surgery	Standard	3	3	1	0	0	REFER TO PHYSICIAN	0.2967	LOW
12	PA-561574	Bariatric surgery	Standard	4	4	0	0	0	APPROVE	0.9829	HIGH
13	PA-561611	Bariatric surgery	Standard	2	3	0	1	0	REQUEST MORE INFORMATION	0.7463	MEDIUM
14	PA-561648	In-laboratory sleep study	Standard	2	3	0	0	0	APPROVE	0.9676	HIGH
15	PA-561685	In-laboratory sleep study	Standard	2	2	1	0	0	REFER TO PHYSICIAN	0.2818	LOW



Criterion	Result	As the policy states it	Answers missing
Minimum age	Met	The member is aged 18 years or over.	
Conservative treatment or red flag	Met	The member has completed at least 6 whole weeks of conservative treatment, or a red flag symptom is reported.	
Repeat imaging interval	Met	A scan of the same region less than 12 months ago is repeated only when new neurological symptoms are reported.	



Criterion	Result	As the policy states it	Answers missing
Minimum age	Met	The member is aged 18 years or over.	
Body mass index	Met	The body mass index is 40.0 or higher, or 35.0 or higher with a listed condition. The listed conditions are hypertension, obstructive sleep apnoea and type 2 diabetes.	
Weight-management programme	Not met	The member has spent at least 6 months in a supervised weight-management programme.	
Psychological assessment	Met	A pre-operative psychological assessment has been completed.	

Review queue
Request detail
Decision queue
Audit trail
Authorisation chat

ClearAuth AI

Maria Okafor
UM Nurse Reviewer

Sign out

Request detail

Request

PA-561537 Bariatric surgery

Bariatric surgery (BAR-WLS) Screening: REFER TO PHYSICIAN Likelihood LOW Queue position 11 of 40

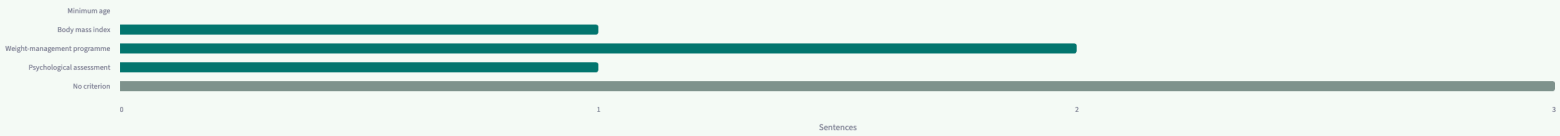
From Central Bramley Bariatric Centre (Bariatric surgery, In network), received by Fax. Documents sent: 3. Site of service: Hospital inpatient. Screened under Medical-Policy-Bariatric-Surgery.pdf.

Criteria met	Not met	Not documented	Approval likelihood	Expected stay
3 of 4	1	0	0.2967	2.2 days

Expected stay: 2.2 days against 2 requested: a longer stay is not expected.

Screening Evidence Questionnaire Review

Sentences of the documents sent that bear on each criterion of Medical-Policy-Bariatric-Surgery.pdf. The questionnaire decides each result; the documents are evidence.



Specialist letter, NTE-880852

Background I saw this 38-year-old woman in clinic today.
Bears on no criterion

Functional limitation She is breathless on minimal exertion.
Bears on no criterion

Failed medication A course of orlistat did not lead to sustained weight loss.
Bears on Weight-management programme

Other condition Background of hypertension on two medicines.
Bears on Body mass index

Background Please contact me if you need any further information.
Bears on no criterion

Dietitian report, NTE-880857

Weight-management programme Mrs Lindqvist has attended a supervised weight-management programme for 4 months.
Bears on Weight-management programme

Psychology report, NTE-880865

Prior investigation The psychological assessment has been completed and she is suitable for surgery.
Bears on Psychological assessment

Review queue
Request detail
Decision queue
Audit trail
Authorisation chat

ClearAuth AI

Maria Okafor
UM Nurse Reviewer

Sign out

Request detail

Request

PA-561537 Bariatric surgery

Bariatric surgery (BAR-WLS) Screening: REFER TO PHYSICIAN Likelihood LOW Queue position 11 of 40

From Central Bramley Bariatric Centre (Bariatric surgery, In network), received by Fax. Documents sent: 3. Site of service: Hospital inpatient. Screened under Medical-Policy-Bariatric-Surgery.pdf.

Criteria met	Not met	Not documented	Approval likelihood	Expected stay
3 of 4	1	0	0.2967	2.2 days

Expected stay: 2.2 days against 2 requested: a longer stay is not expected.

Screening Evidence Questionnaire Review

Criterion	Question	Answer, as the Rules read it
Minimum age	the member's age in years	38
Body mass index	the member's height, in centimetres	167.6
Body mass index	the member's weight, in kilograms	118.8
Body mass index	every long-term condition the member has, or that there is none	Hypertension
Weight-management programme	the whole months of the supervised weight-management programme	4
Psychological assessment	whether the pre-operative psychological assessment is completed	Completed

Review queue
Request detail
Decision queue
Audit trail
Authorisation chat

ClearAuth AI

Maria Okafor
UM Nurse Reviewer

Sign out

Request detail

Request

PA-561537 Bariatric surgery

Bariatric surgery (BAR-WLS) Screening: REFER TO PHYSICIAN Likelihood LOW Queue position 11 of 40

From Central Bramley Bariatric Centre (Bariatric surgery, In network), received by Fax. Documents sent: 3. Site of service: Hospital inpatient. Screened under Medical-Policy-Bariatric-Surgery.pdf.

Criteria met	Not met	Not documented	Approval likelihood	Expected stay
3 of 4	1	0	0.2967	2.2 days

Expected stay: 2.2 days against 2 requested: a longer stay is not expected.

Screening Evidence Questionnaire Review

What do you want looked at?

why hypertension for this case?

Start the review

The agents are reviewing the request: 5 model calls. (4.7 seconds)

Review queue
Request detail
Decision queue
Audit trail
Authorisation chat

ClearAuth AI

Maria Okafor
UM Nurse Reviewer

Sign out

Decision queue

2 waiting for a decision

Review to decide

PA-561537 | Bariatric surgery, Standard

PA-561537 | Bariatric surgery, Standard
PA-561981 | MRI of the lumbar spine, Standard

Met	Not met	Not documented	Approval likelihood
3	1	0	0.2967

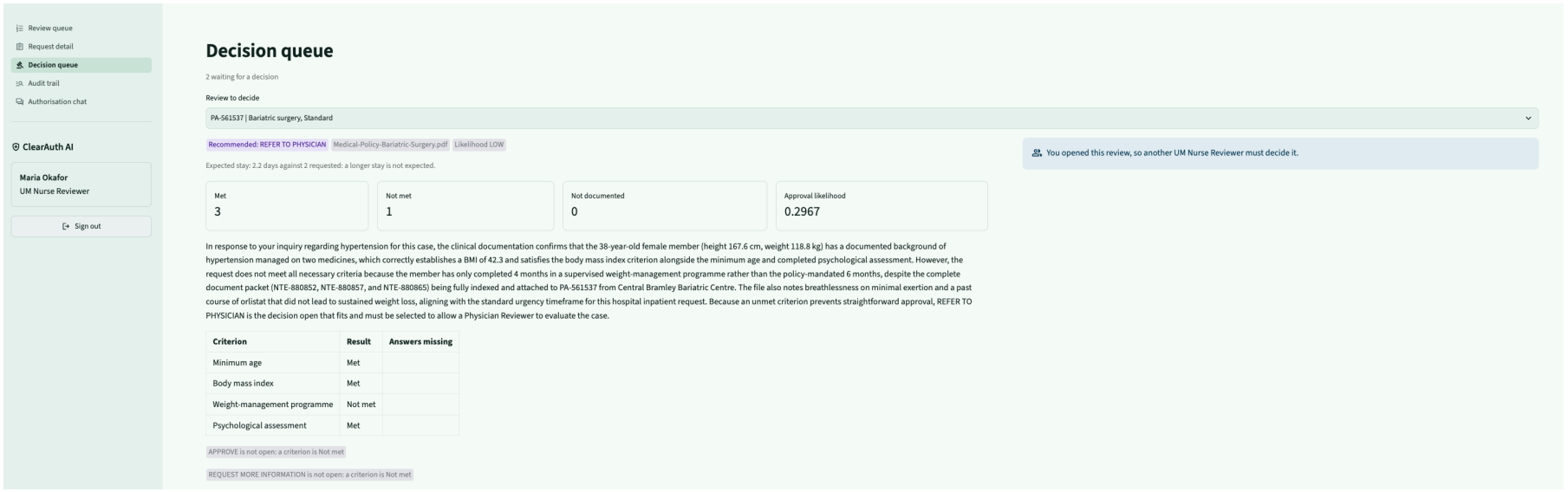
In response to your inquiry regarding hypertension for this case, the clinical documentation confirms that the 38-year-old female member (height 167.6 cm, weight 118.8 kg) has a documented background of hypertension managed on two medicines, which correctly establishes a BMI of 42.3 and satisfies the body mass index criterion alongside the minimum age and completed psychological assessment. However, the request does not meet all necessary criteria because the member has only completed 4 months in a supervised weight-management programme rather than the policy-mandated 6 months, despite the complete document packet (NTE-880852, NTE-880857, and NTE-880865) being fully indexed and attached to PA-561537 from Central Bramley Bariatric Centre. The file also notes breathlessness on minimal exertion and a past course of orlistat that did not lead to sustained weight loss, aligning with the standard urgency timeframe for this hospital inpatient request. Because an unmet criterion prevents straightforward approval, REFER TO PHYSICIAN is the decision open that fits and must be selected to allow a Physician Reviewer to evaluate the case.

Criterion	Result	Answers missing
Minimum age	Met	
Body mass index	Met	
Weight-management programme	Not met	
Psychological assessment	Met	

APPROVE is not open: a criterion is Not met

REQUEST MORE INFORMATION is not open: a criterion is Not met

The review, agent by agent, opened by maria


Review queue
Request detail
Decision queue
Audit trail
Authorisation chat
ClearAuth AI
Maria Okafor
UM Nurse Reviewer
Sign out
Decision queue
2 waiting for a decision
Review to decide
PA-561537 | Bariatric surgery, Standard
Recommended: REFER TO PHYSICIAN
Medical-Policy-Bariatric-Surgery.pdf
Likelihood LOW
Expected stay: 2.2 days against 2 requested: a longer stay is not expected.
You opened this review, so another UM Nurse Reviewer must decide it.
Met
3
Not met
1
Not documented
0
Approval likelihood
0.2967
In response to your inquiry regarding hypertension for this case, the clinical documentation confirms that the 38-year-old female member (height 167.6 cm, weight 118.8 kg) has a documented background of hypertension managed on two medicines, which correctly establishes a BMI of 42.3 and satisfies the body mass index criterion alongside the minimum age and completed psychological assessment. However, the request does not meet all necessary criteria because the member has only completed 4 months in a supervised weight-management programme rather than the policy-mandated 6 months, despite the complete document packet (NTE-880852, NTE-880857, and NTE-880865) being fully indexed and attached to PA-561537 from Central Bramley Bariatric Centre. The file also notes breathlessness on minimal exertion and a past course of orlistat that did not lead to sustained weight loss, aligning with the standard urgency timeframe for this hospital inpatient request. Because an unmet criterion prevents straightforward approval, REFER TO PHYSICIAN is the decision open that fits and must be selected to allow a Physician Reviewer to evaluate the case.
Criterion
Result
Answers missing
Minimum age
Met
Body mass index
Met
Weight-management programme
Not met
Psychological assessment
Met
APPROVE is not open: a criterion is Not met
REQUEST MORE INFORMATION is not open: a criterion is Not met


Review queue
Request detail
Decision queue
Audit trail
Authorisation chat
ClearAuth AI
Maria Okafor
UM Nurse Reviewer
Sign out
The review, agent by agent, opened by maria
What was asked
why hypertension for this case?
Plan
PRIORITY: Verifying comorbidity documentation for standard inpatient bariatric surgery eligibility
CHECKS: 1. Establish whether hypertension is documented in the submitted specialist letter, dietitian report, or psychology report.
2. Confirm the complete document packet (NTE-880852, NTE-880857, and NTE-880865) has been fully indexed and attached to PA-561537.
3. Verify that the requested inpatient site of service aligns with the clinical details provided by Central Bramley Bariatric Centre.
4. Check that the standard urgency timeframe is maintained for this faxed submission regarding member age 38.
RATIONALE: These checks directly address the clinical question raised about hypertension while ensuring all submitted evidence is accounted for before the UM Nurse Reviewer evaluates the case against the bariatric surgery policy. Focusing on documentation completeness and site of service prevents premature screening errors and administrative delays.
Criteria
CRITERIA MET: - 1 Minimum age. The member is aged 18 years or over. (the member's age in years: 38)
- 2 Body mass index. The body mass index is 40.0 or higher, or 35.0 or higher with a listed condition. The listed conditions are hypertension, obstructive sleep apnoea and type 2 diabetes. (the member's height, in centimetres: 167.6; the member's weight, in kilograms: 118.8; every long-term condition the member has, or that there is none: Hypertension; the body mass index worked out from them and kept to one decimal: 42.3)
- 4 Psychological assessment. A pre-operative psychological assessment has been completed. (whether the pre-operative psychological assessment is completed: Completed)
CRITERIA NOT MET: - 3 Weight-management programme. The member has spent at least 6 months in a supervised weight-management programme. (the whole months of the supervised weight-management programme: 4) - The policy requires at least 6 months in a supervised weight-management programme, but the member has only completed 4 months.
NOT DOCUMENTED: - None
CRITERIA SUMMARY: The member satisfies the age, body mass index, and psychological assessment criteria for bariatric surgery under PA-561537. However, the request does not meet the weight-management programme criterion, as the member has only completed 4 months instead of the required 6 months.
Evidence
EVIDENCE FOUND: Background: NTE-880852
Functional limitation: NTE-880852
Failed medication: NTE-880852
Other condition: NTE-880852
Weight-management programme: NTE-880857
Prior investigation: NTE-880865
DISAGREEMENTS: None
CHECKS TO MAKE: 1. Name each medicine and what happened: no benefit, a side effect, or the maximum dose reached. [A course of orlistat did not lead to sustained weight loss.]
2. Check each medicine was for the condition of the request. [The specialist letter documents a course of orlistat for weight management relative to the bariatric surgery request.]
3. Never turn a list of medicines in a letter into weeks of treatment; the questionnaire decides. [The questionnaire records 4 months for the supervised weight-management programme.]
4. Note a medicine still taken, and whether it helps at all. [Orlistat is noted as a past course that did not lead to sustained weight loss, with background hypertension managed on two medicines.]
5. Gather the failed medicines into the review for a Physician Reviewer. [Orlistat is noted as a failed medication for the Physician Reviewer.]
6. Read the score on the questionnaire when the policy asks for one. [No specific functional limitation score is recorded on the questionnaire.]
7. Read each sentence of this kind for a score, and compare it with the questionnaire's. [The specialist letter notes breathlessness on minimal exertion without providing a numerical score.]
8. Treat a score beyond its scale as not documented, never as a value. [No out-of-scale score is present in the documentation.]
9. Describe the limitation in the review in the writer's own words. [The member is breathless on minimal exertion.]
10. Note a limitation that seems out of keeping with the score, for a Physician Reviewer. [Breathlessness on minimal exertion is documented without a conflicting questionnaire score.]
11. Match each investigation to the questionnaire answer it bears on. [The psychology report assessment matches the questionnaire answer confirming the pre-operative psychological assessment is completed.]
12. Compare each grade, interval and result with the questionnaire, and flag any difference. [The completed psychological assessment in the report aligns with the questionnaire answer.]
13. For a recent scan of the same region, look for new neurological symptoms since it. [No recent scan or neurological symptoms are applicable to this psychological assessment.]
14. Read a home sleep test as inconclusive, negative, positive or not done. [No home sleep test is part of this prior investigation evidence.]
15. Never take an assessment that is booked or pending as completed. [The psychological assessment is documented as completed and suitable for surgery rather than pending.]

DISAGREEMENTS: None

CHECKS TO MAKE: 1. Name each medicine and what happened: no benefit, a side effect, or the maximum dose reached. [A course of orlistat did not lead to sustained weight loss.]
2. Check each medicine was for the condition of the request. [The specialist letter documents a course of orlistat for weight management relative to the bariatric surgery request.]
3. Never turn a list of medicines in a letter into weeks of treatment; the questionnaire decides. [The questionnaire records 4 months for the supervised weight-management programme.]
4. Note a medicine still taken, and whether it helps at all. [Orlistat is noted as a past course that did not lead to sustained weight loss, with background hypertension managed on two medicines.]
5. Gather the failed medicines into the review for a Physician Reviewer. [Orlistat is noted as a failed medication for the Physician Reviewer.]
6. Read the score on the questionnaire when the policy asks for one. [No specific functional limitation score is recorded on the questionnaire.]
7. Read each sentence of this kind for a score, and compare it with the questionnaire's. [The specialist letter notes breathlessness on minimal exertion without providing a numerical score.]
8. Treat a score beyond its scale as not documented, never as a value. [No out-of-scale score is present in the documentation.]
9. Describe the limitation in the review in the writer's own words. [The member is breathless on minimal exertion.]
10. Note a limitation that seems out of keeping with the score, for a Physician Reviewer. [Breathlessness on minimal exertion is documented without a conflicting questionnaire score.]
11. Match each investigation to the questionnaire answer it bears on. [The psychology report assessment matches the questionnaire answer confirming the pre-operative psychological assessment is completed.]
12. Compare each grade, interval and result with the questionnaire, and flag any difference. [The completed psychological assessment in the report aligns with the questionnaire answer.]
13. For a recent scan of the same region, look for new neurological symptoms since it. [No recent scan or neurological symptoms are applicable to this psychological assessment.]
14. Read a home sleep test as inconclusive, negative, positive or not done. [No home sleep test is part of this prior investigation evidence.]
15. Never take an assessment that is booked or pending as completed. [The psychological assessment is documented as completed and suitable for surgery rather than pending.]
16. Read the programme's months on the questionnaire. [The questionnaire states 4 months for the supervised weight-management programme.]
17. Read each sentence of this kind for the programme's length in months. [The dietitian report states Mrs Lindqvist has attended a supervised weight-management programme for 4 months.]
18. Check the programme was supervised by a dietitian or a physician, with contact each month. [The dietitian report confirms attendance at a supervised weight-management programme for 4 months.]
19. Flag a length in a document that differs from the questionnaire's. [The 4-month length in the dietitian report matches the 4 months on the questionnaire.]
20. Never let weight lost lengthen a programme: months decide. [The programme duration is fixed at 4 months matching the questionnaire.]
21. Read the other conditions on the questionnaire as the plan's twelve names. [The questionnaire lists hypertension as the member's long-term condition.]
22. Match each condition named in the documents to one of the twelve. [The specialist letter documents hypertension on two medicines, matching the questionnaire.]
23. Flag a condition in the documents that the questionnaire does not list. [No unlisted condition is presented in the documents.]
24. Check whether one of the policy's listed conditions is among those on the questionnaire. [Hypertension is listed on the questionnaire.]
25. For an inpatient operation, note diabetes and the number of conditions for the expected stay. [Hypertension is noted as a background condition for the inpatient bariatric surgery request, with no diabetes documented.]

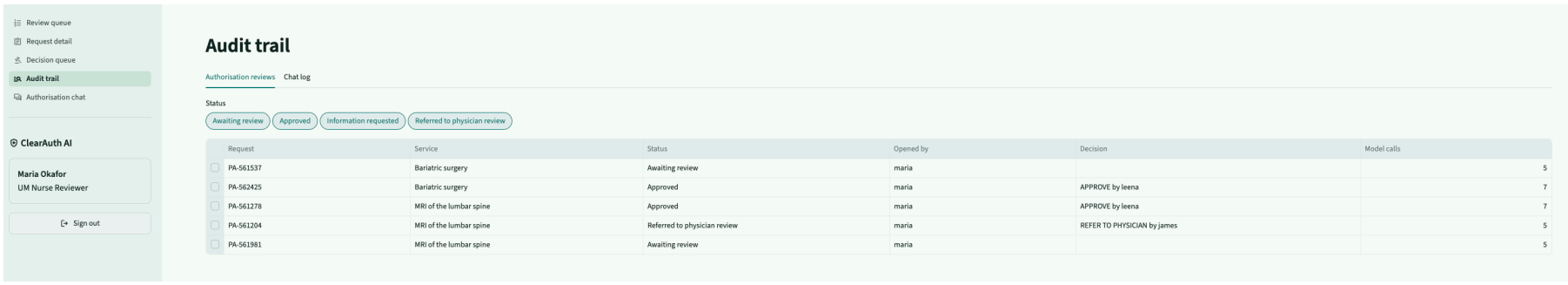
EVIDENCE SUMMARY: The clinical documentation includes a specialist letter noting a 38-year-old woman with breathlessness on minimal exertion, a past course of orlistat that did not lead to sustained weight loss, and background hypertension on two medicines. A dietitian report confirms 4 months of attendance at a supervised weight-management programme, perfectly matching the questionnaire. Additionally, a psychology report verifies that the pre-operative psychological assessment is completed and that the member is suitable for surgery.

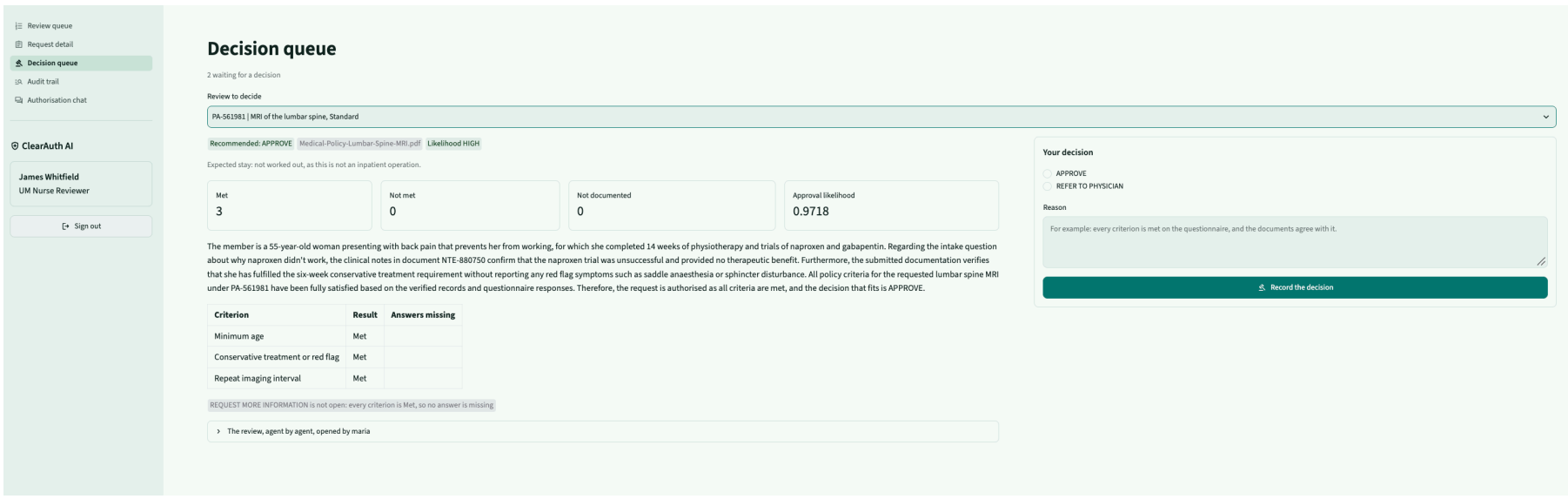
Predictions

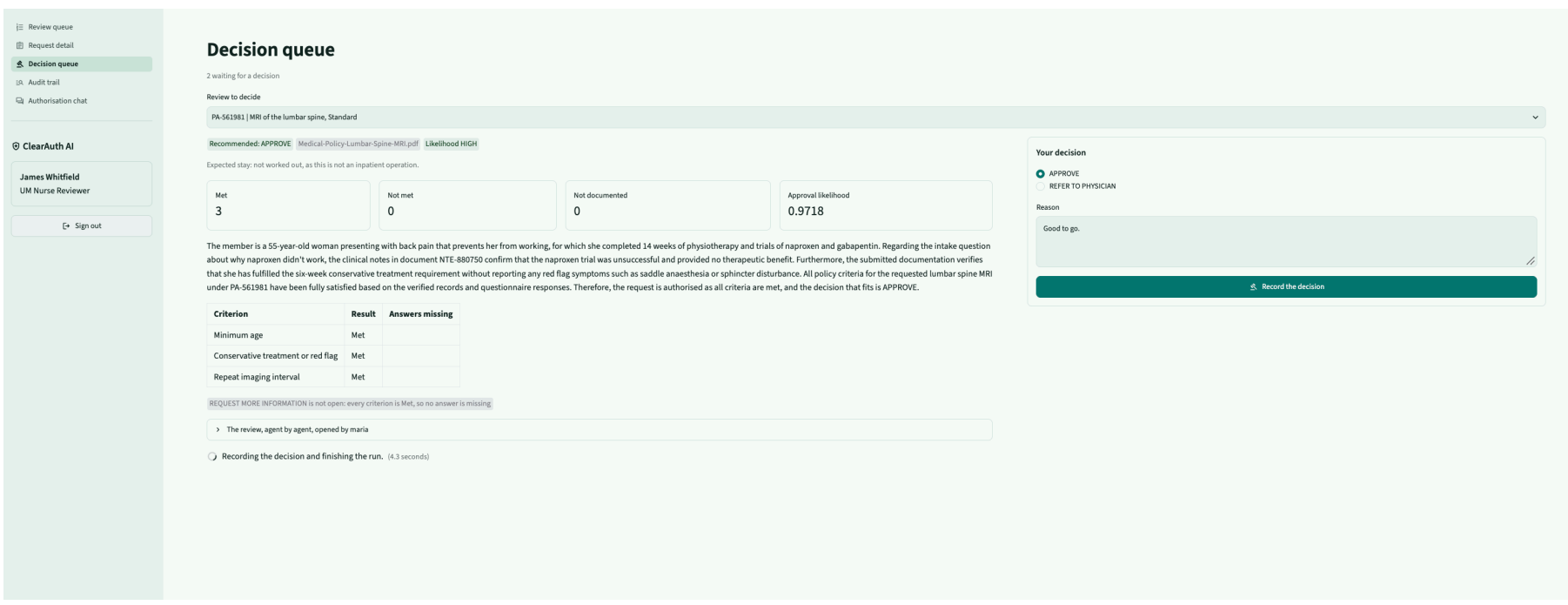
LIKELIHOOD: The approval likelihood is 0.2967, which falls into the LOW band. It is worked out from factors including the member's age of 38, 3 criteria met, 1 criterion not met, 3 documents sent, and the hospital inpatient site of service.

EXPECTED STAY: The expected stay is 2.2 days against 2 requested, so a longer stay is not expected because it is not more than 1 day above the days requested. It rests on clinical and operational inputs including an ASA class of 3, a BMI of 42.3, hypertension, a sleeve gastrectomy, and a Monday planned weekday.

PREDICTION SUMMARY: The low approval likelihood of 0.2967 reflects that one clinical criterion remains unmet alongside the inpatient site of service. Meanwhile, the expected stay of 2.2 days closely aligns with the 2 days requested for this sleeve gastrectomy.







Review queue
Request detail
Decision queue
Audit trail
Authorisation chat

ClearAuth AI

James Whitfield
UM Nurse Reviewer

Sign out

Decision queue

PA-561981 approved: the provider authorisation letter and the member notice are written.

1 waiting for a decision

Review to decide

PA-561537 | Bariatric surgery, Standard

Recommended: REFER TO PHYSICIAN · Medical-Policy-Bariatric-Surgery.pdf · Likelihood LOW

Expected stay: 2.2 days against 2 requested: a longer stay is not expected.

Met	Not met	Not documented	Approval likelihood
3	1	0	0.2967

In response to your inquiry regarding hypertension for this case, the clinical documentation confirms that the 38-year-old female member (height 167.6 cm, weight 118.8 kg) has a documented background of hypertension managed on two medicines, which correctly establishes a BMI of 42.3 and satisfies the body mass index criterion alongside the minimum age and completed psychological assessment. However, the request does not meet all necessary criteria because the member has only completed 4 months in a supervised weight-management programme rather than the policy-mandated 6 months, despite the complete document packet (NTE-880852, NTE-880857, and NTE-880865) being fully indexed and attached to PA-561537 from Central Bramley Bariatric Centre. The file also notes breathlessness on minimal exertion and a past course of orlistat that did not lead to sustained weight loss, aligning with the standard urgency timeframe for this hospital inpatient request. Because an unmet criterion prevents straightforward approval, REFER TO PHYSICIAN is the decision open that fits and must be selected to allow a Physician Reviewer to evaluate the case.

Criterion	Result	Answers missing
Minimum age	Met	
Body mass index	Met	
Weight-management programme	Not met	
Psychological assessment	Met	

APPROVE is not open: a criterion is Not met

REQUEST MORE INFORMATION is not open: a criterion is Not met

› The review, agent by agent, opened by maria

Your decision

REFER TO PHYSICIAN

Reason

For example: every criterion is met on the questionnaire, and the documents agree with it.

Record the decision

Review queue
Request detail
Decision queue
Audit trail
Authorisation chat

ClearAuth AI

James Whitfield
UM Nurse Reviewer

Sign out

Audit trail

Authorisation reviews | Chat log

Status

Awaiting review · Approved · Information requested · Referred to physician review

Request	Service	Status	Opened by	Decision	Model calls
PA-561537	Bariatric surgery	Awaiting review	maria		5
PA-562425	Bariatric surgery	Approved	maria	APPROVE by leena	7
PA-561278	MRI of the lumbar spine	Approved	maria	APPROVE by leena	7
PA-561204	MRI of the lumbar spine	Referred to physician review	maria	REFER TO PHYSICIAN by james	5
PA-561981	MRI of the lumbar spine	Approved	maria	APPROVE by james	7

Review queue
Request detail
Decision queue
Audit trail
Authorisation chat

ClearAuth AI

James Whitfield
UM Nurse Reviewer

Sign out

Authorisation chat

What does the lumbar spine MRI policy require before a scan is approved?

MEDICAL POLICY

According to the medical policy for MRI of the lumbar spine (RAD-LSM), the service requires that the member is aged 18 years or over. The policy also requires that the member has completed at least 6 whole weeks of conservative treatment, or that a red flag symptom is reported. Additionally, a scan of the same region less than 12 months ago is repeated only when new neurological symptoms are reported.

› Sources

What should a reviewer check when an imaging report is sent?

EVIDENCE GUIDANCE

According to the evidence guide for prior investigations, the reviewer makes five checks in a specific order when an imaging report is sent. First, the reviewer matches each investigation to the questionnaire answer it bears on. Second, the reviewer compares each grade, interval and result with the questionnaire and flags any difference. Third, for a recent scan of the same region, the reviewer looks for new neurological symptoms since it. Fourth, the reviewer reads a home sleep test as inconclusive, negative, positive or not done. Fifth, the reviewer never takes an assessment that is booked or pending as completed.

› Sources

What does the lumbar spine MRI policy require before a scan is approved?

MEDICAL POLICY

According to the medical policy for MRI of the lumbar spine (RAD-LSM), the service requires that the member is aged 18 years or over. The policy also requires that the member has completed at least 6 whole weeks of conservative treatment, or that a red flag symptom is reported. Additionally, a scan of the same region less than 12 months ago is repeated only when new neurological symptoms are reported.

› Sources

What does the lumbar spine MRI policy require before a scan is approved?

MEDICAL POLICY

According to the medical policy for MRI of the lumbar spine (RAD-LSM), the service requires that the member is aged 18 years or over. The policy also requires that the member has completed at least 6 whole weeks of conservative treatment, or that a red flag symptom is reported. Additionally, a scan of the same region less than 12 months ago is repeated only when new neurological symptoms are reported.

› Sources

And how many weeks does it ask for?

MEDICAL POLICY

According to the medical policy for MRI of the lumbar spine (RAD-LSM), the service requires that the member has completed at least 6 whole weeks of conservative treatment, or that a red flag symptom is reported.

› Sources

What is the status of PA-561204?

REQUEST STATUS

REV-54562729CBAF, request PA-561204, MRI of the lumbar spine (RAD-LSM), reviewed under Medical-Policy-Lumbar-Spine-MRI.pdf and opened by maria: Referred to physician review.
Screening REFER TO PHYSICIAN: Minimum age Met; Conservative treatment or red flag Not met; Repeat imaging interval Met. The decisions open: REFER TO PHYSICIAN.
Approval likelihood 0.2591, LOW; expected stay not worked out, as this is not an inpatient operation.
Evidence: 7 sentences read (Background 2, Conservative treatment 1, Failed medication 1, Functional limitation 1, Prior investigation 1, Ruled out 1); disagreements flagged: 0.
james, UM Nurse Reviewer, decided REFER TO PHYSICIAN. The reason given: "Good to go."

What did you just tell me?

MEDICAL POLICY

According to the medical policy for MRI of the lumbar spine (RAD-LSM), the service requires that the member has completed at least 6 whole weeks of conservative treatment, or that a red flag symptom is reported.

› Sources

Ask about a medical policy, the review rules, an evidence guide, or a request by its reference and member number

Review queue
Request detail
Audit trail
Authorisation chat

ClearAuth AI

Carlos Mendes
Intake Coordinator

Sign out

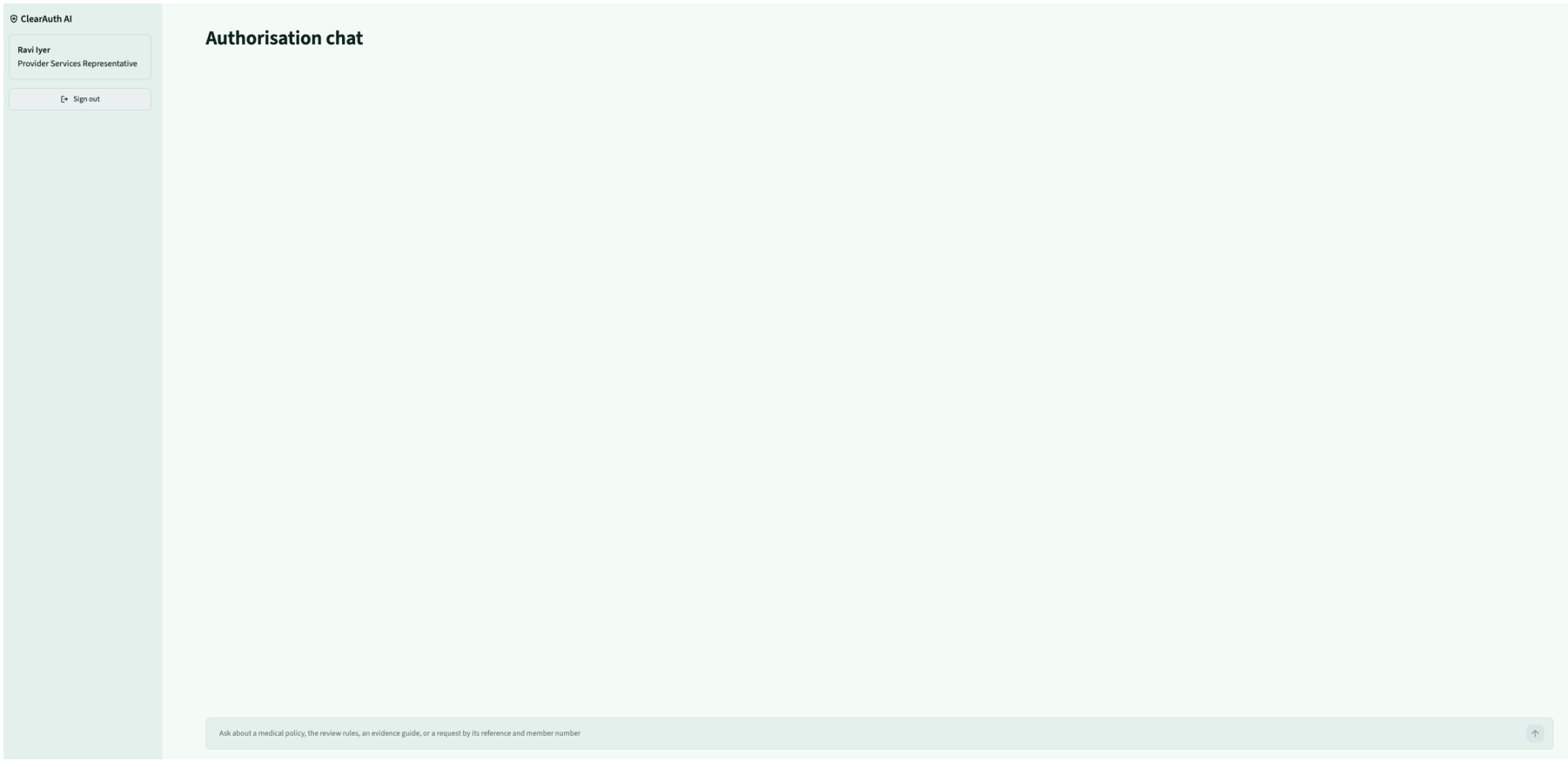
Review queue

Requests in the queue	Urgent	Every criterion met	Answers missing	A criterion not met
40	2	17	9	14

Service

All services · MRI of the lumbar spine · MRI of the cervical spine · Total knee replacement · Bariatric surgery · In-laboratory sleep study

#	Request	Service	Urgency	Docs	Met	Not met	Not doc.	Disagreements	Screening	Likelihood	Band
1	PA-561278	MRI of the lumbar spine	Urgent	1	3	0	0	0	APPROVE	0.9751	HIGH
2	PA-562203	MRI of the cervical spine	Urgent	4	2	1	0	0	REFER TO PHYSICIAN	0.3489	LOW
3	PA-561204	MRI of the lumbar spine	Standard	2	2	1	0	0	REFER TO PHYSICIAN	0.2591	LOW
4	PA-561241	MRI of the cervical spine	Standard	2	3	0	0	0	APPROVE	0.9777	HIGH
5	PA-561315	MRI of the lumbar spine	Standard	3	3	0	0	0	APPROVE	0.9844	HIGH
6	PA-561352	MRI of the lumbar spine	Standard	2	2	1	0	0	REFER TO PHYSICIAN	0.2118	LOW
7	PA-561389	MRI of the cervical spine	Standard	1	2	0	1	0	REQUEST MORE INFORMATION	0.6959	MEDIUM
8	PA-561426	Total knee replacement	Standard	3	5	0	0	0	APPROVE	0.9972	HIGH
9	PA-561463	Total knee replacement	Standard	3	4	1	0	0	REFER TO PHYSICIAN	0.5046	MEDIUM
10	PA-561500	Total knee replacement	Standard	2	4	0	1	0	REQUEST MORE INFORMATION	0.6531	MEDIUM
11	PA-561537	Bariatric surgery	Standard	3	3	1	0	0	REFER TO PHYSICIAN	0.2967	LOW
12	PA-561574	Bariatric surgery	Standard	4	4	0	0	0	APPROVE	0.9829	HIGH
13	PA-561611	Bariatric surgery	Standard	2	3	0	1	0	REQUEST MORE INFORMATION	0.7463	MEDIUM
14	PA-561648	In-laboratory sleep study	Standard	2	3	0	0	0	APPROVE	0.9676	HIGH
15	PA-561685	In-laboratory sleep study	Standard	2	2	1	0	0	REFER TO PHYSICIAN	0.2818	LOW



IMPLEMENTATION STEPS

Every command is run from the Project folder.

STEP 0. INSTALL AND CONFIGURE

```
cd Project
pip install -r requirements.txt
```

STEP 1. CREATE THE DATABASE

```
python3 run_database_setup.py
```

STEP 2. CLEAN THE RAW DATA

```
python3 run_data_pipeline.py
```

STEP 3. TRAIN THE MODELS

```
python3 run_model_training.py
```

STEP 4. BUILD THE KNOWLEDGE BASE

```
python3 run_knowledge_indexing.py
```

STEP 5. START THE REST SERVICE

Leave this running and use a second terminal for anything else.

```
python3 run_api_service.py
```

STEP 6. START THE DASHBOARD

In a third terminal, with the service still running:

```
cd Project
python3 -m streamlit run run_dashboard.py
```